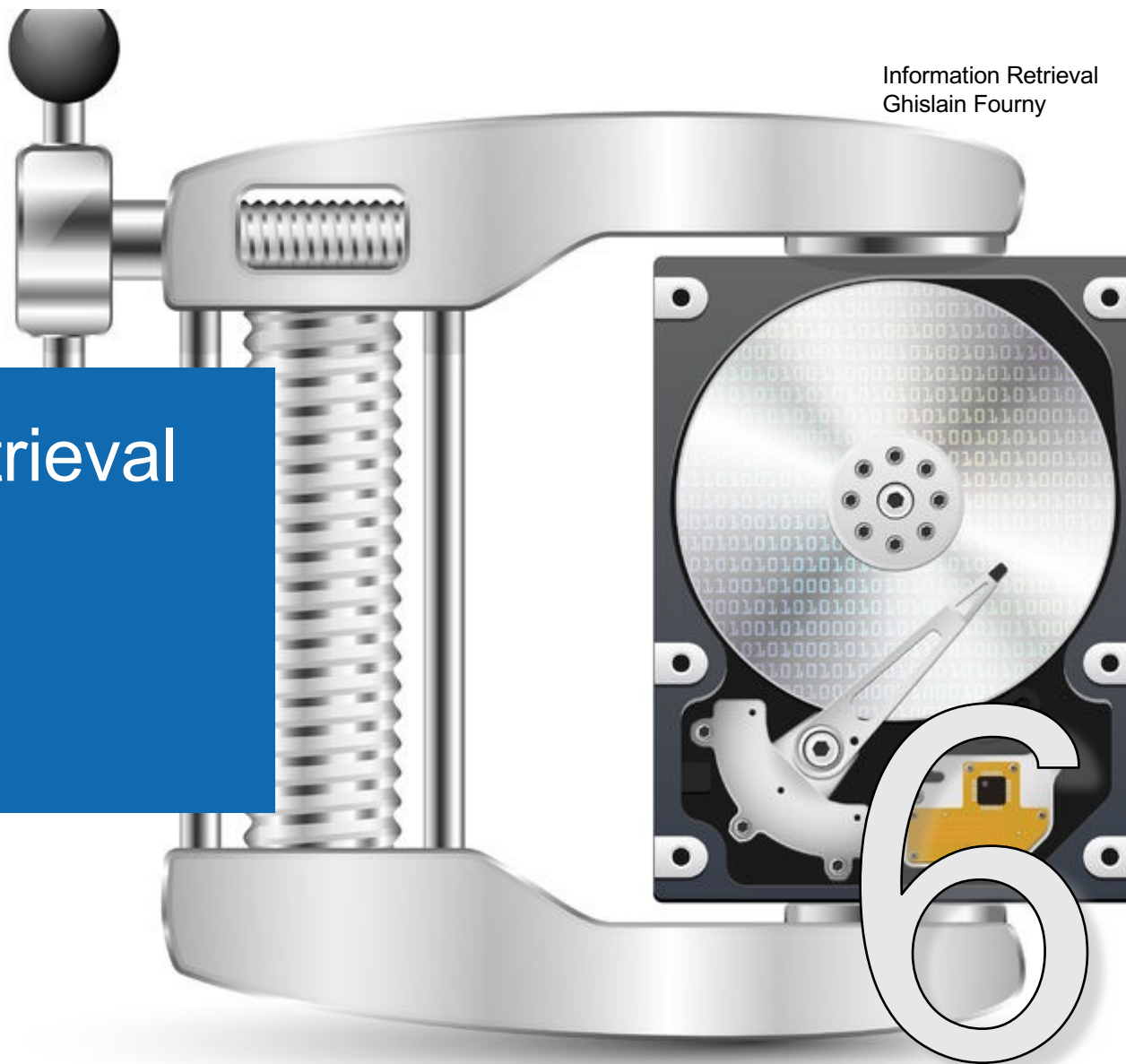
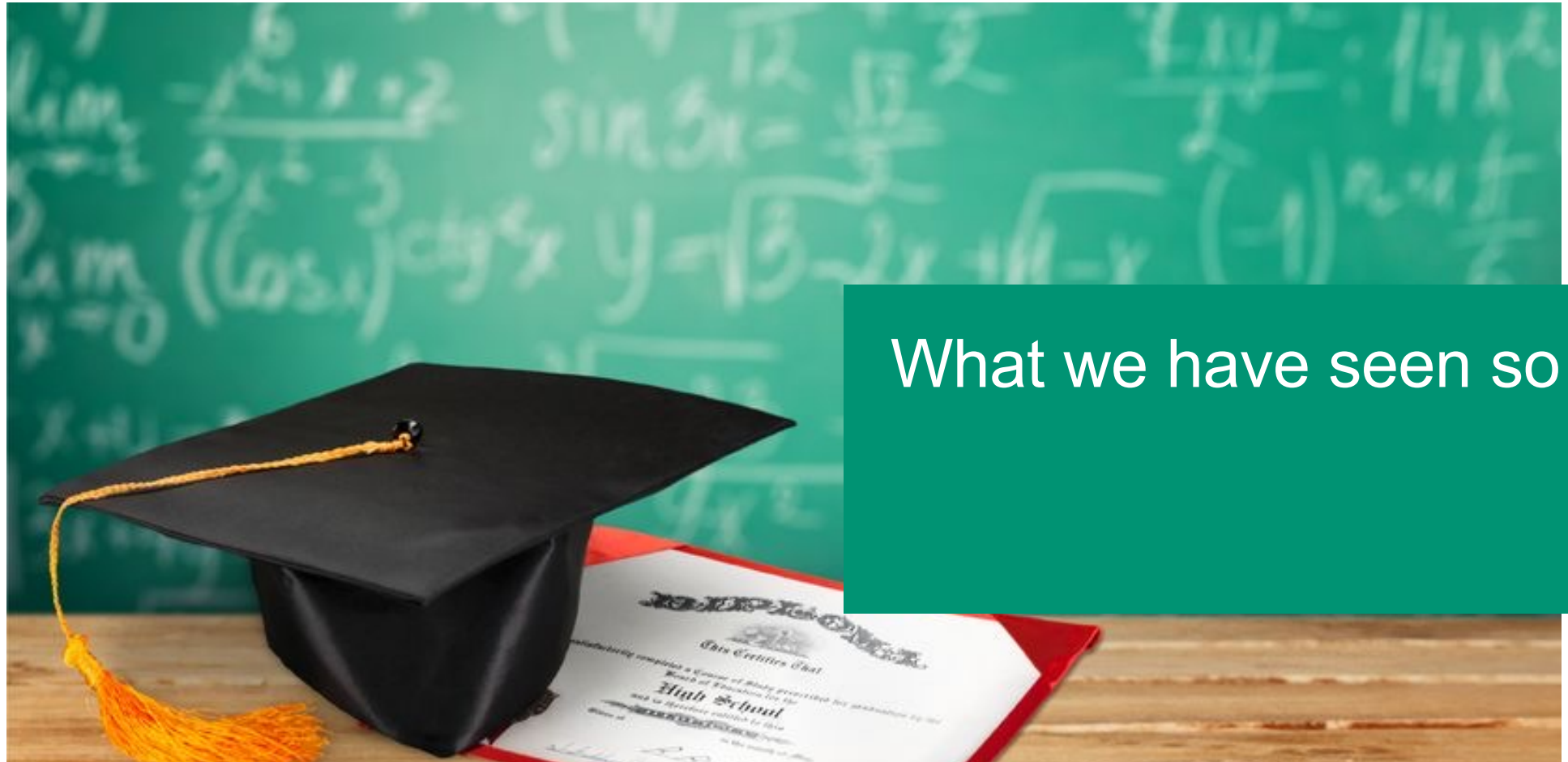


Information Retrieval

6. Index compression

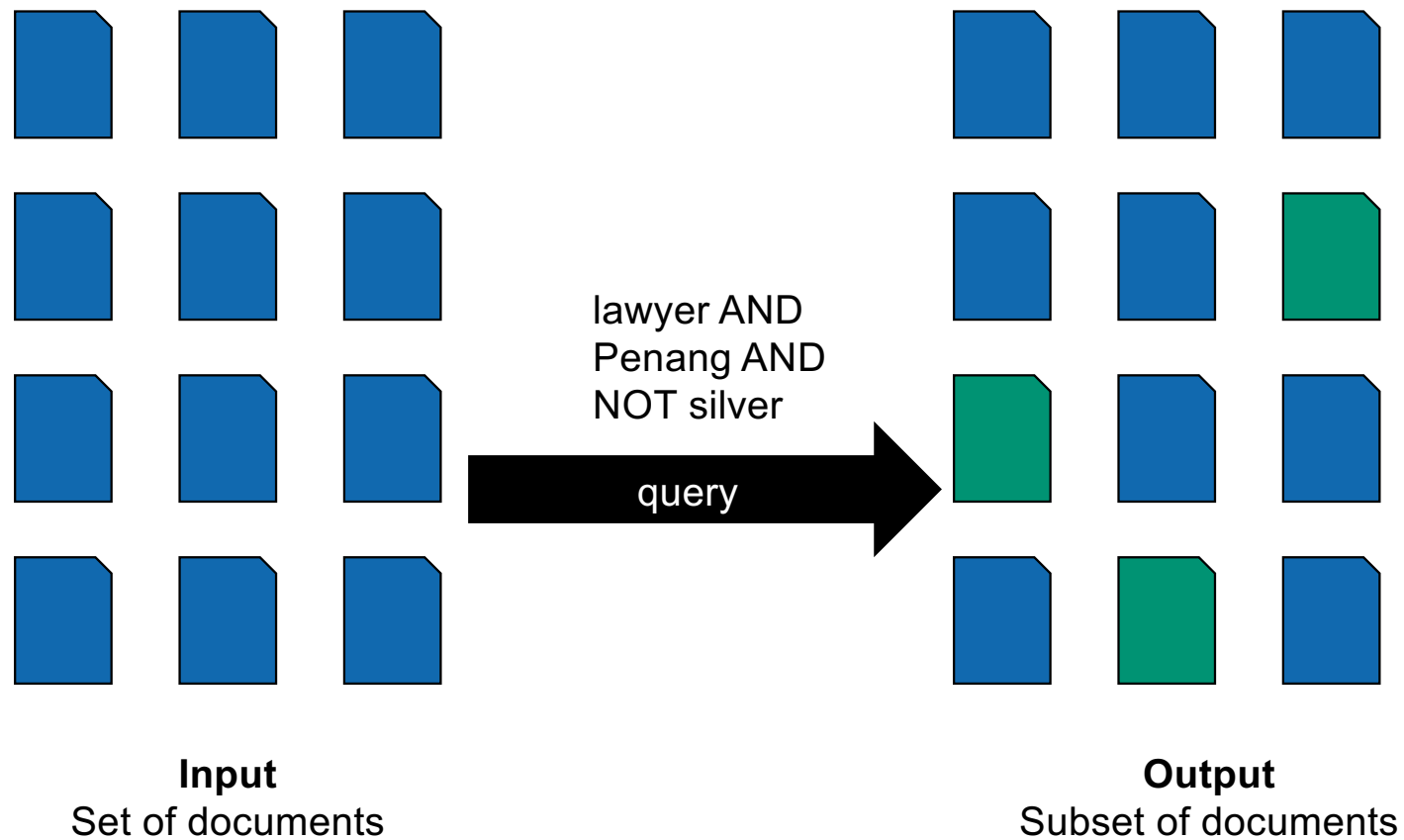
Ghislain Fourny
Spring 2022





What we have seen so far

Boolean retrieval

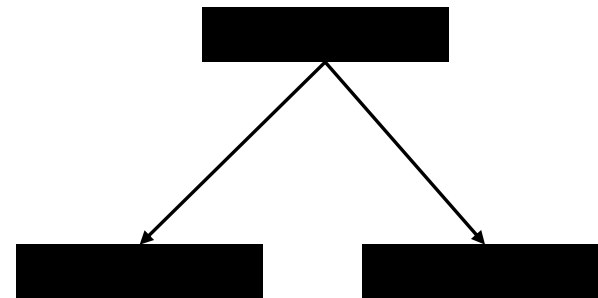


Standard inverted index

ETH	→	6	1	2	3	5	6	8
Zürich	→	5	3	4	7	8	9	
computer	→	5	1	2	4	5	7	
data	→	5	1	3	5	8	9	
CPU	→	4	2	3	4	7		
information	→	6	1	2	4	5	8	9
retrieval	→	4	3	5	7	8		

Search structures

Hash tables



Trees (B, B+)

Additional features

comput*

Wildcards

cmputer

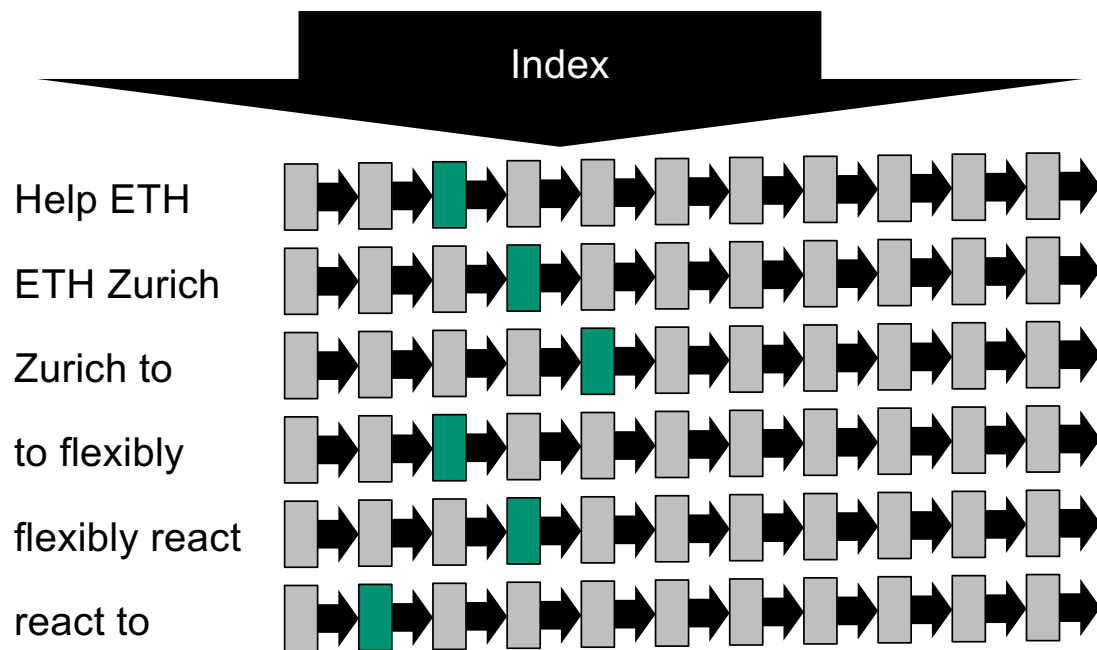
Spell correction

"Pfäffikon SZ"

Phrase search

Bi-word indices (Phrase search feature)

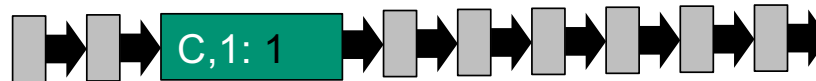
Help ETH Zurich to flexibly react to new challenges and to set new accents in the future.



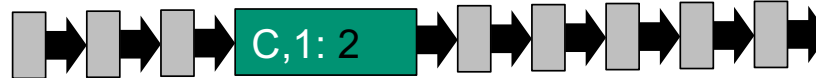
Positional index (phrase search feature)

"ETH Zurich"

Help



ETH



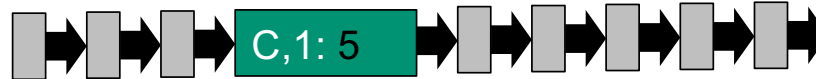
Zurich



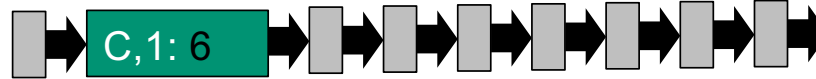
to



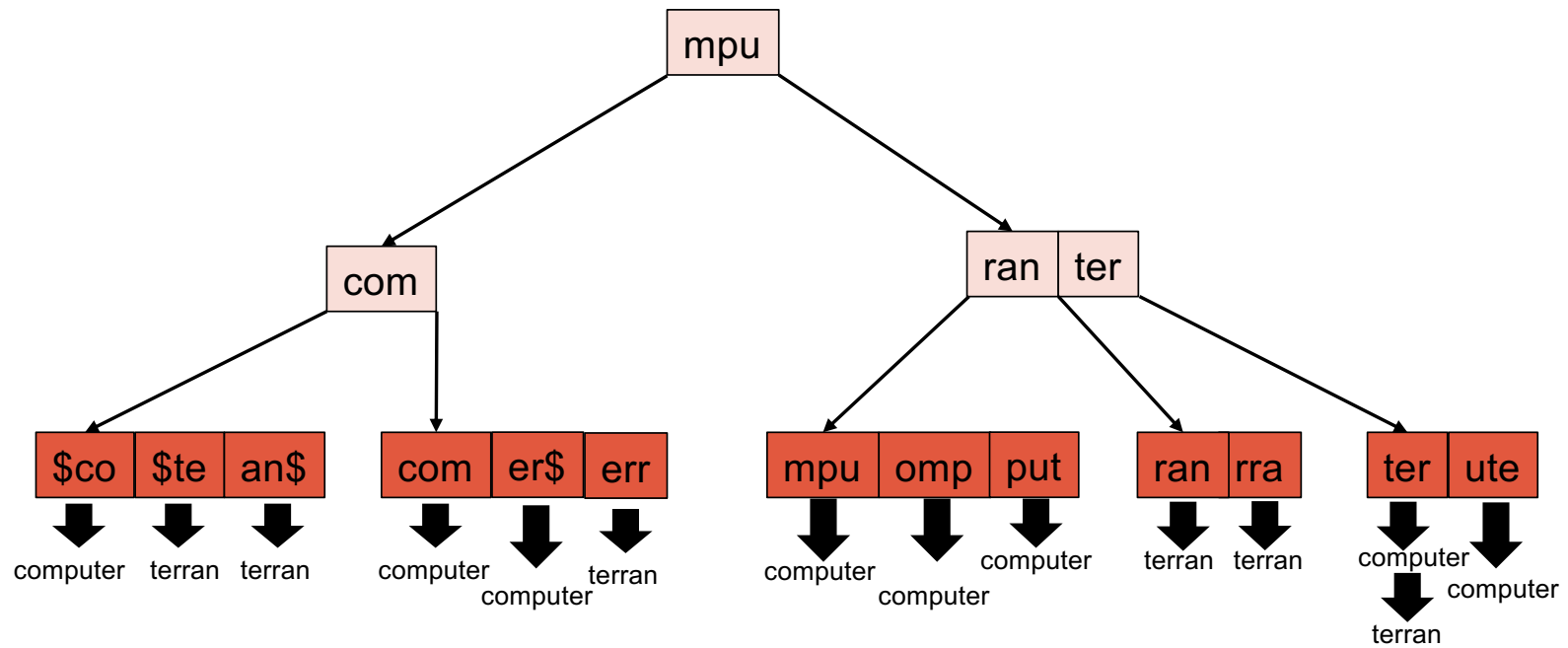
flexibly



react



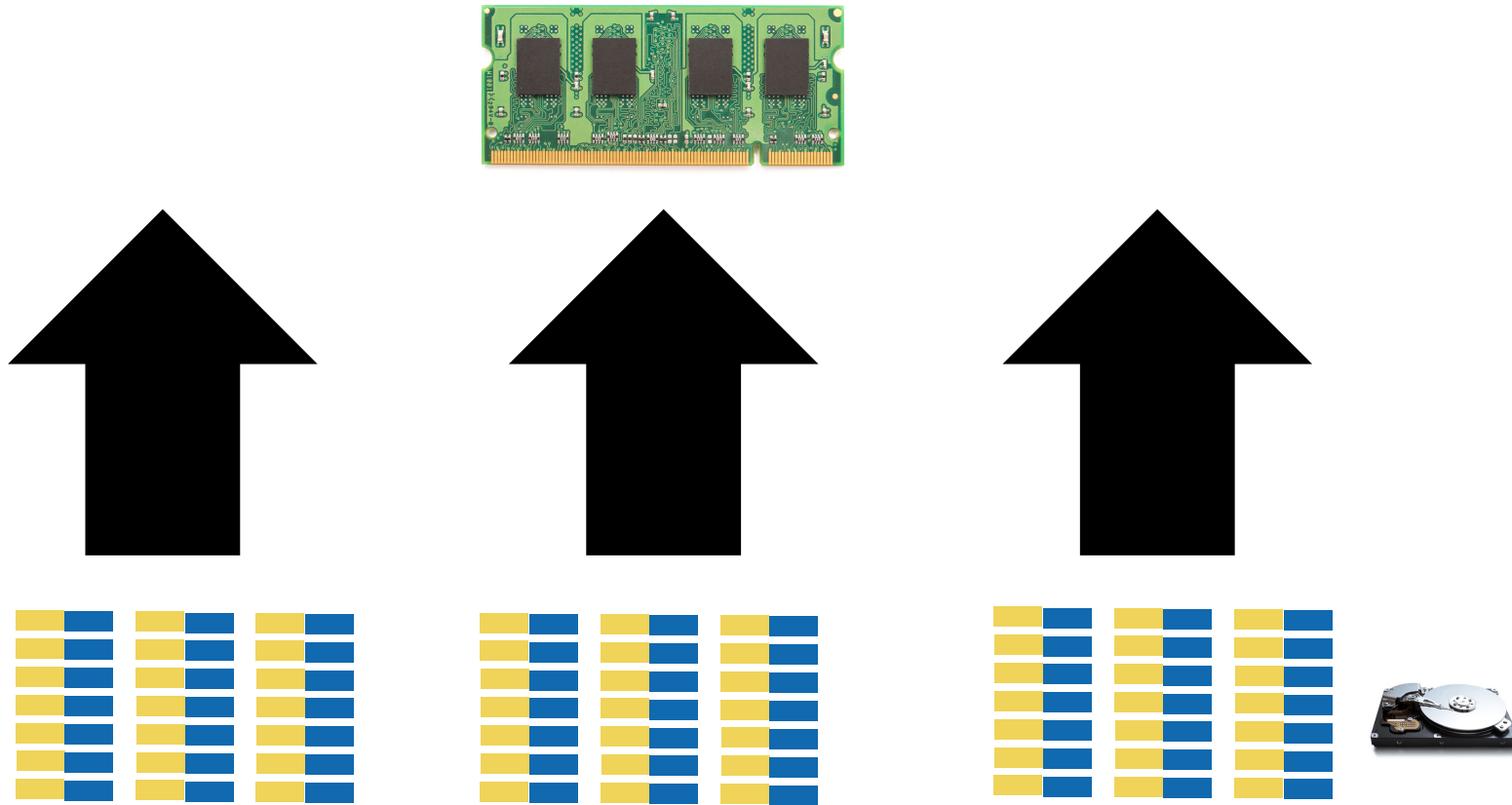
Trigram index (wildcard, spell correction)



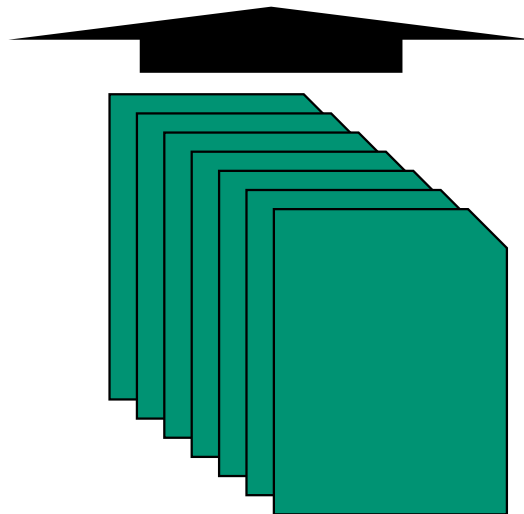
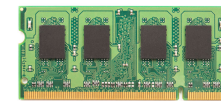
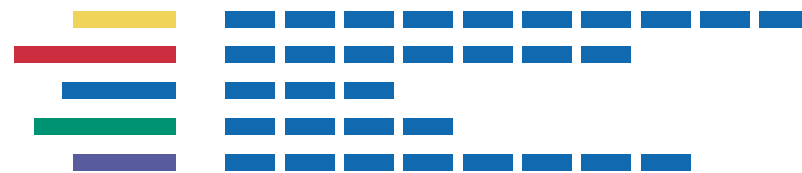
TermIDs

t1	1	t1	2	t1	3	...
t2	3	t2	4	t2	7	...
t3	1	t3	2	t3	4	...
t4	1	t4	3	t4	5	...
t5	2	t5	3	t5	4	...
t6	1	t6	2	t6	4	...
t7	3	t7	5	t7	7	...

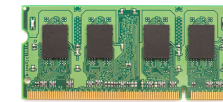
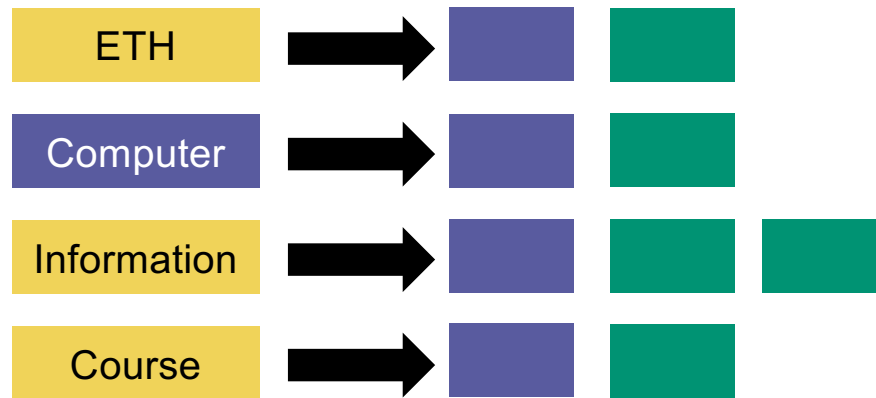
Blocked Sort-Based Indexing



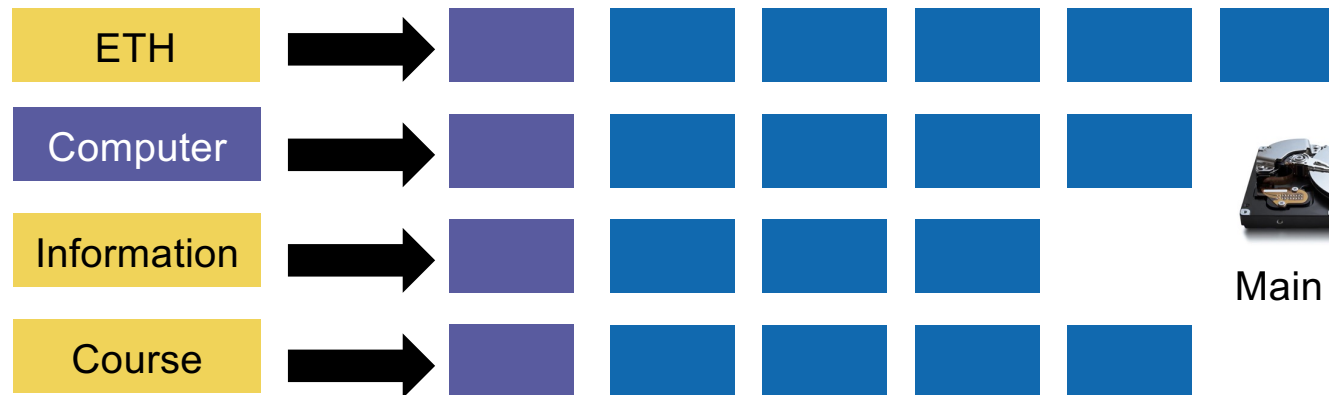
Single-Pass In-Memory Indexing



Auxiliary Index



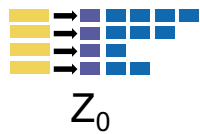
Auxiliary index



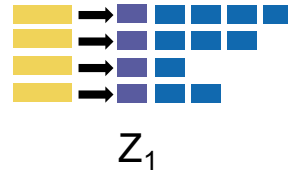
Main index

Logarithmic Merging

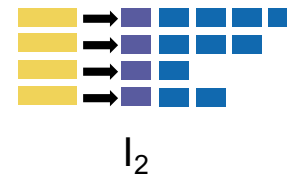
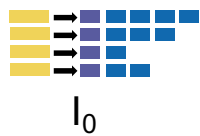
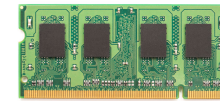
n postings

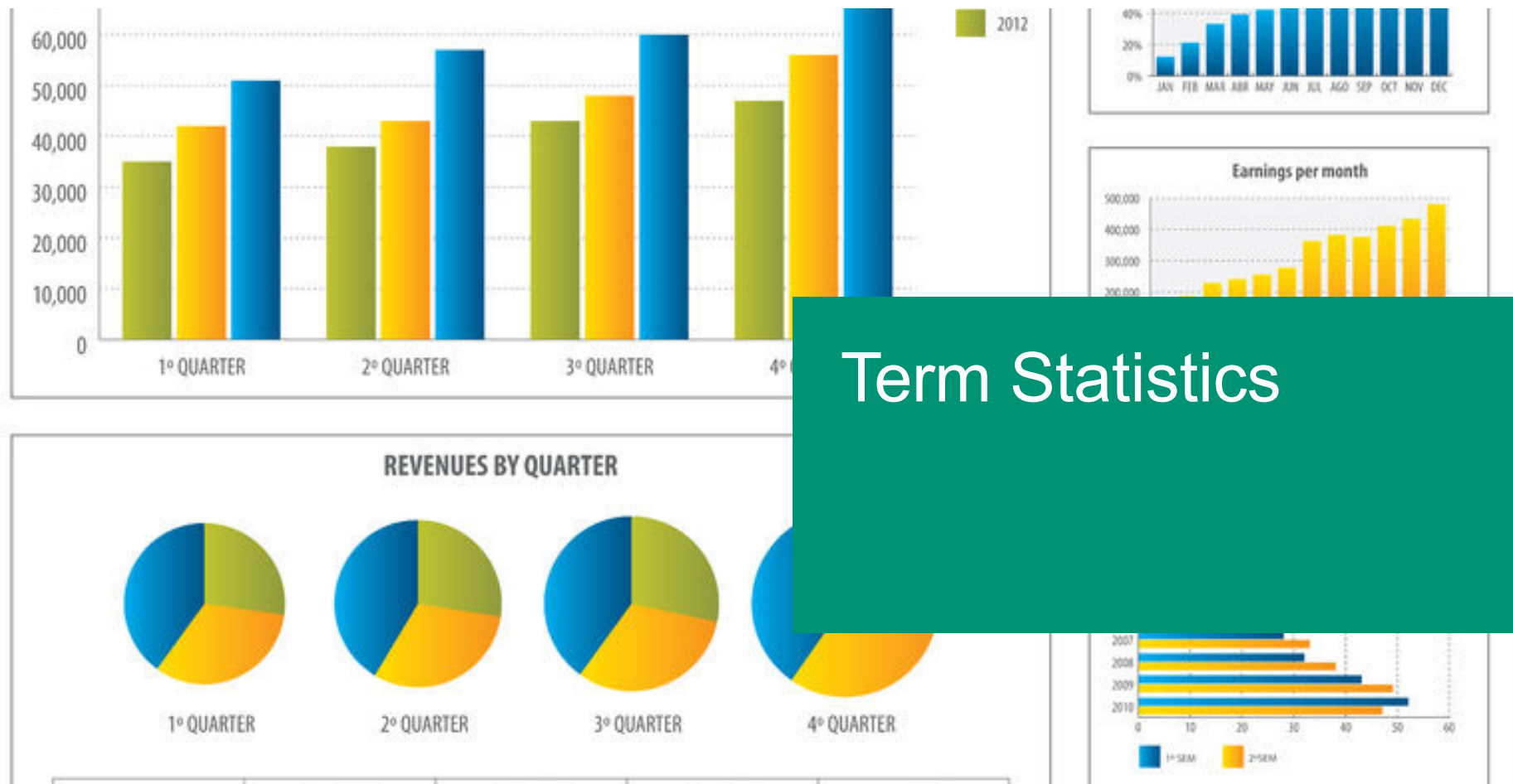


$2n$ postings



$4n$ postings





Term Statistics

Number of terms



Number of terms



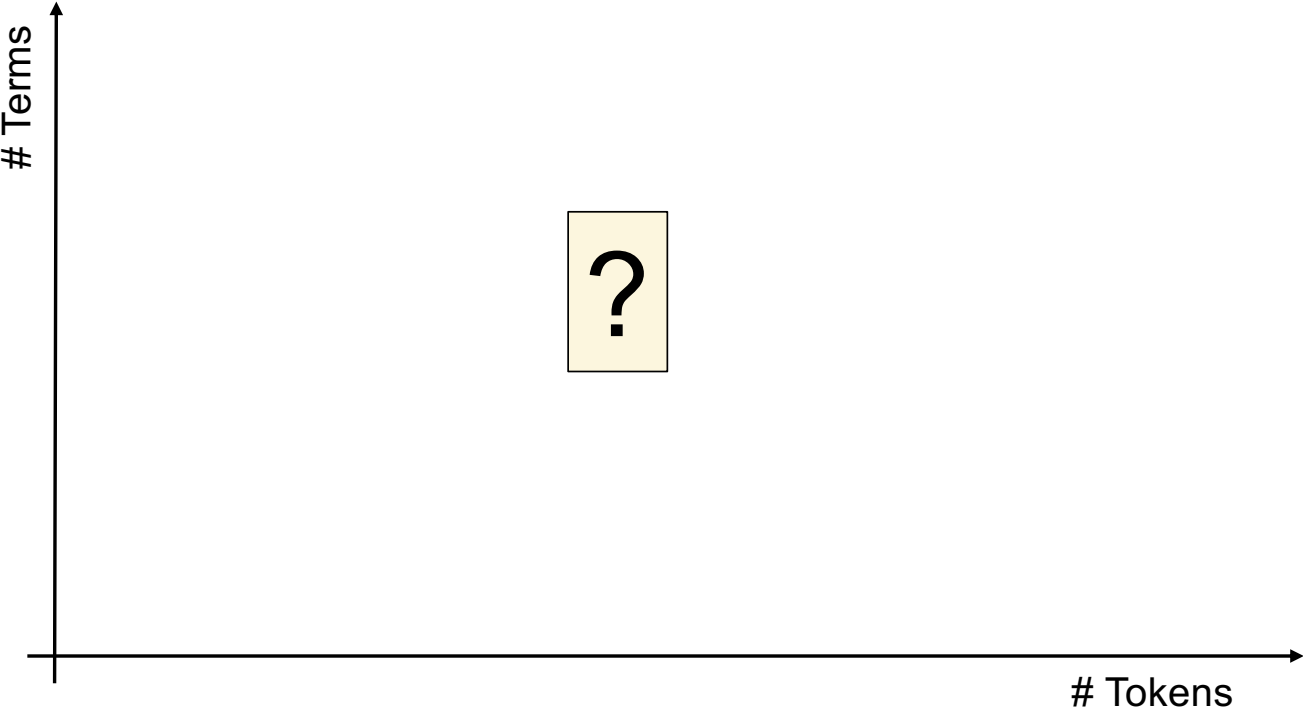
Notations used in the book

N: number of **documents**

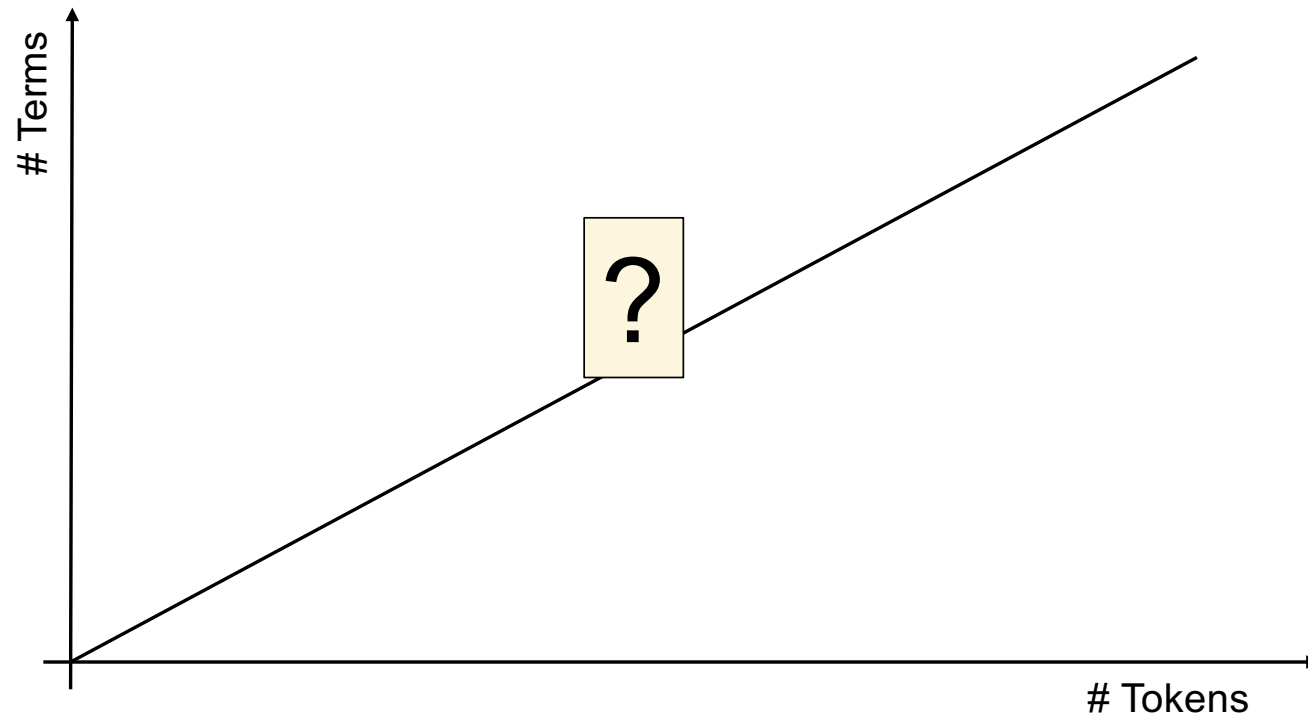
T: number of **tokens (positional postings)**

M: number of **terms (or types if stemming/lemmatization)**

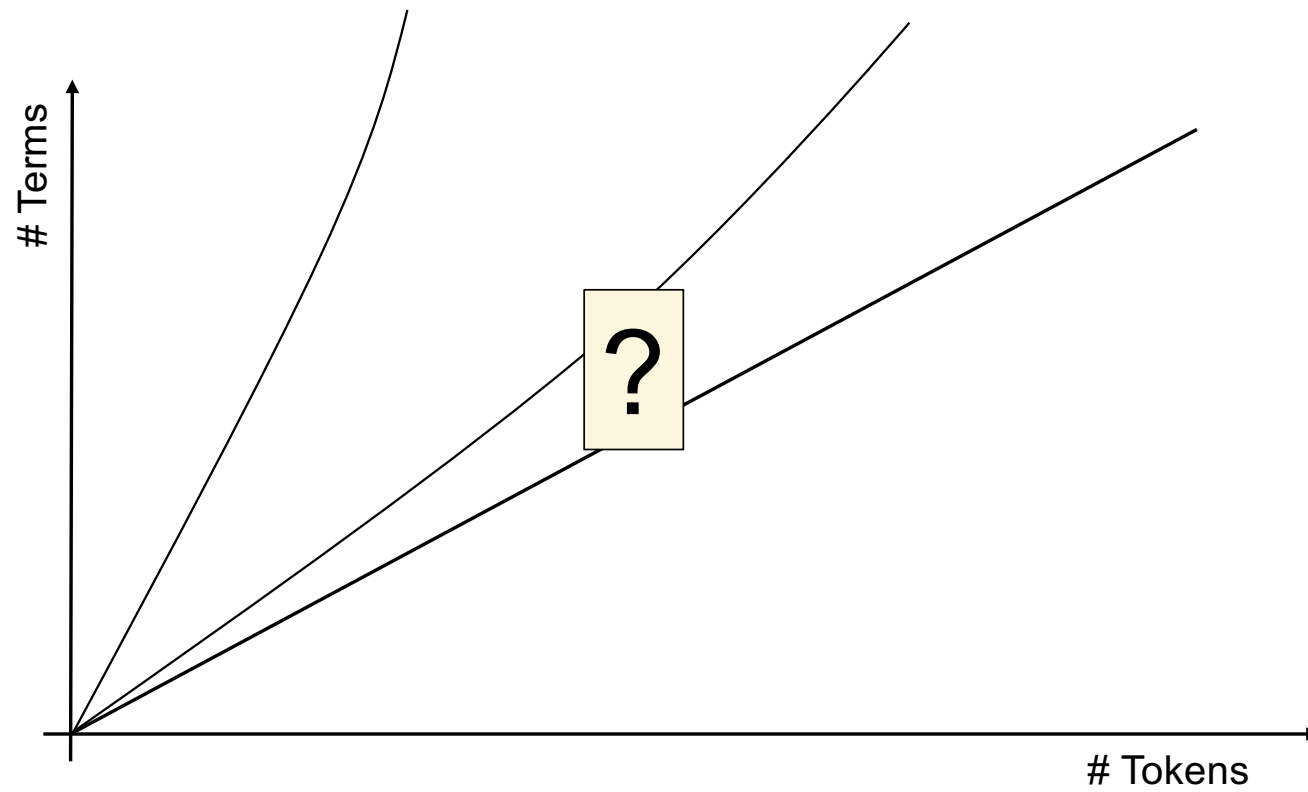
Number of terms



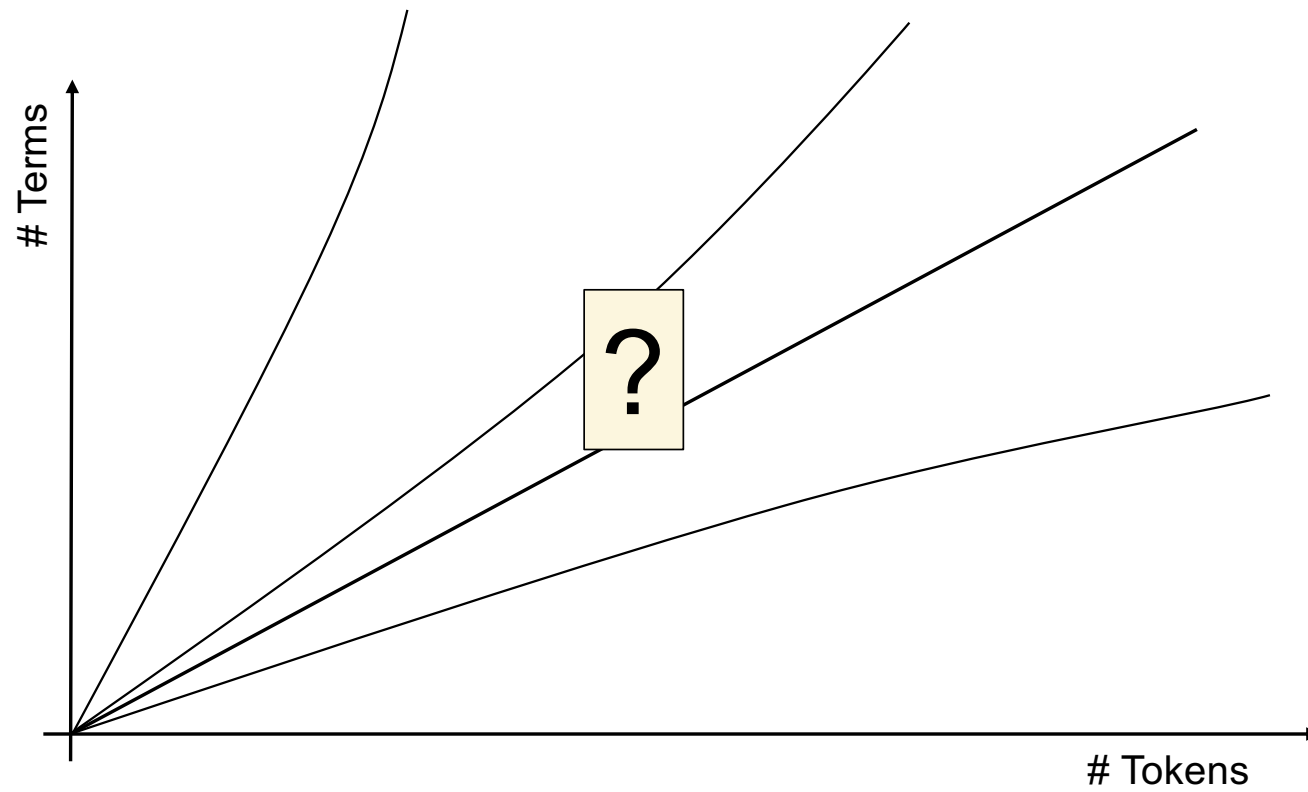
Number of terms



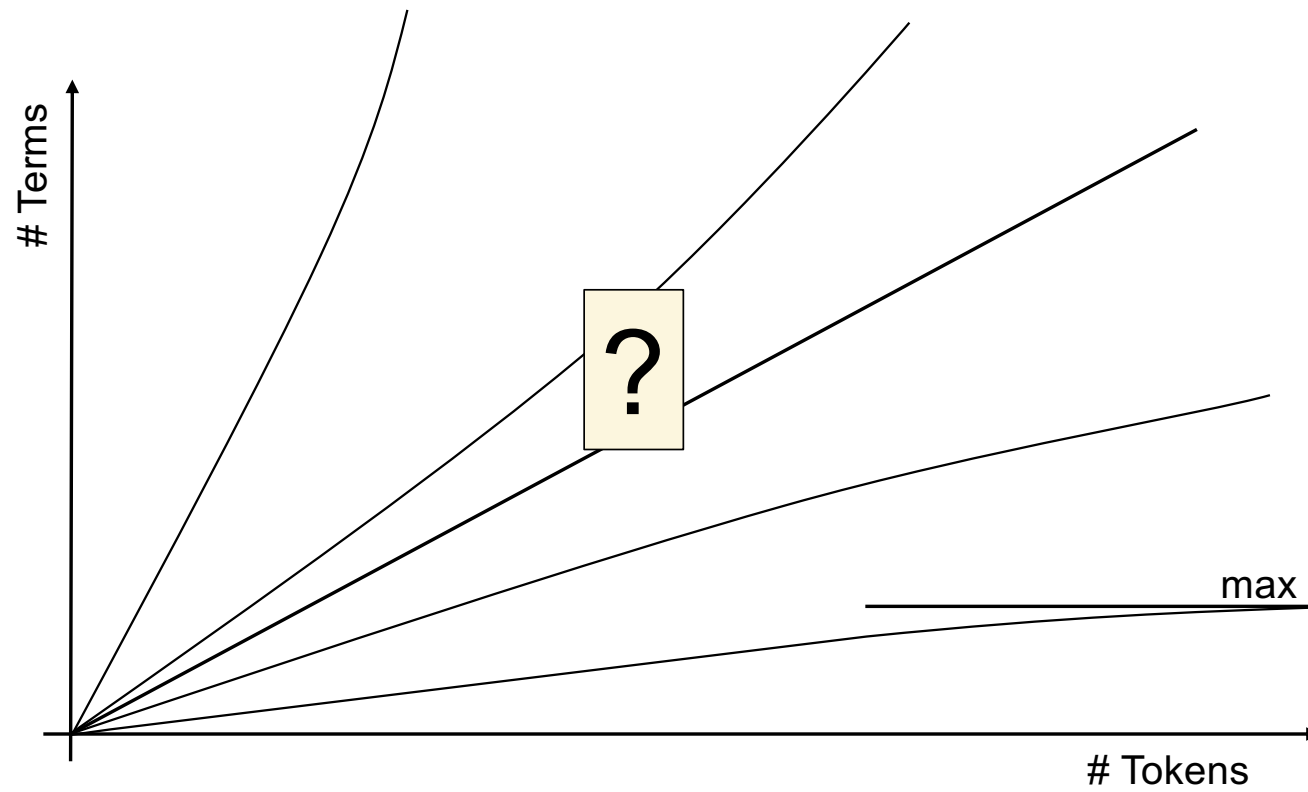
Number of terms



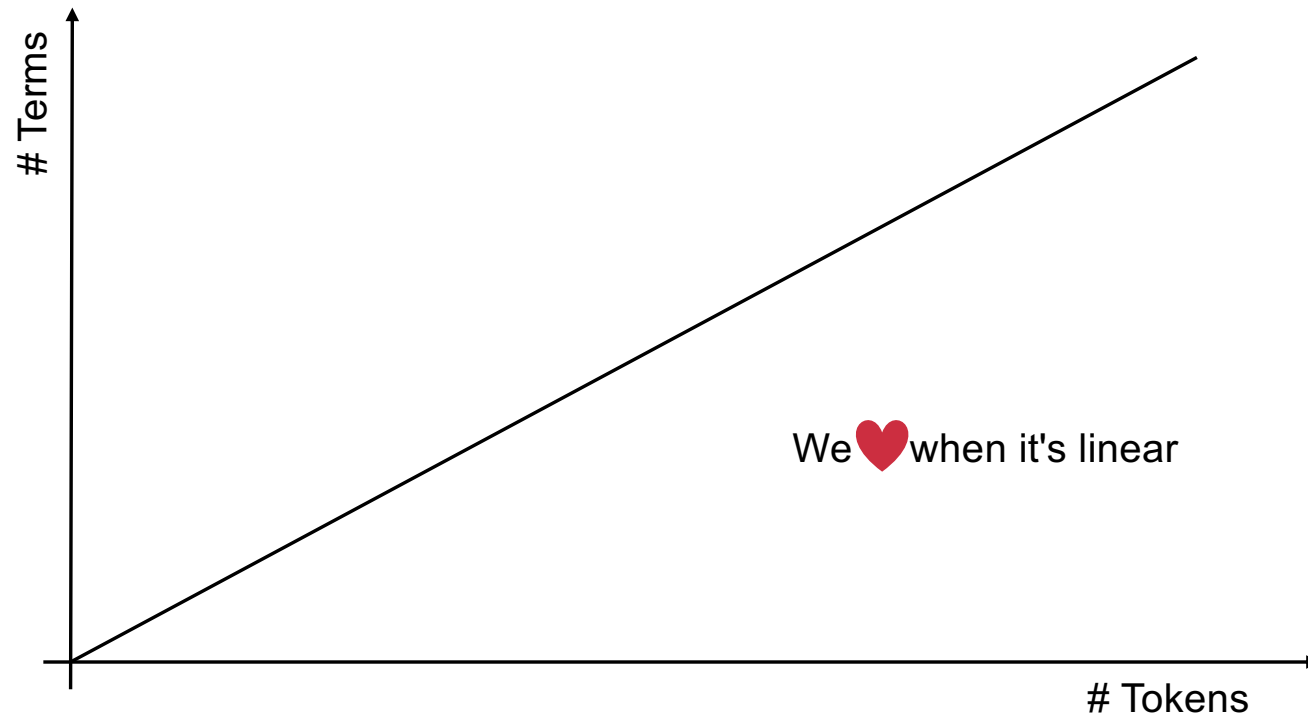
Number of terms



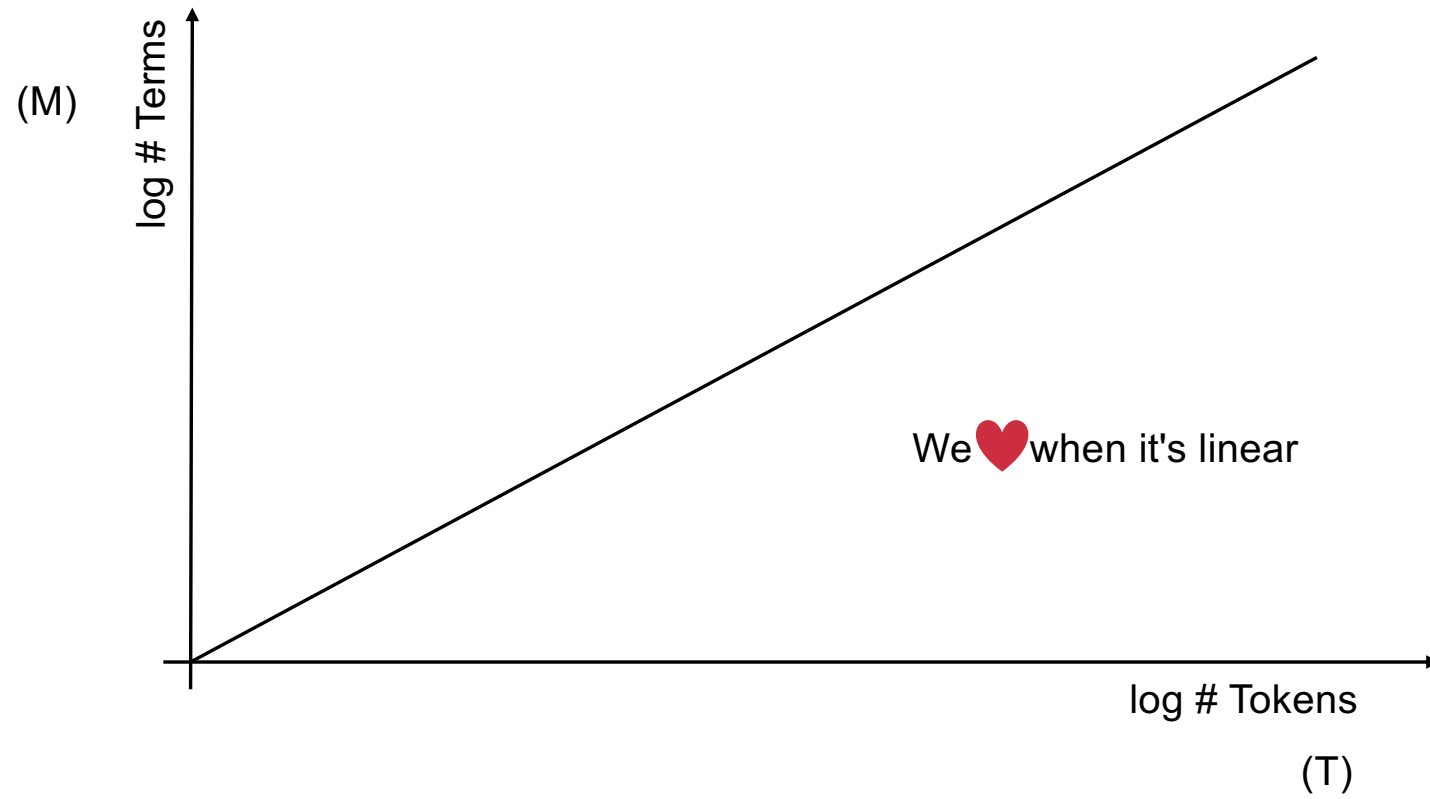
Number of terms



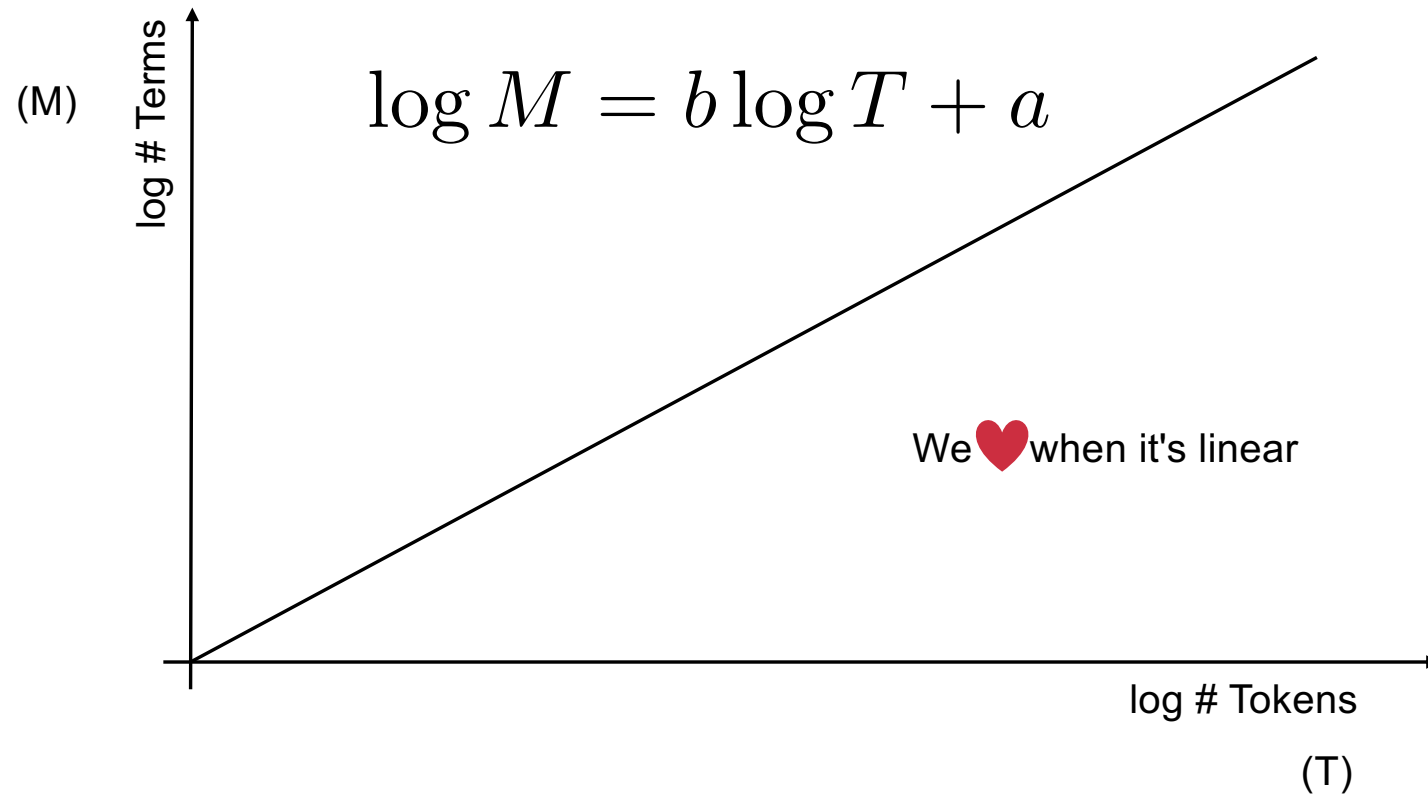
Number of terms



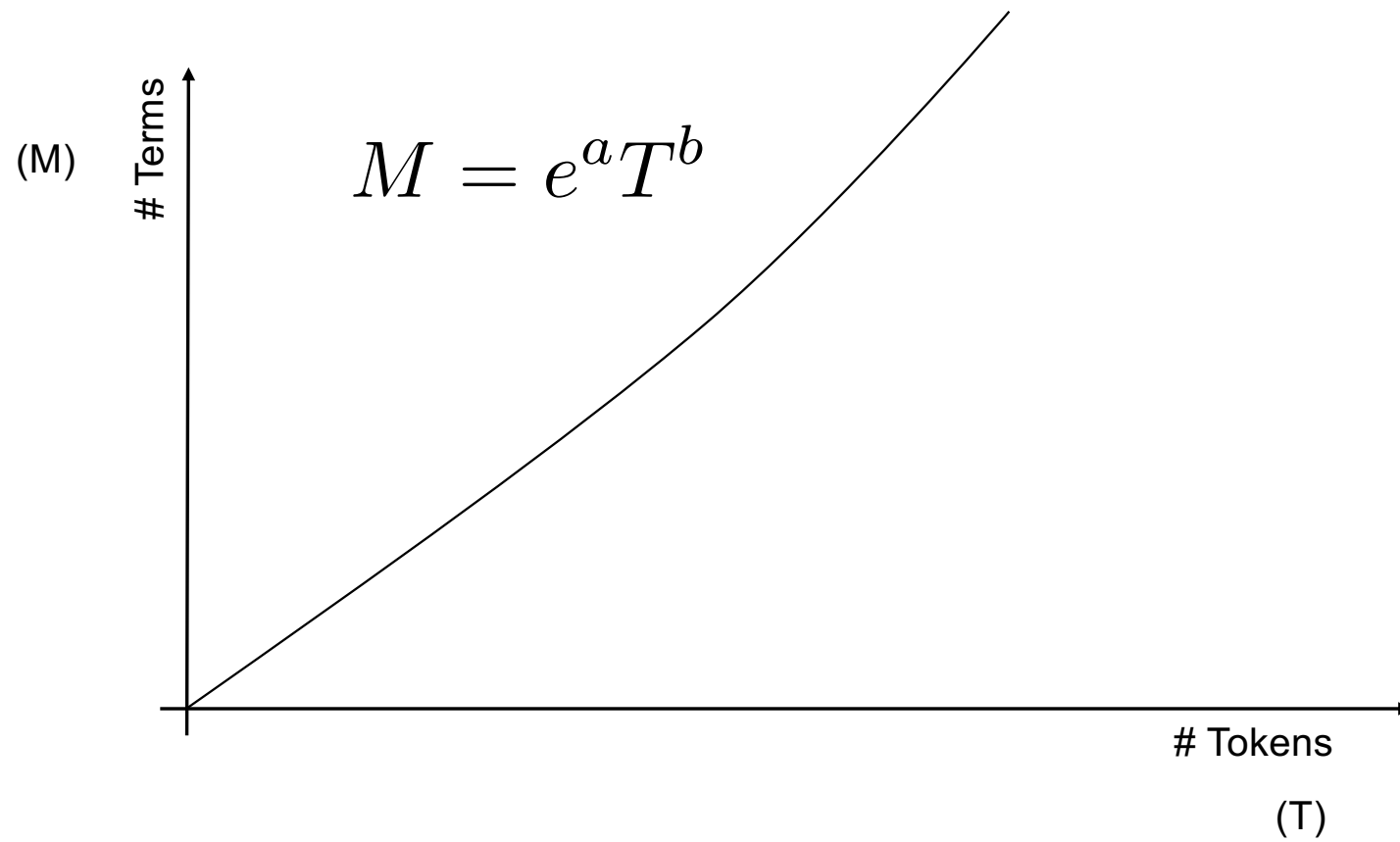
Log-log scale



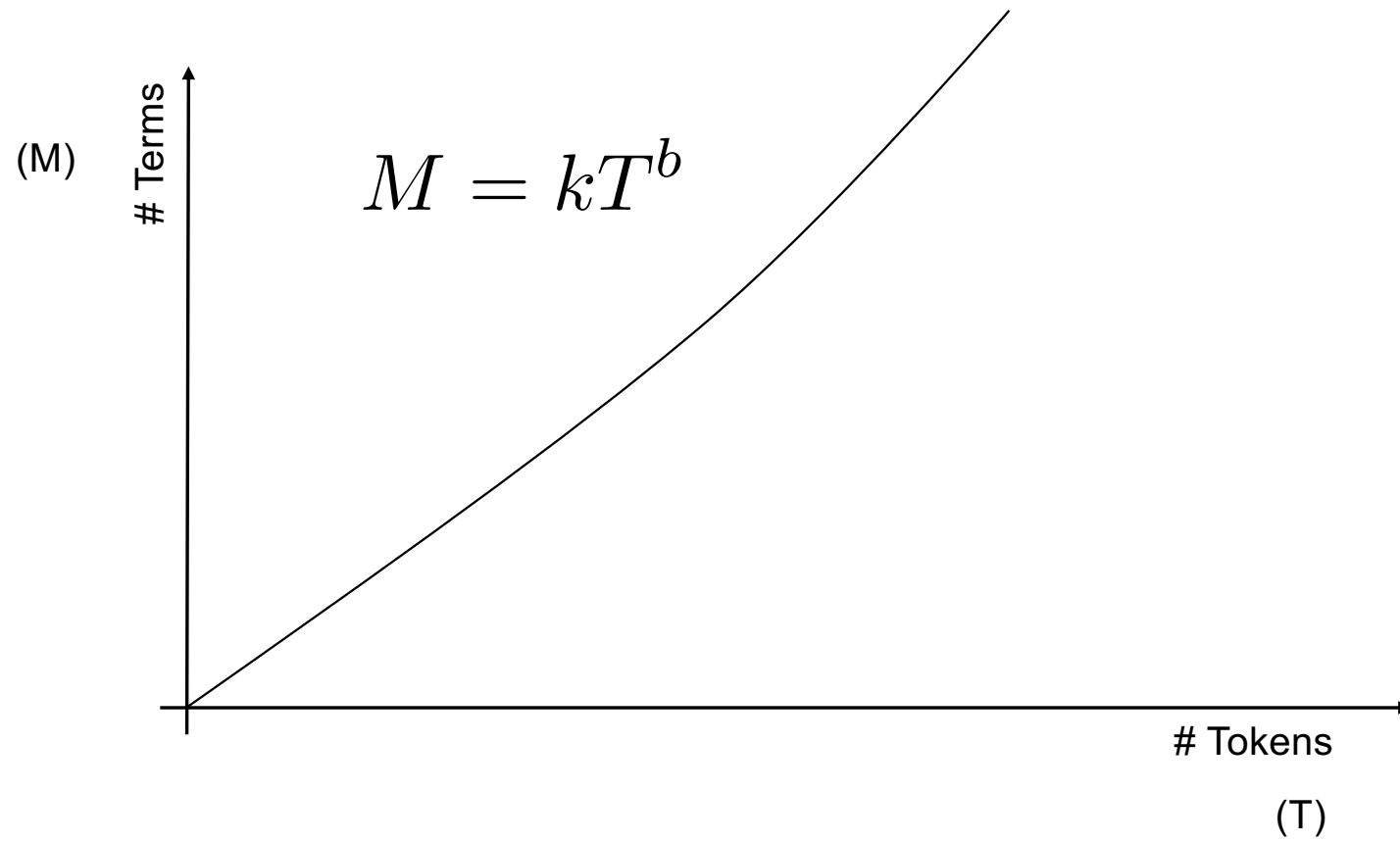
Log-log scale



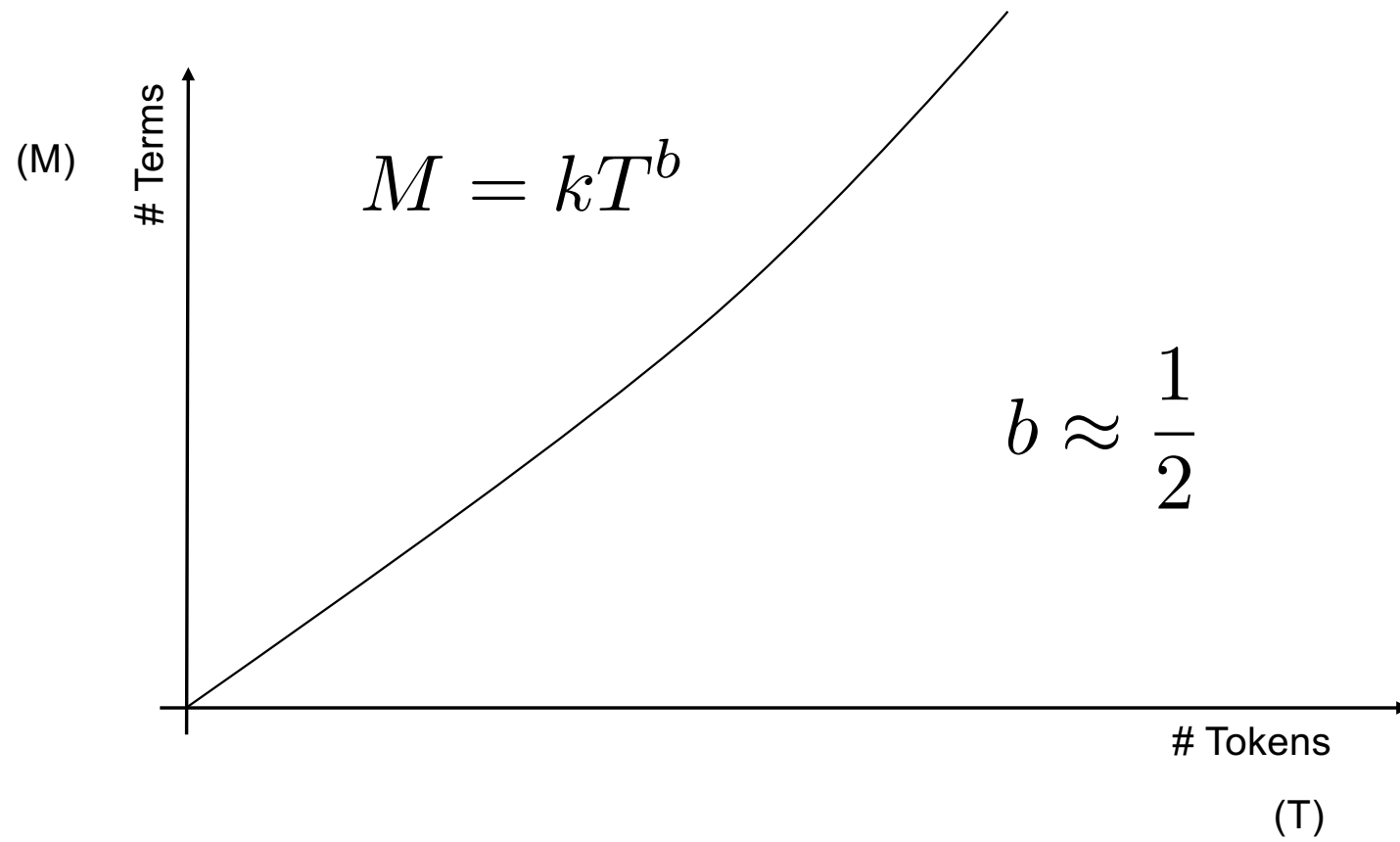
"Exponential" growth



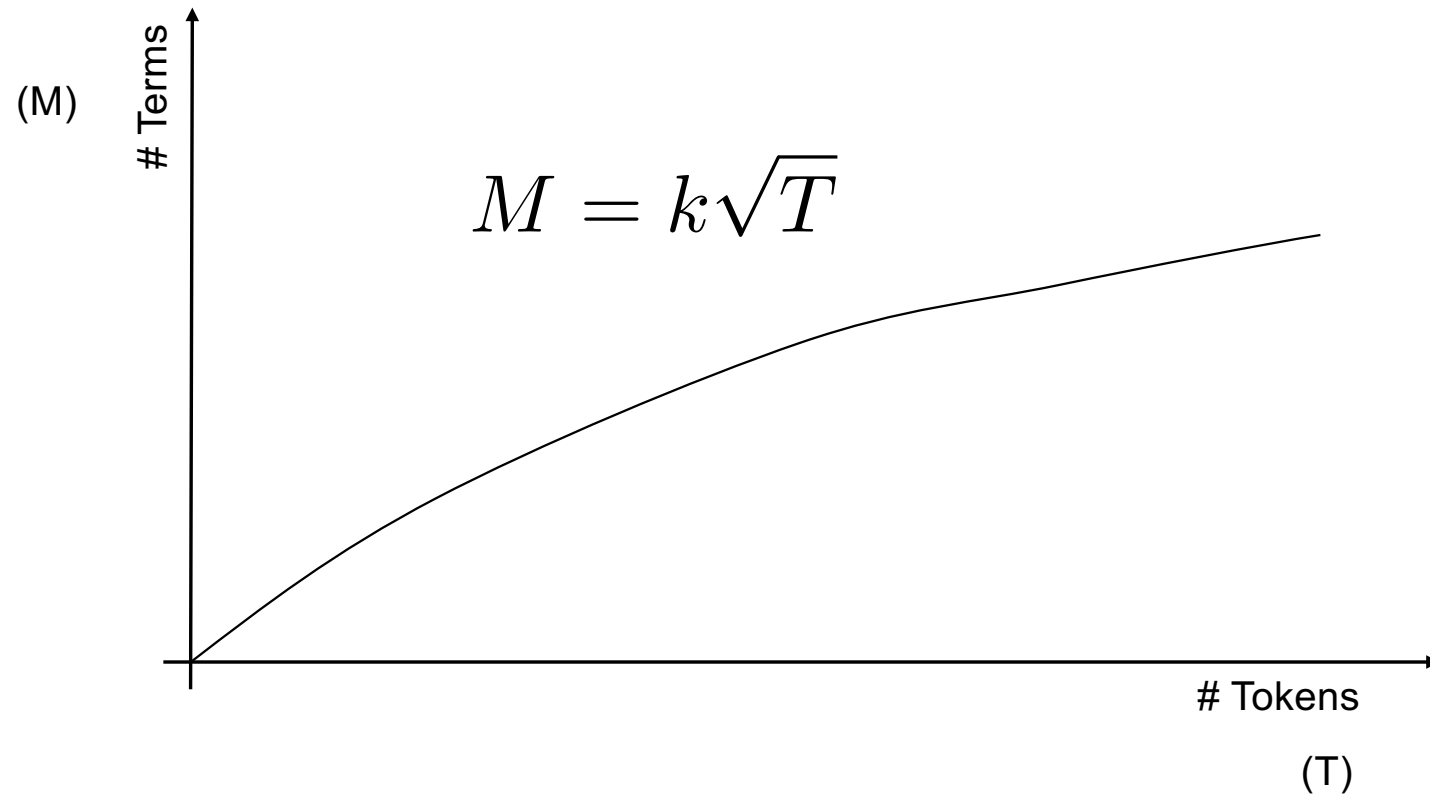
Heaps' law



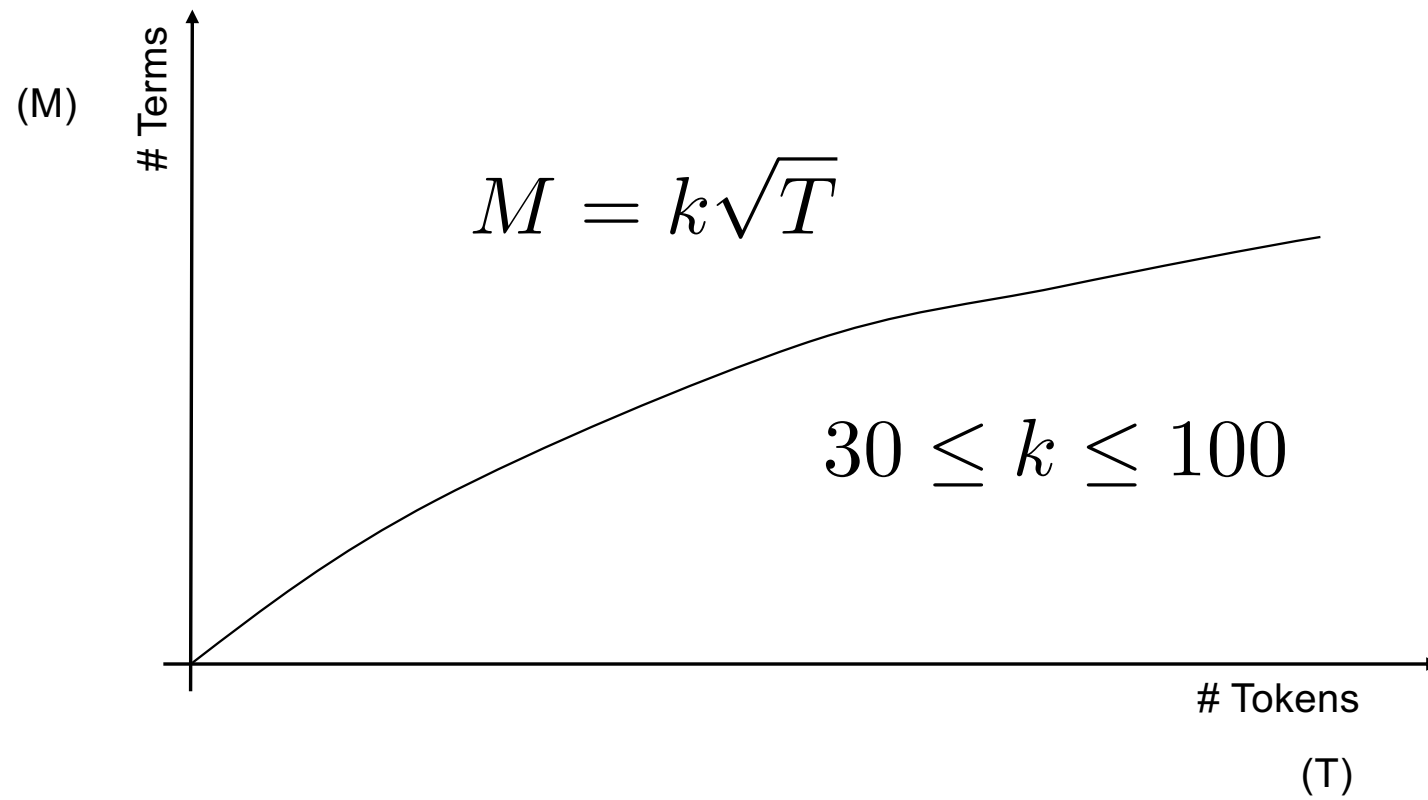
In practice



In practice



In practice



Distribution of terms

Distribution of terms

the: 56,271,872

were: 3,323,884

nearer: 51,456

moderate: 19,245

champion: 9400

stocks: 6,537

parallelogram: 503

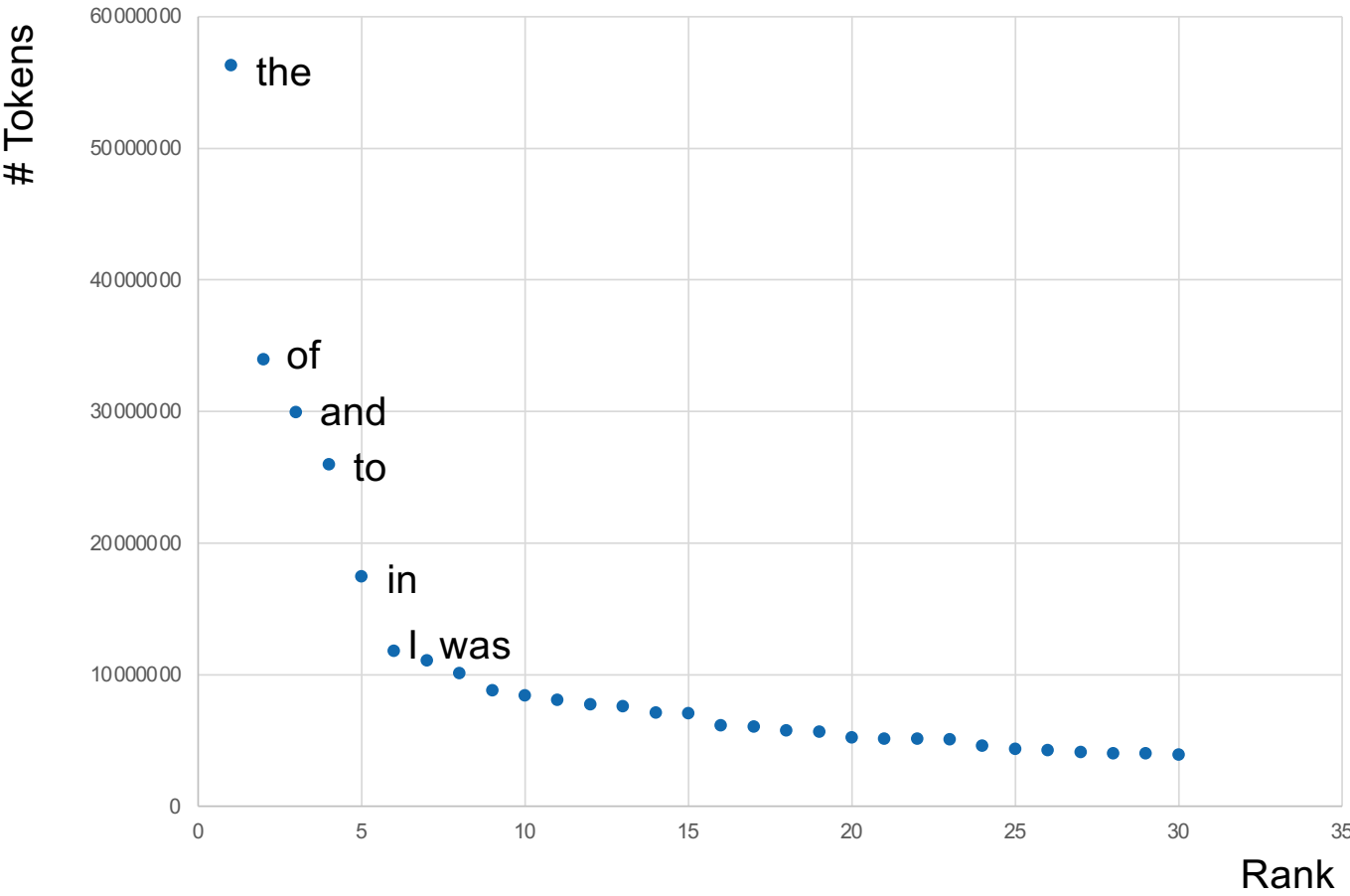
pachyderm: 79

capacitance: 45

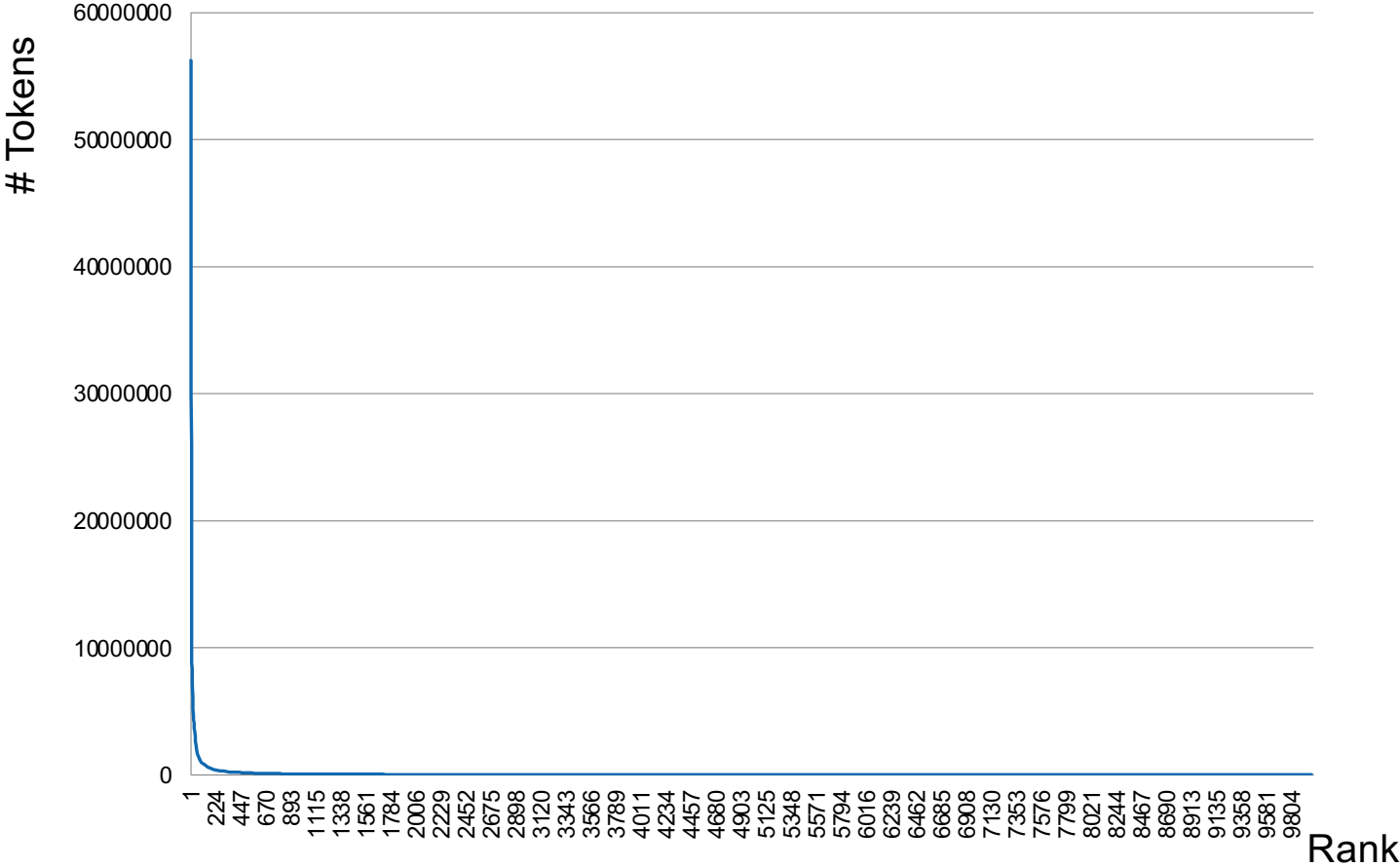
germanium: 12

sesquipedal: 7

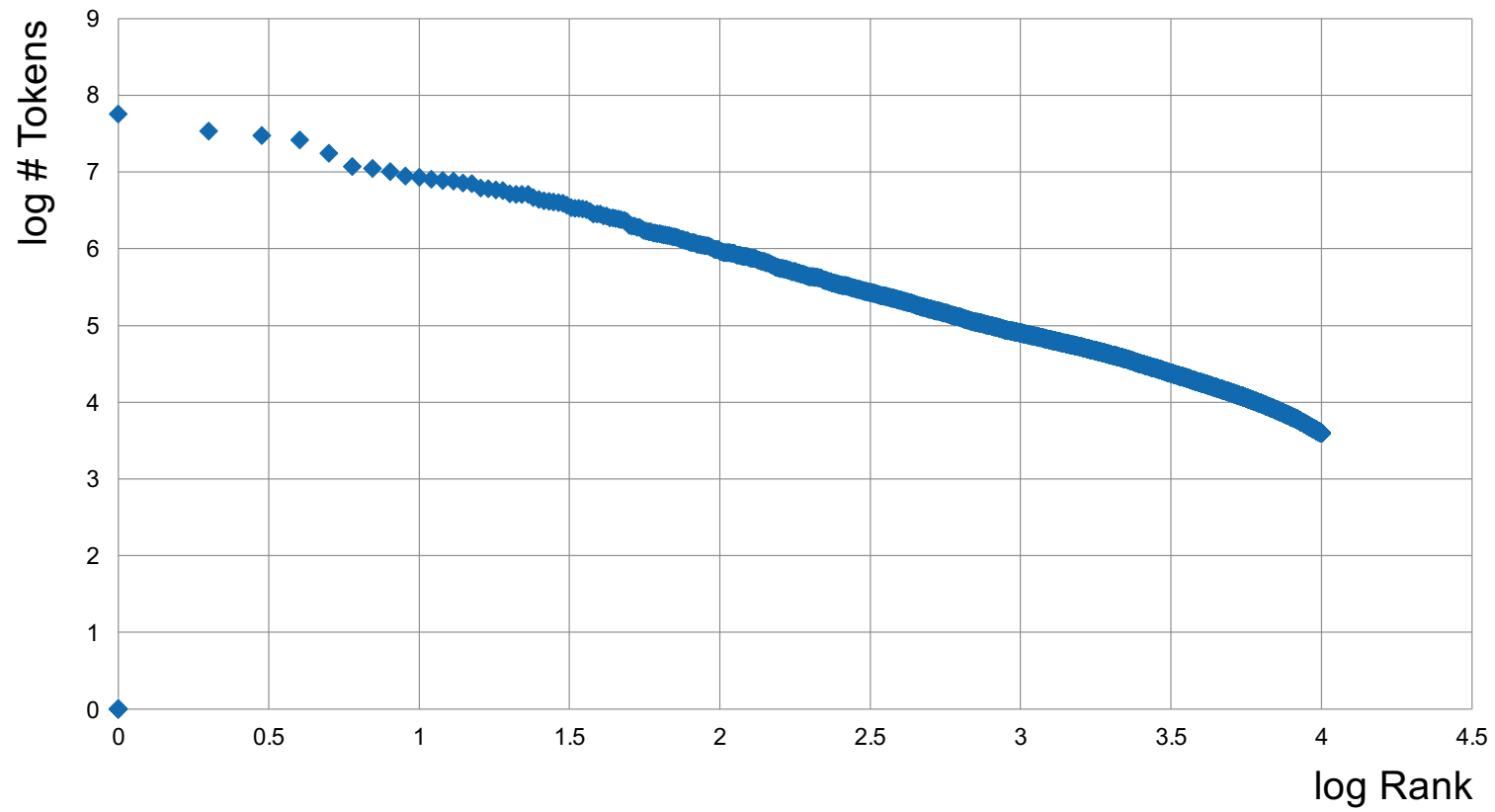
Distribution of terms



Distribution of terms



log-log scale

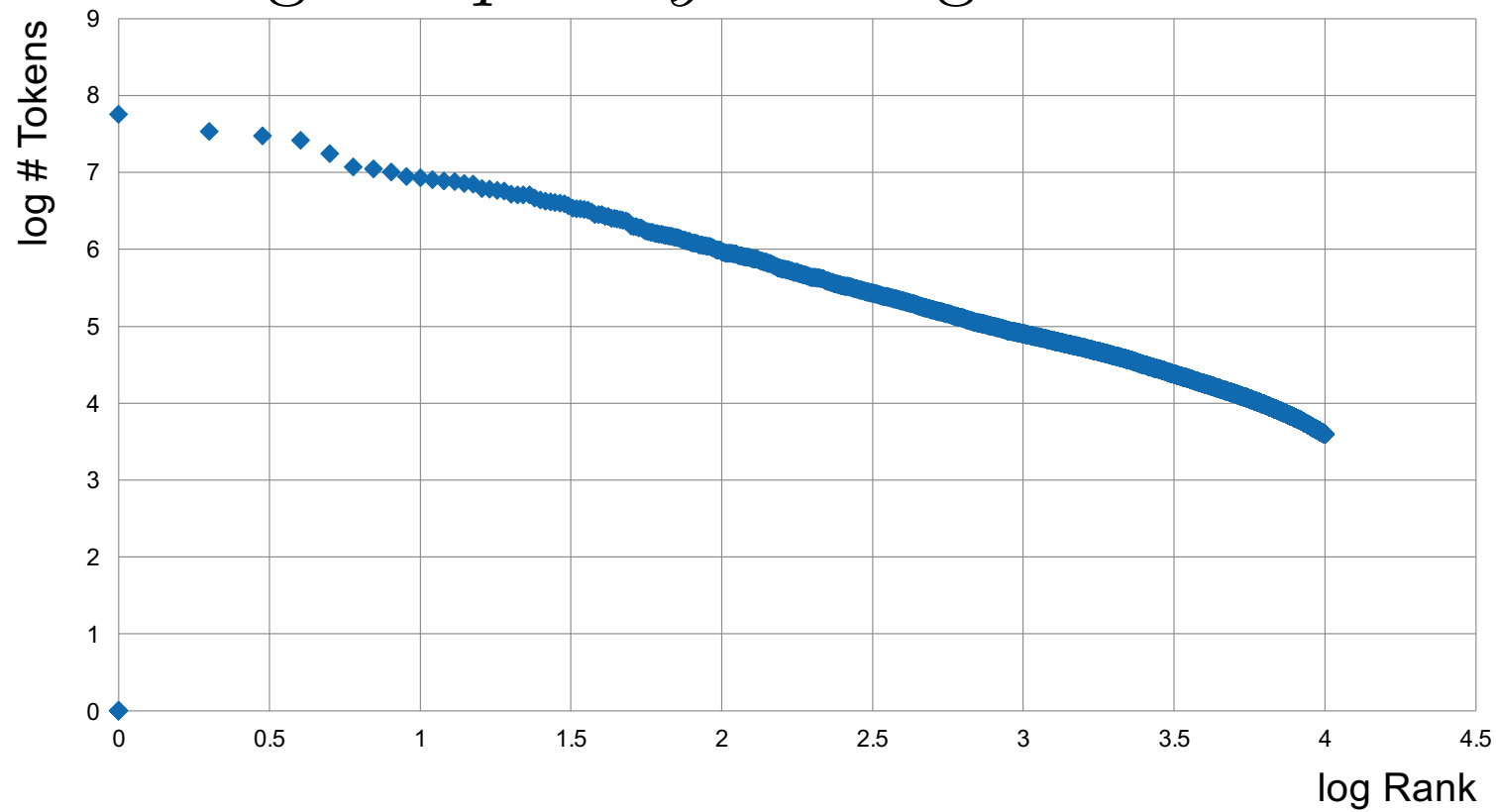


Zipf's law

$$\log \textit{Frequency} = a \log \textit{Rank} + b$$

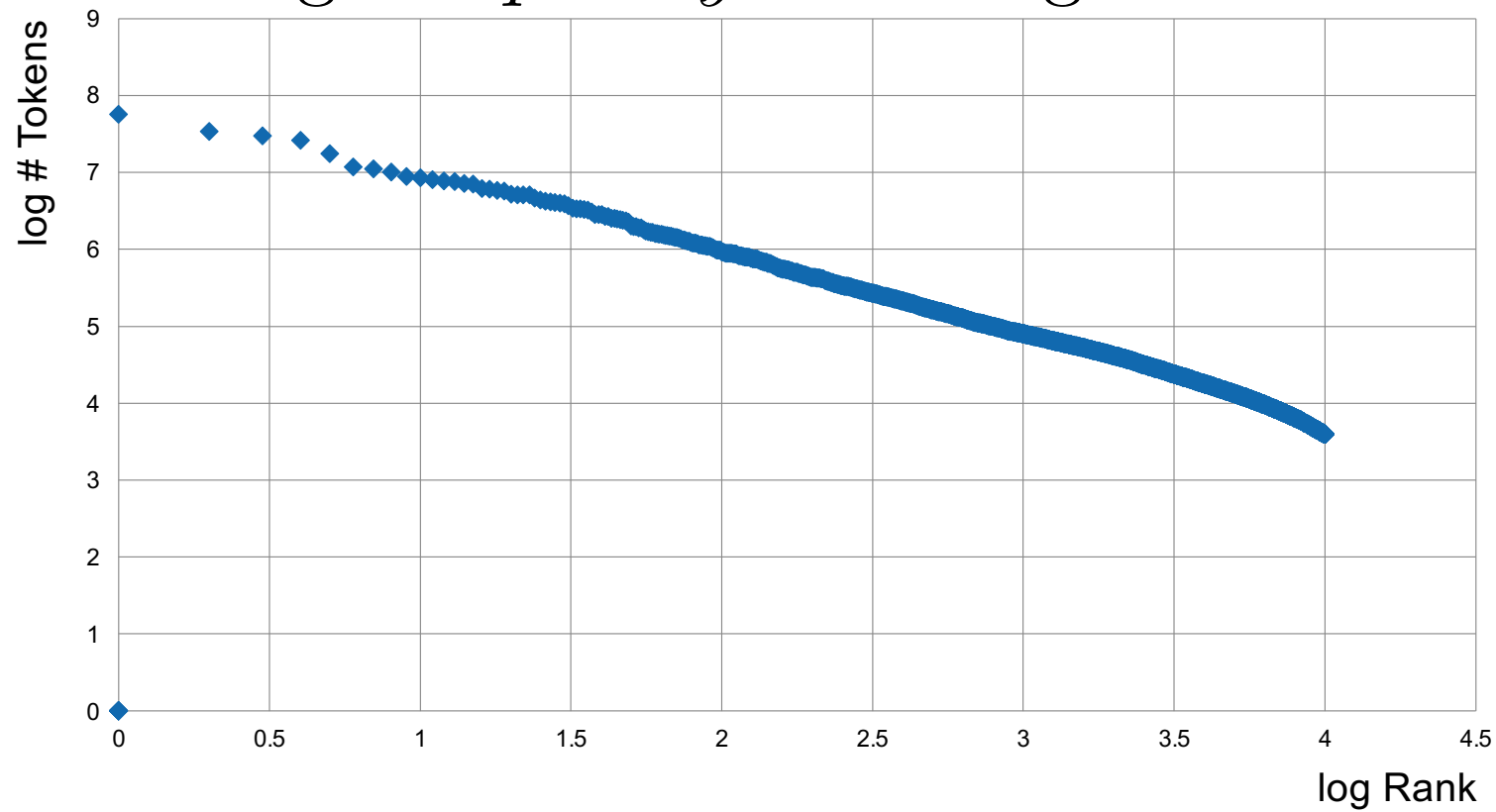
Zipf's law

$$\log \textit{Frequency} = a \log \textit{Rank} + b$$

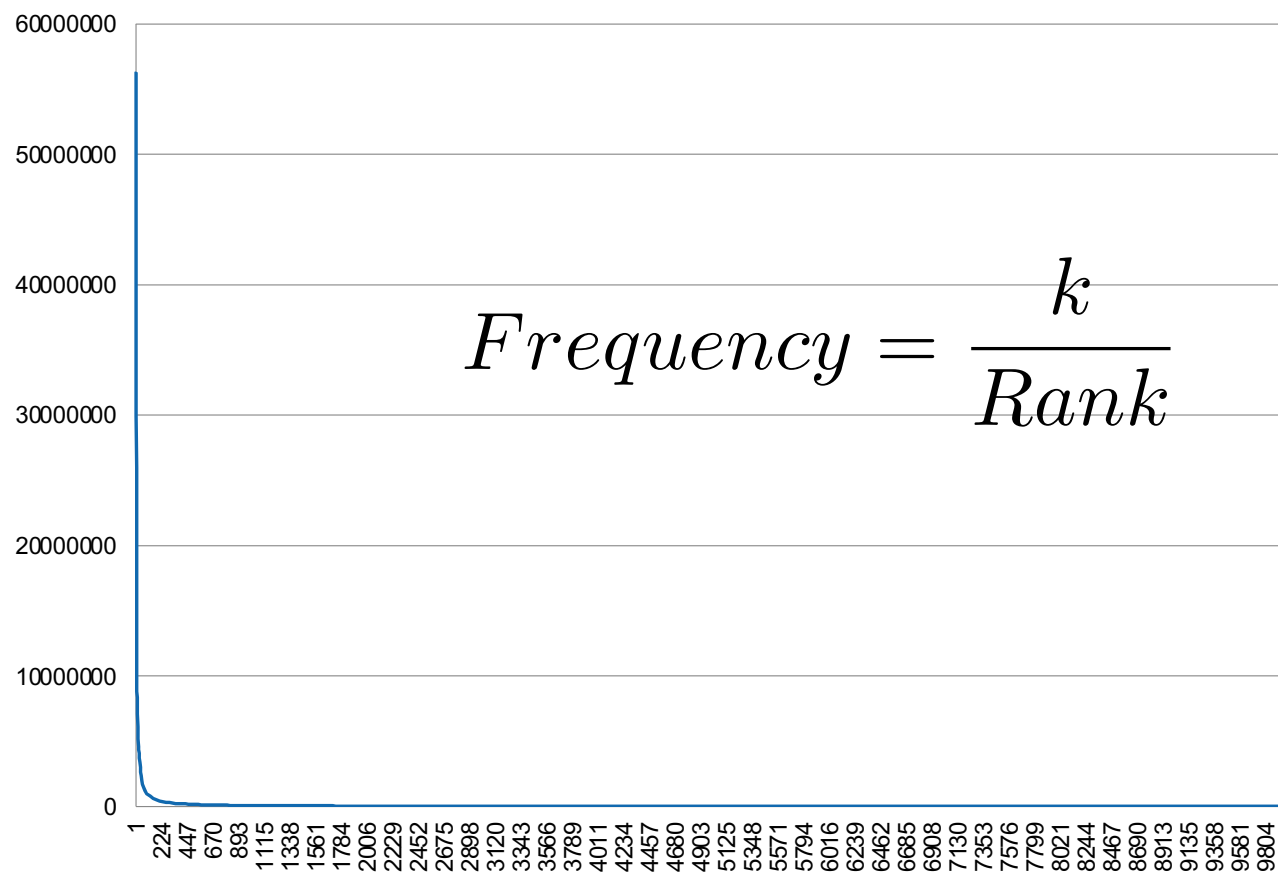


Zipf's law

$$\log \textit{Frequency} = b - \log \textit{Rank}$$



Zipf's law



Compression techniques already covered

Compression techniques already covered

~~1~~~~2~~~~3~~~~4~~~~5~~

Remove numbers

Compression techniques already covered

~~1~~~~2~~~~3~~~~4~~~~5~~

Remove numbers

Apple  apple

Case folding

Compression techniques already covered

~~1 2 3 4 5~~

Remove numbers

Apple  apple

Case folding

~~and of the~~

Remove stopwords

Compression techniques already covered

~~1 2 3 4 5~~

Remove numbers

Apple  apple

Case folding

~~and of the~~

Remove stopwords

computing  compute

Stemming

Compression techniques already covered

~~1 2 3 4 5~~

Remove numbers

Apple → apple

Case folding

~~and of the~~

Remove stopwords

computing → compute

Stemming

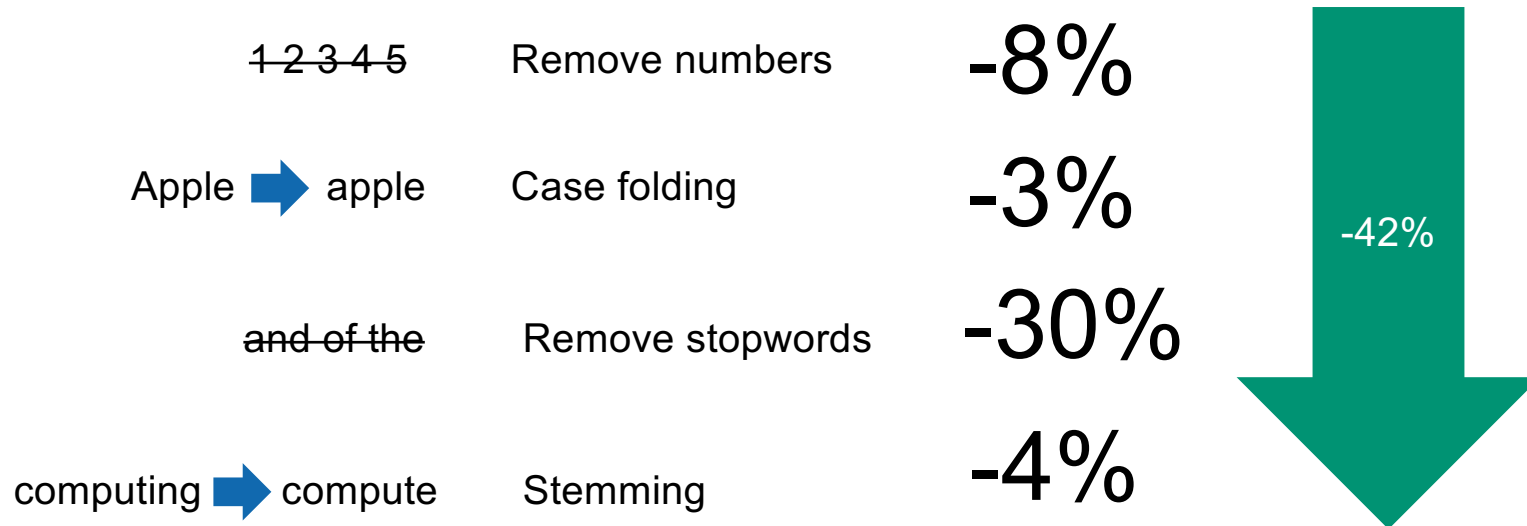
This reduces the size
of the dictionary!

Impact (number of terms/types)

1 2 3 4 5	Remove numbers	-2%	 -33%
Apple → apple	Case folding	-17%	
and of the	Remove stopwords	-0%	
computing → compute	Stemming	-17%	

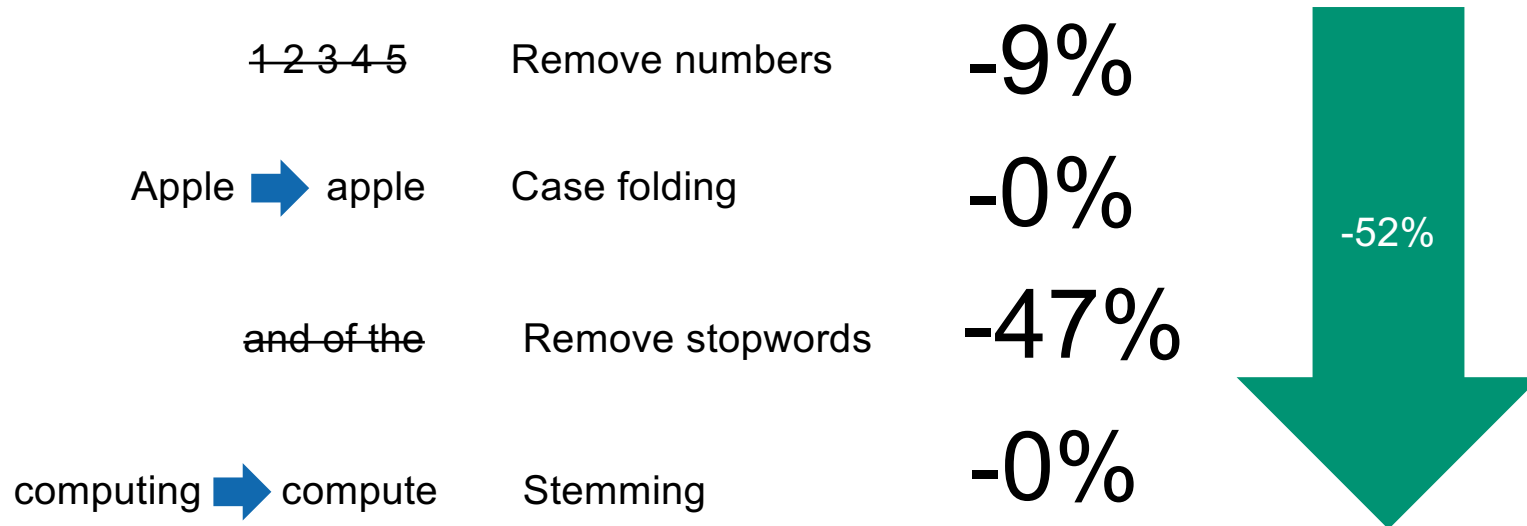
Source: Information Retrieval book

Impact (number of nonpositional postings)

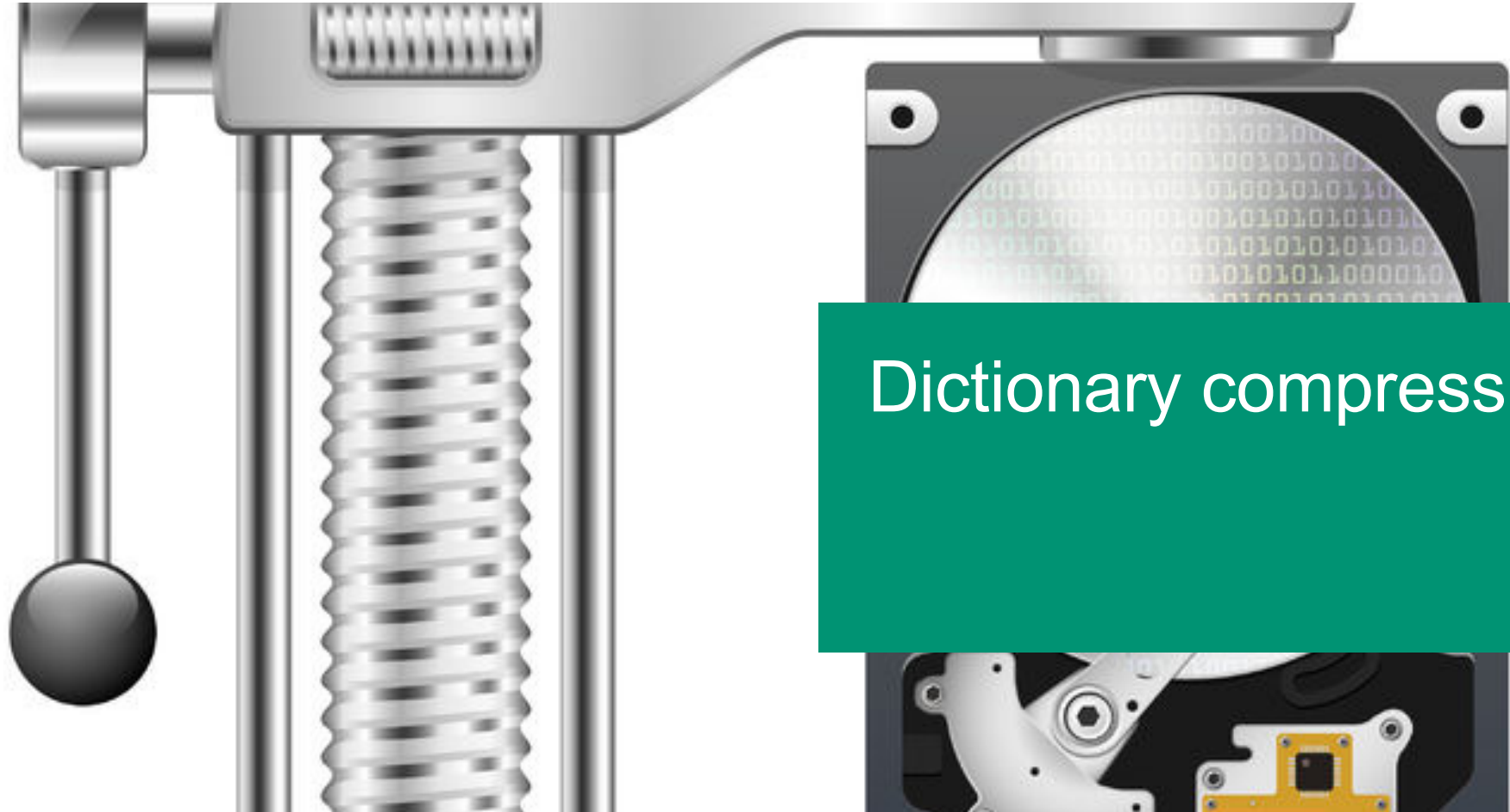


Source: Information Retrieval book

Impact (number of tokens = positional postings)



Source: Information Retrieval book

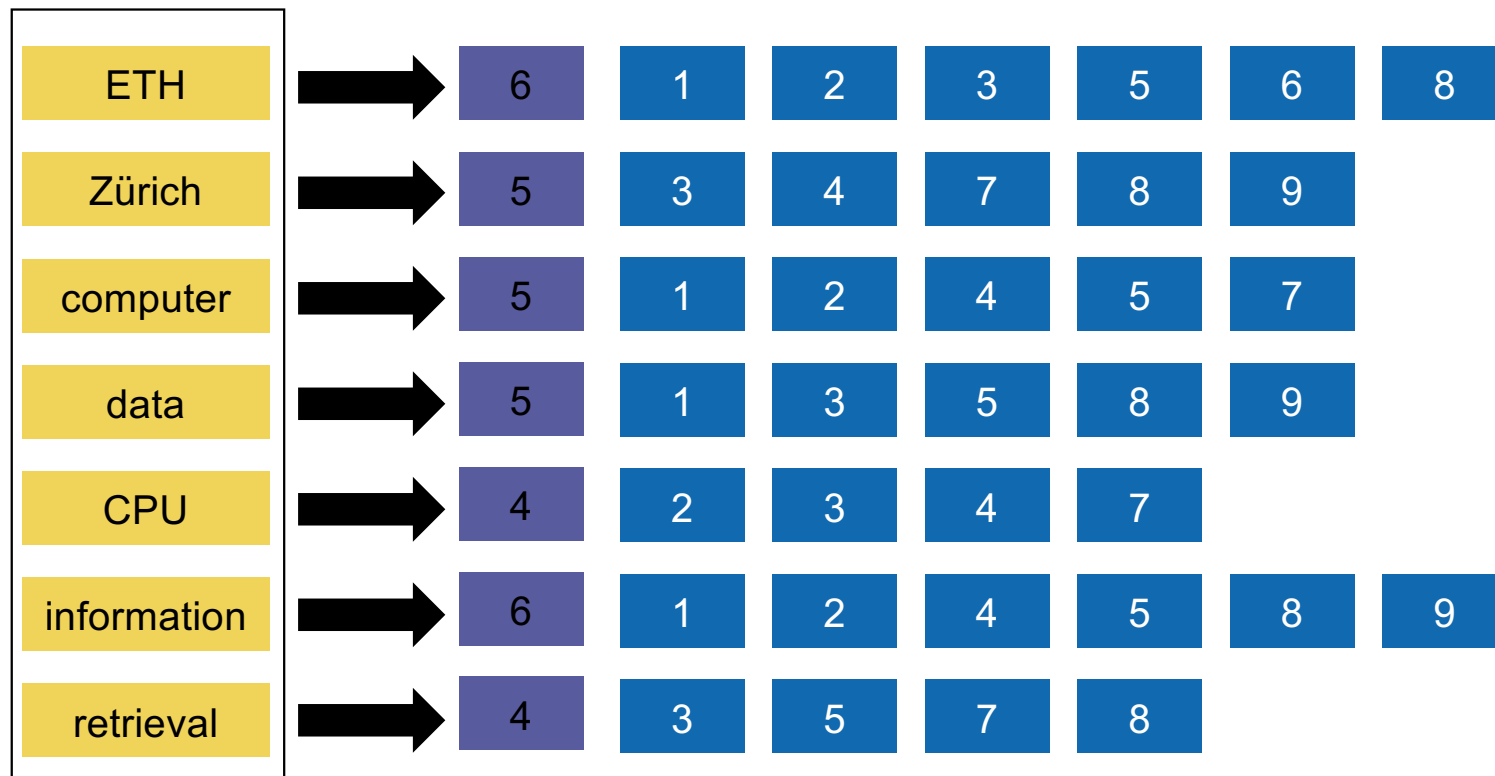


Dictionary compression

Standard inverted index

ETH	→	6	1	2	3	5	6	8
Zürich	→	5	3	4	7	8	9	
computer	→	5	1	2	4	5	7	
data	→	5	1	3	5	8	9	
CPU	→	4	2	3	4	7		
information	→	6	1	2	4	5	8	9
retrieval	→	4	3	5	7	8		

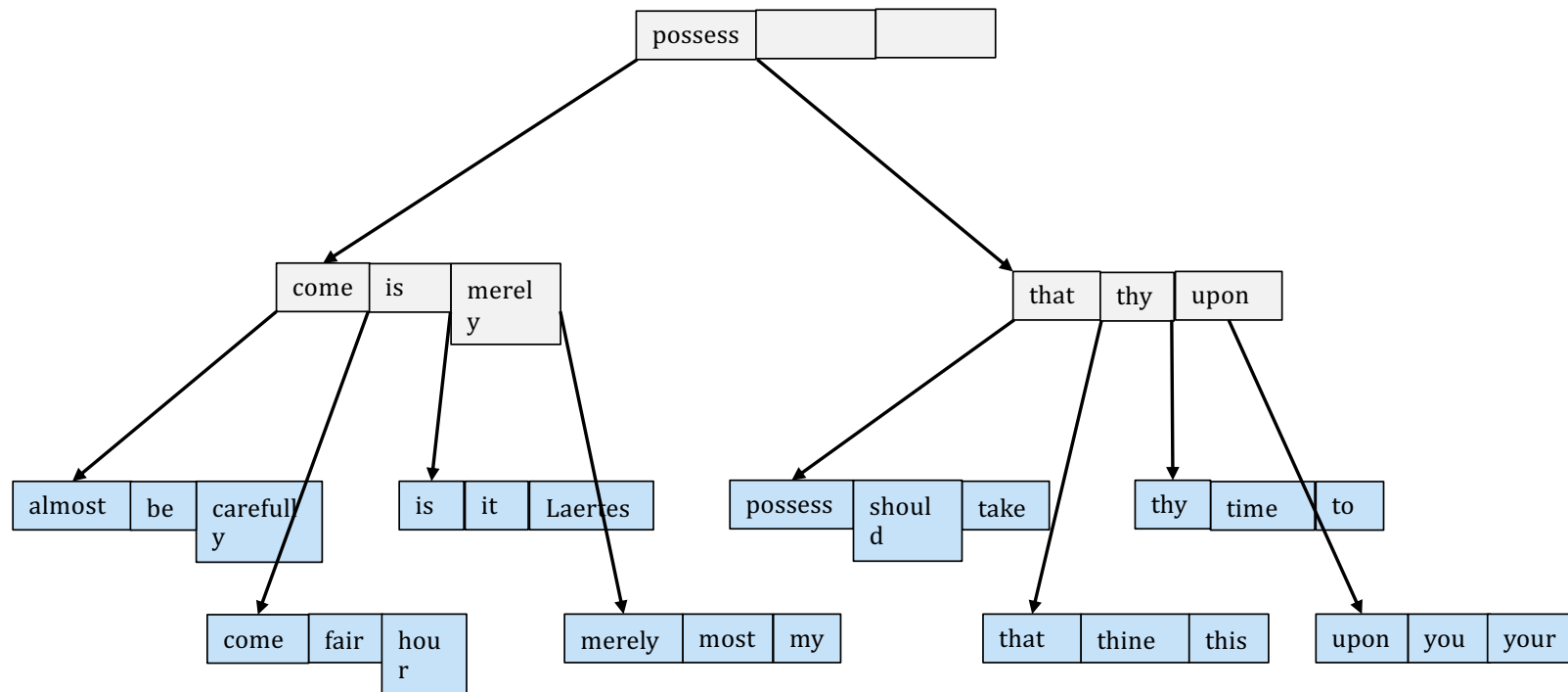
Standard inverted index



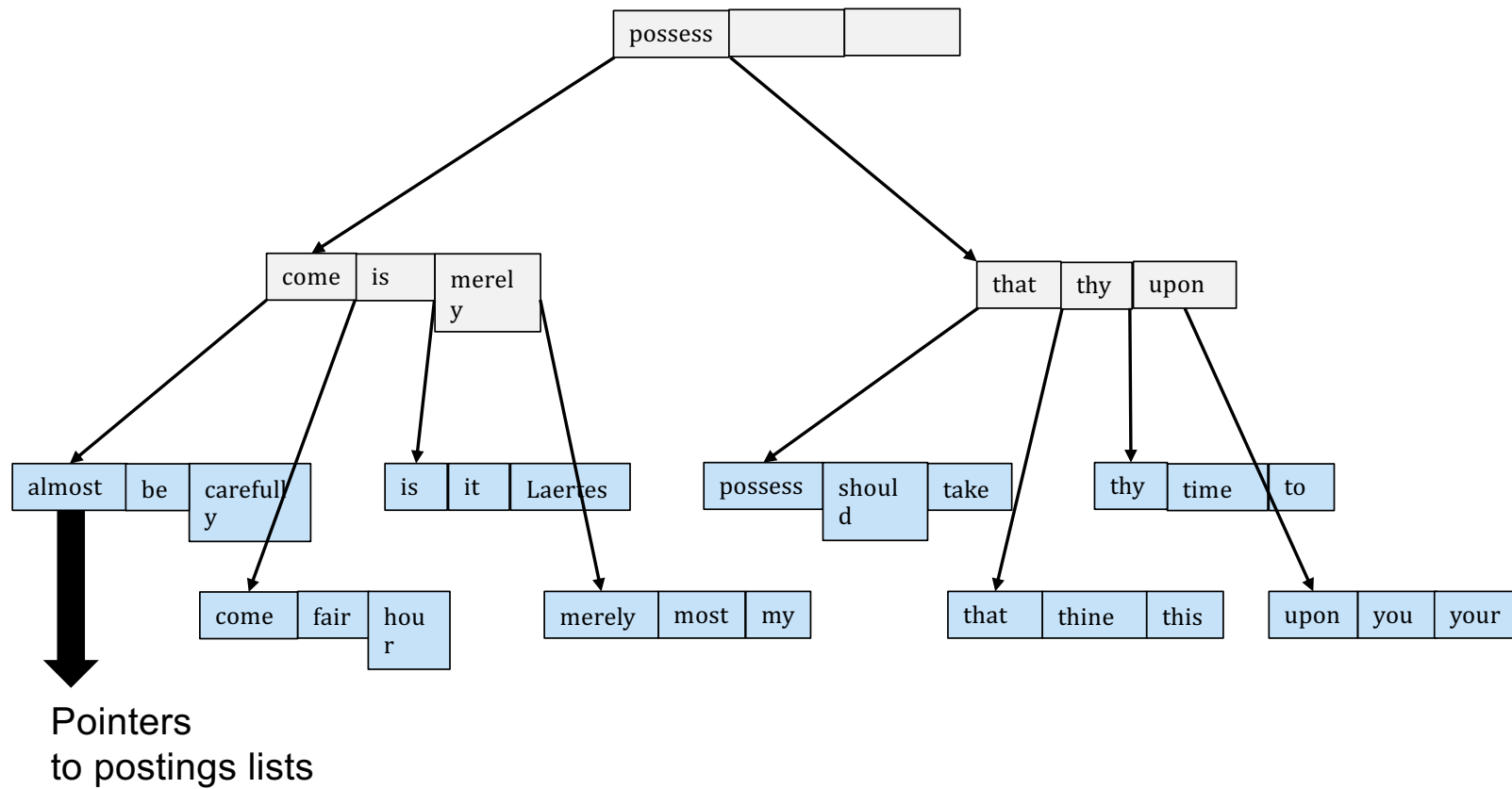
Let us start compressing the dictionary.

Status quo

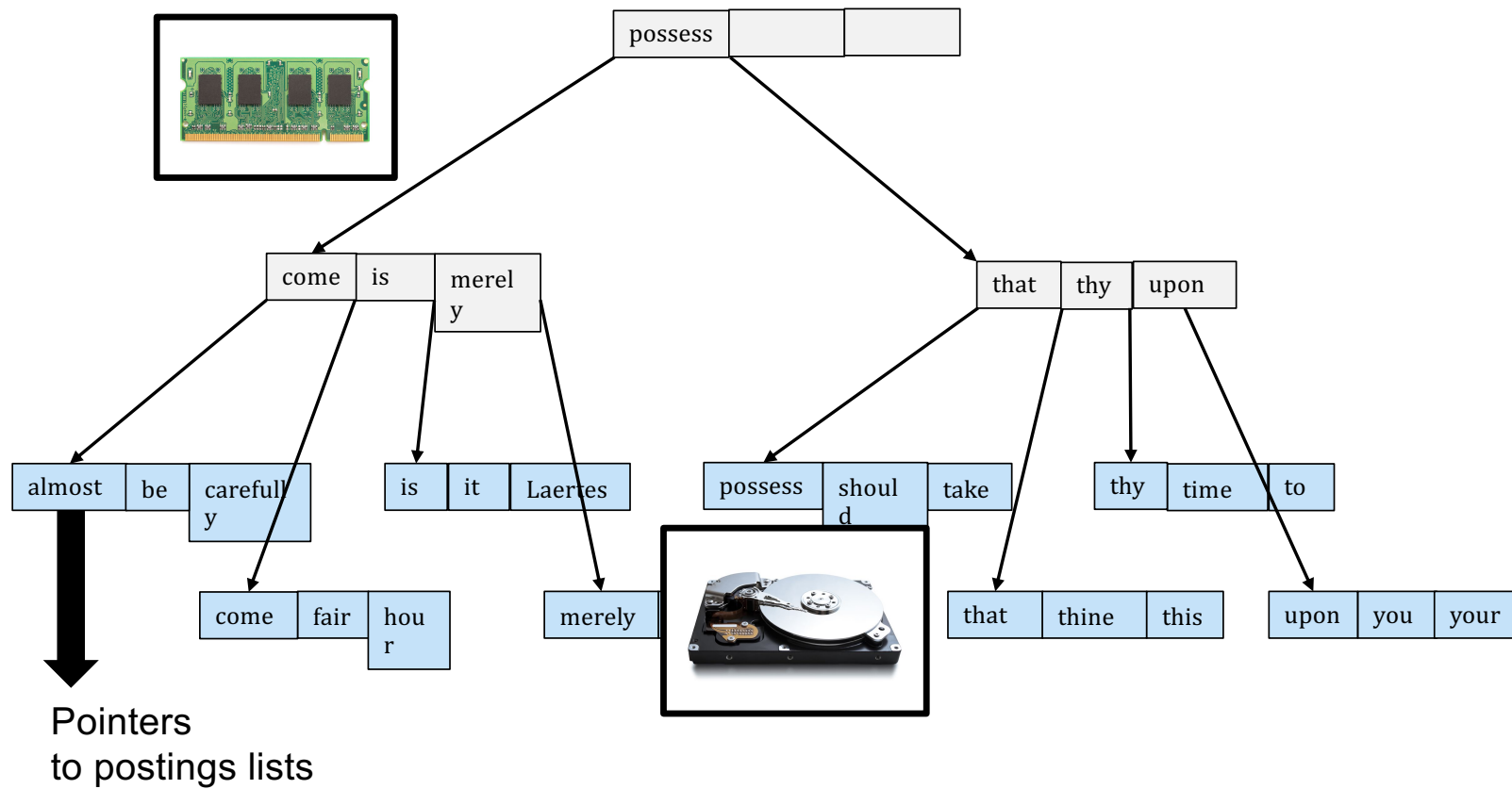
Status quo: Dictionary stored as a B+ tree



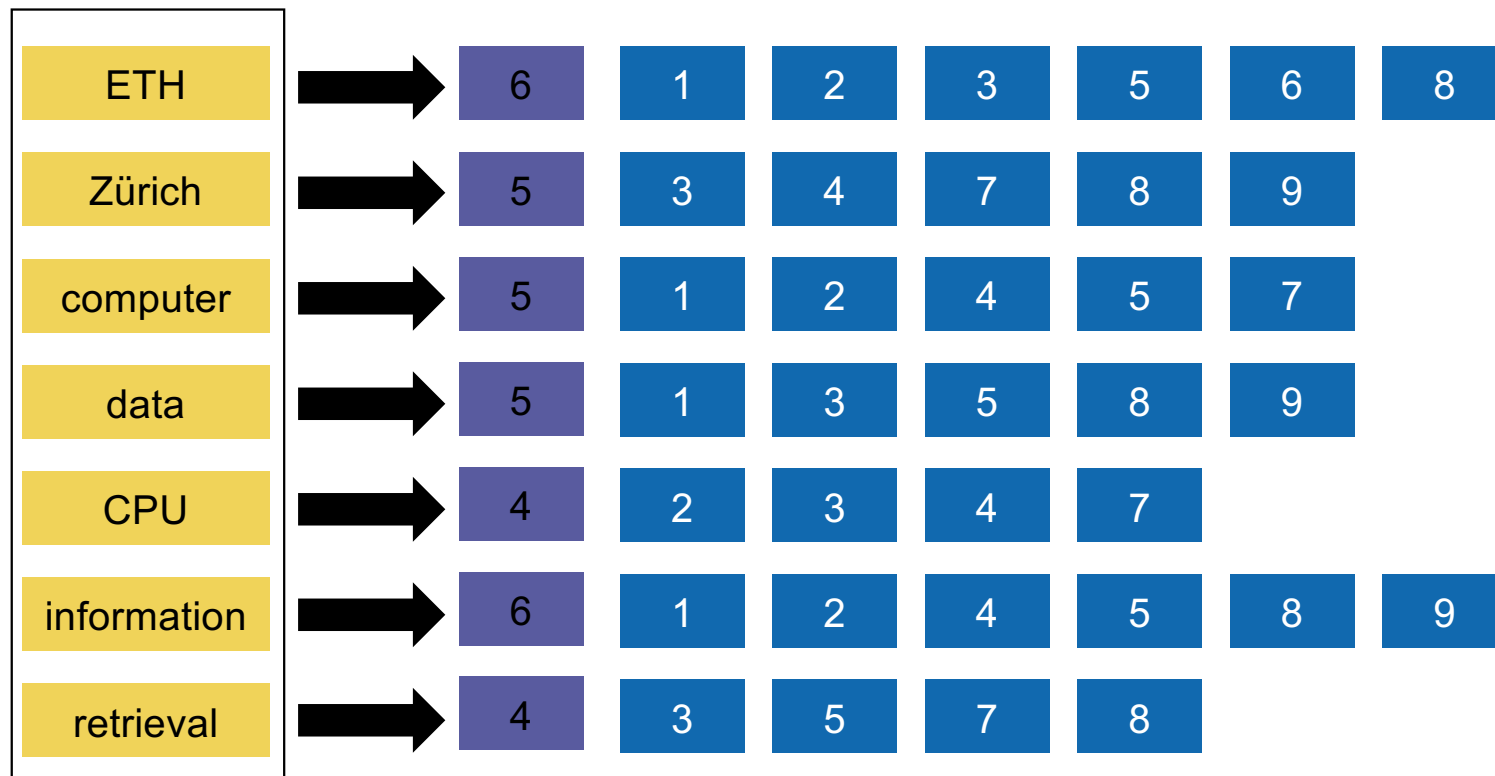
Status quo: Dictionary stored as a B+ tree



Status quo: Dictionary stored as a B+ tree

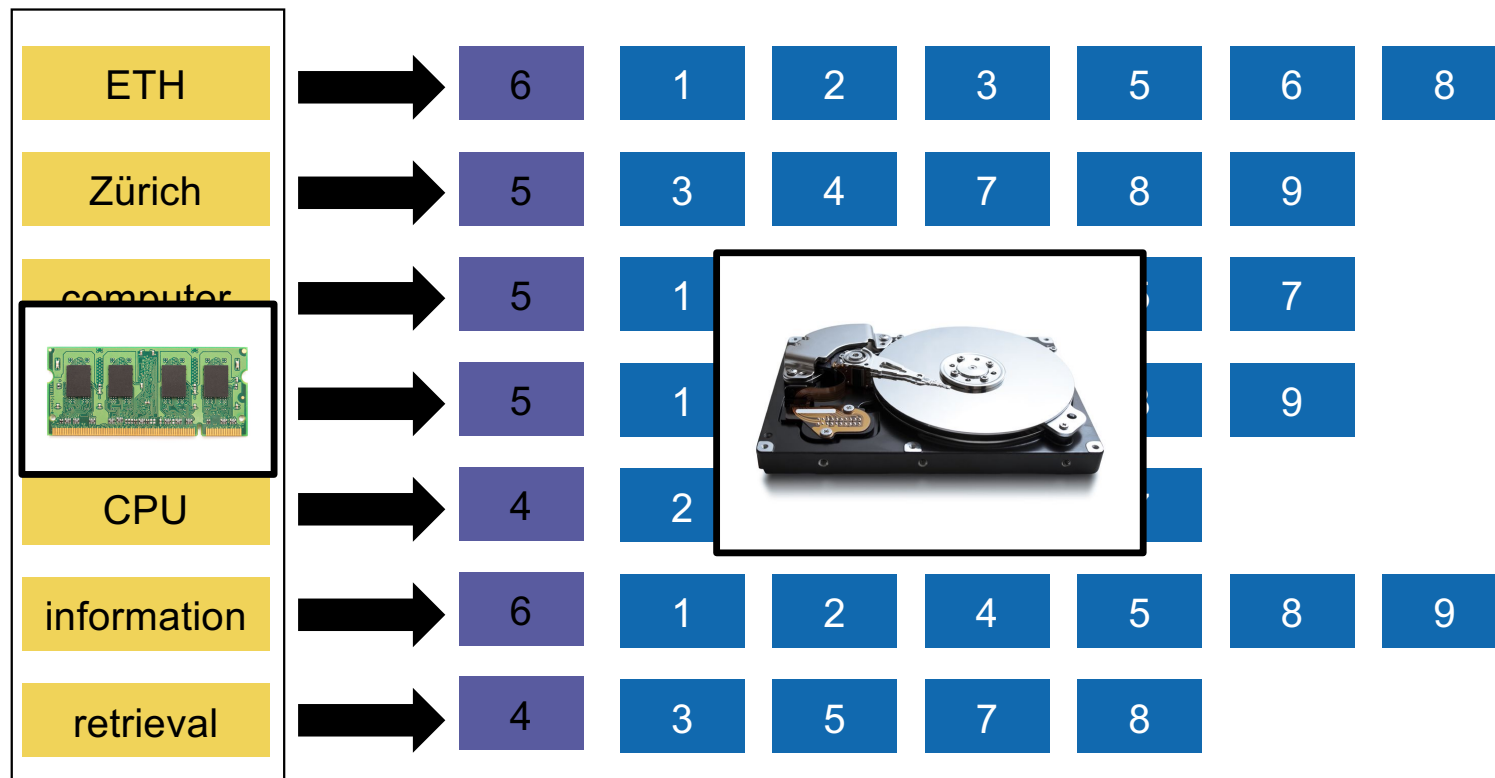


Standard inverted index



Let us start compressing the dictionary.

Standard inverted index

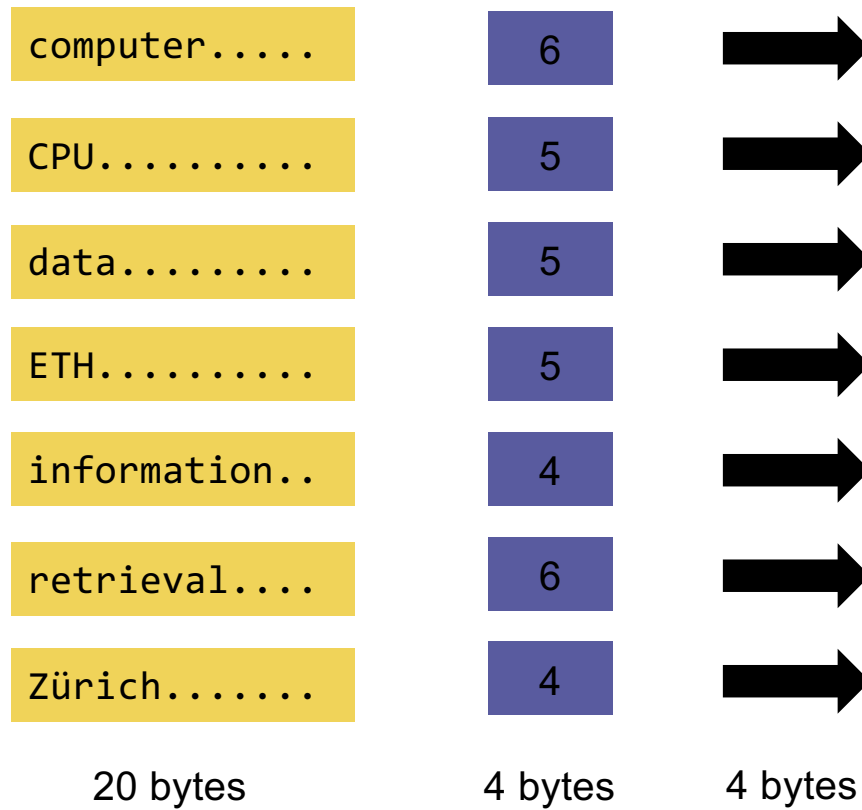


We can then make it fit in RAM.

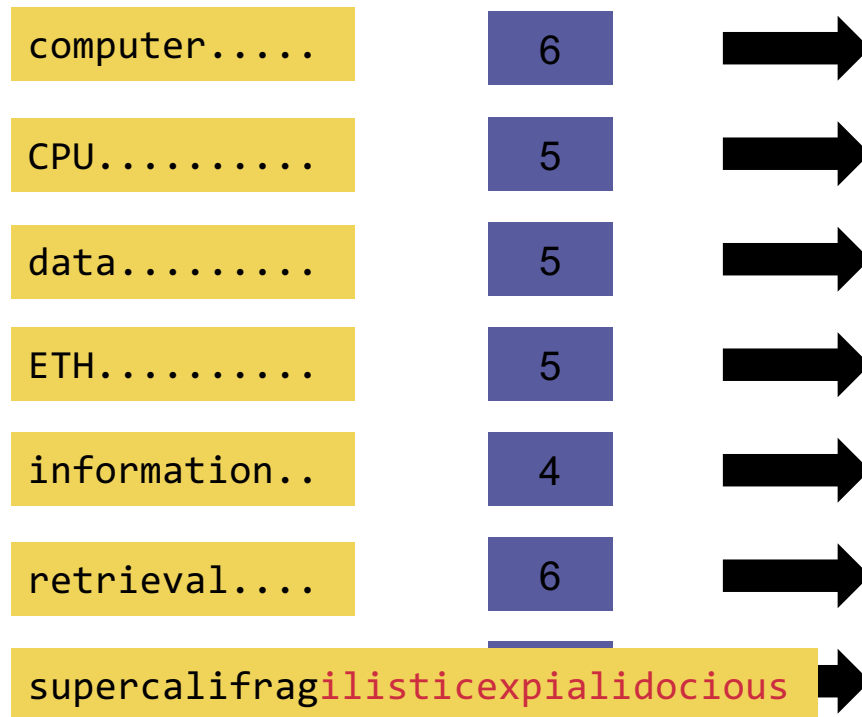
Approach 1: Array

computer.....	6	➡
CPU.....	5	➡
data.....	5	➡
ETH.....	5	➡
information..	4	➡
retrieval....	6	➡
Zürich.....	4	➡

Approach 1: Array



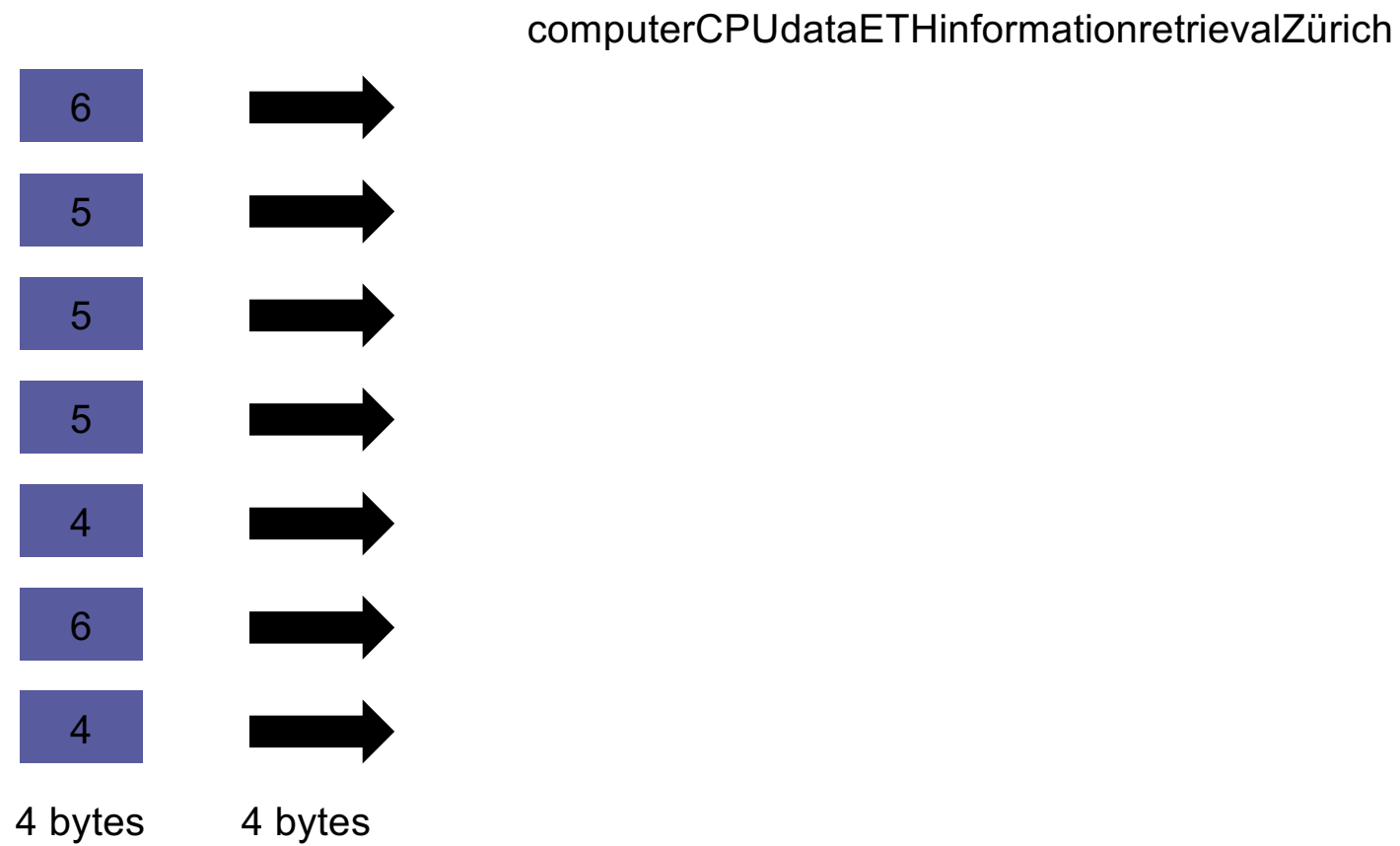
Approach 1: Issue



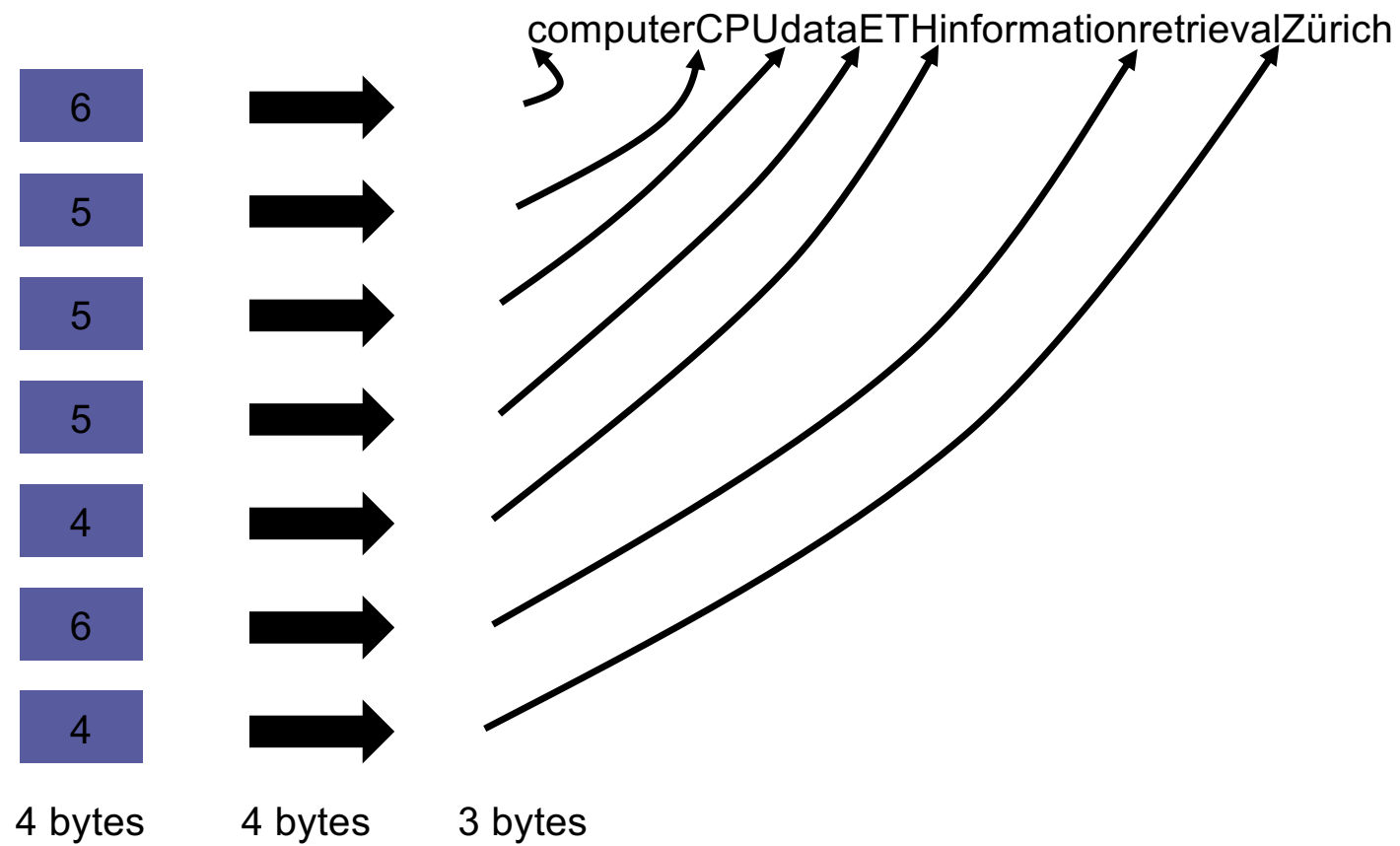
Approach 2: String

computerCPUdataETHinformationretrievalZürich

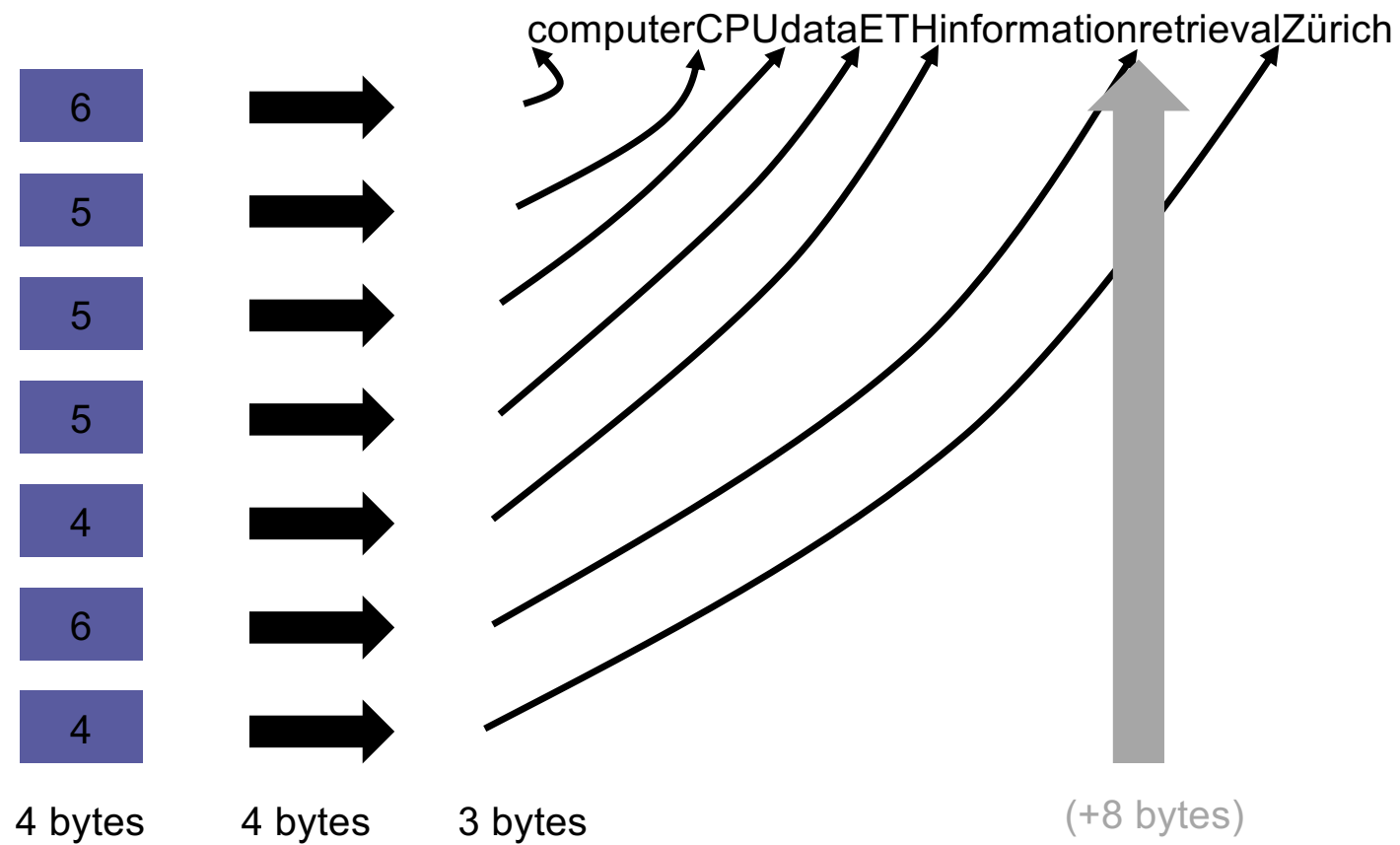
Approach 2: String



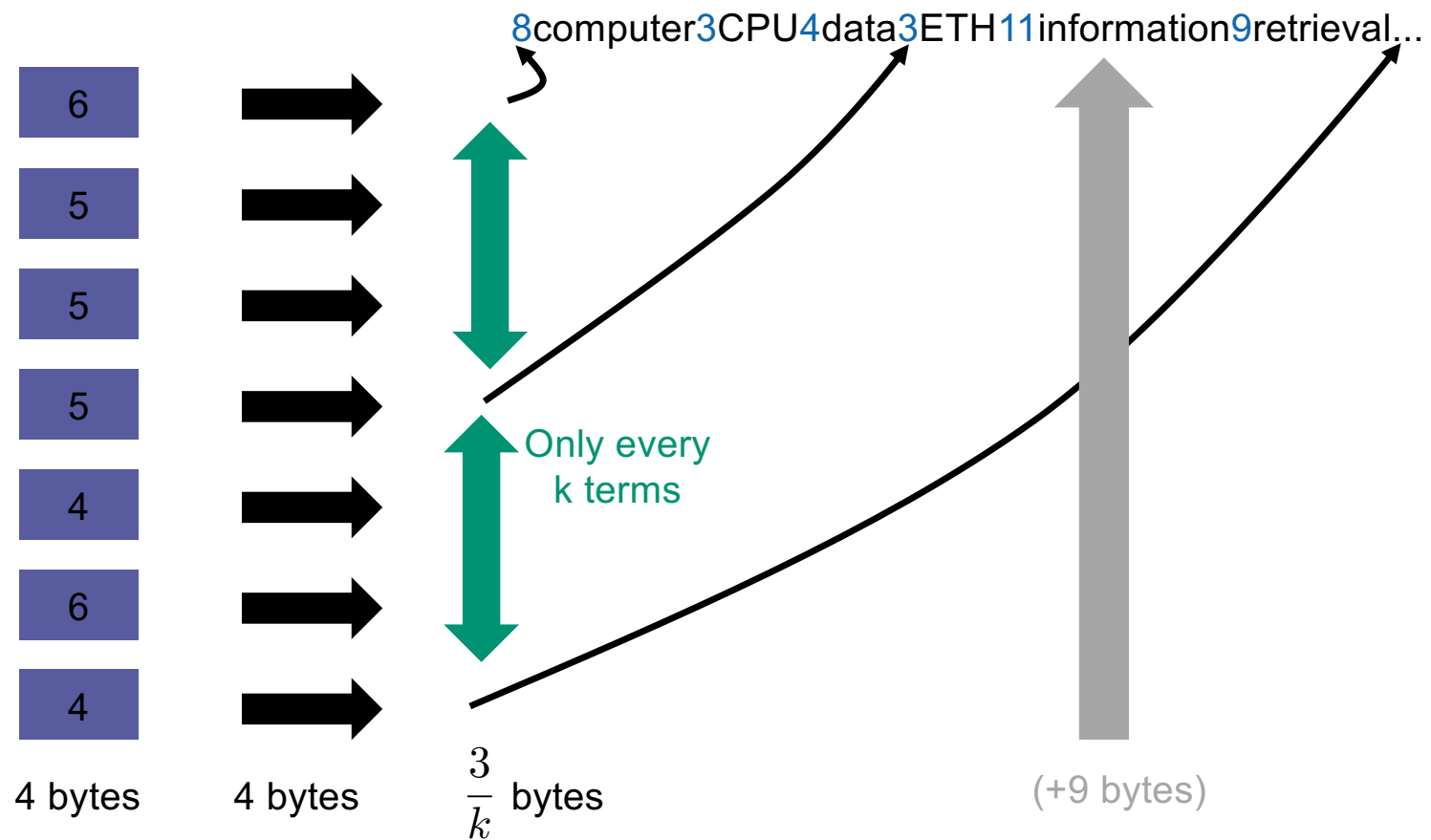
Approach 2: String



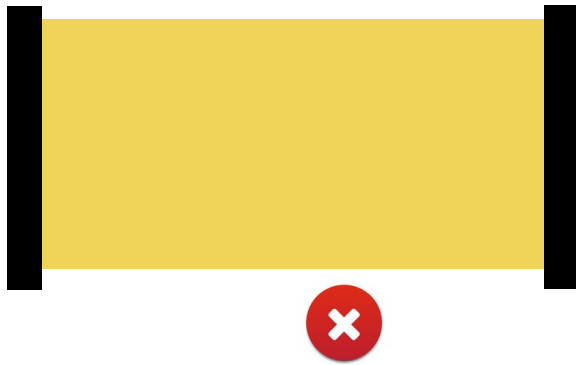
Approach 2: String



Approach 3: Blocked storage



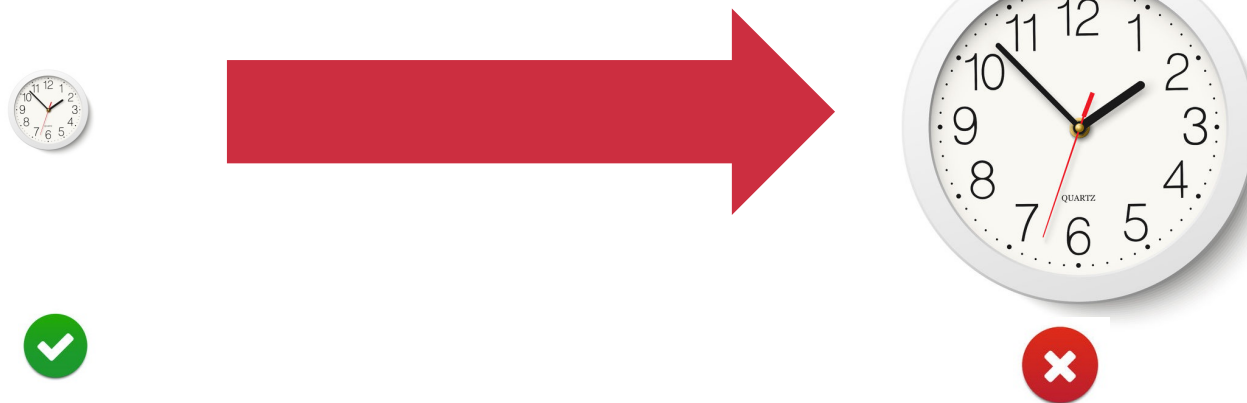
No free lunch



No free lunch



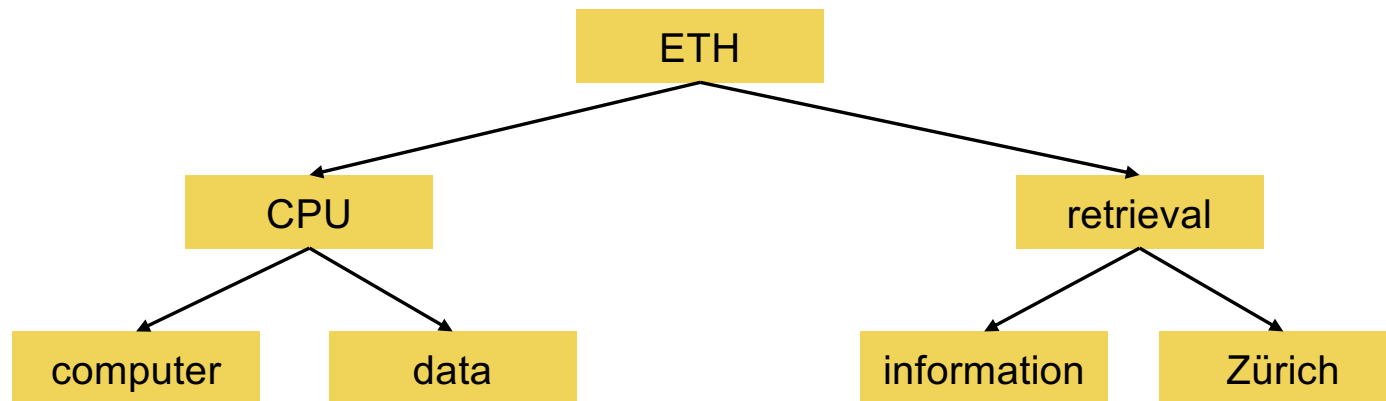
No free lunch



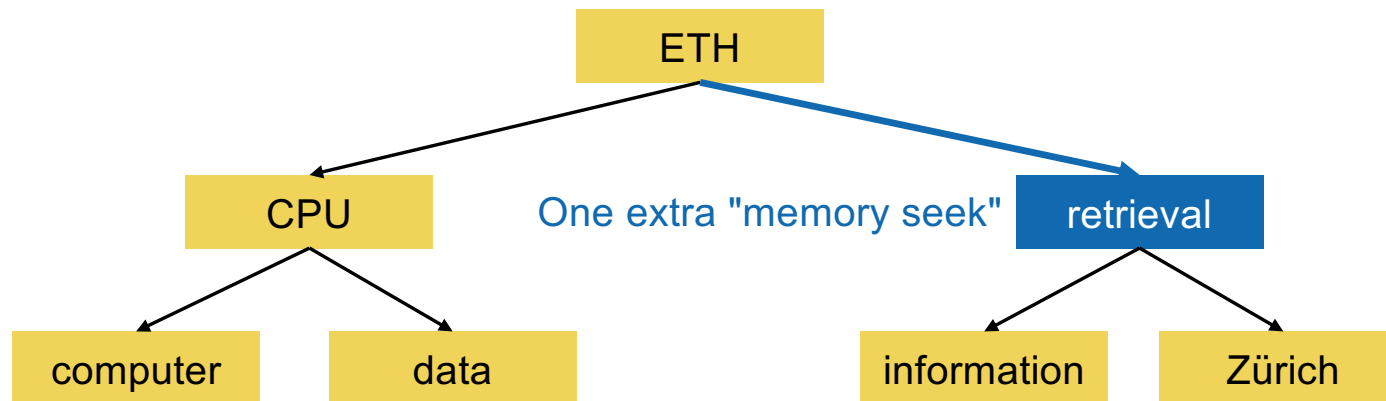
Compromise between space and time



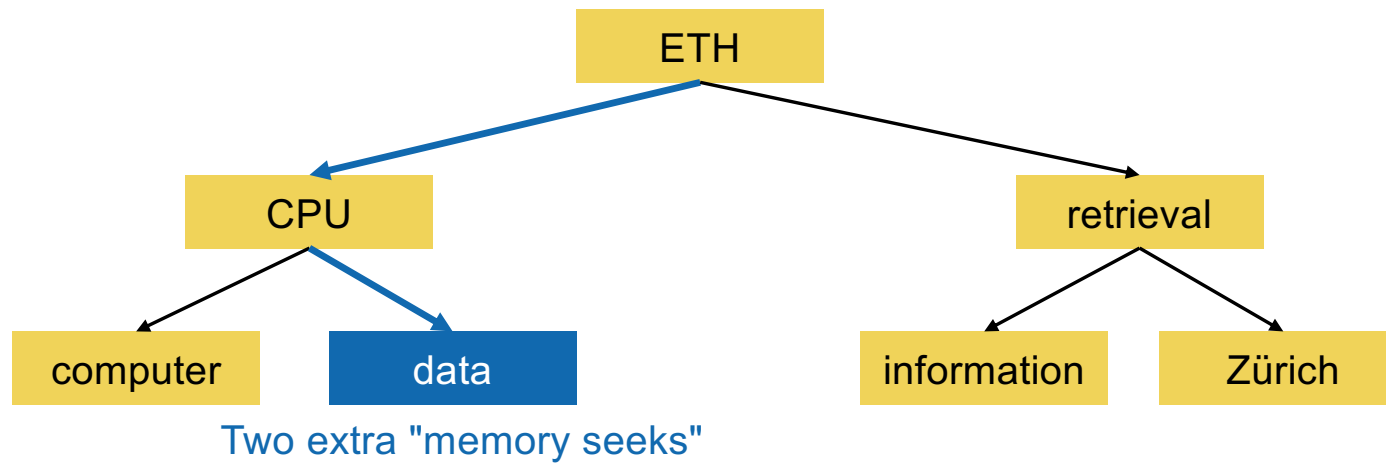
Binary search steps (no blocking)



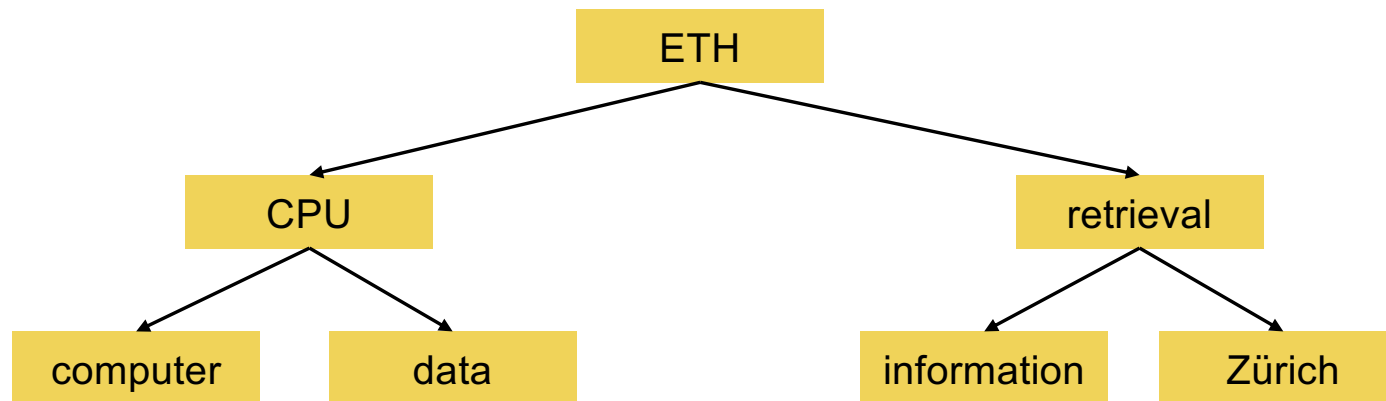
Binary search steps (no blocking)



Binary search steps (no blocking)

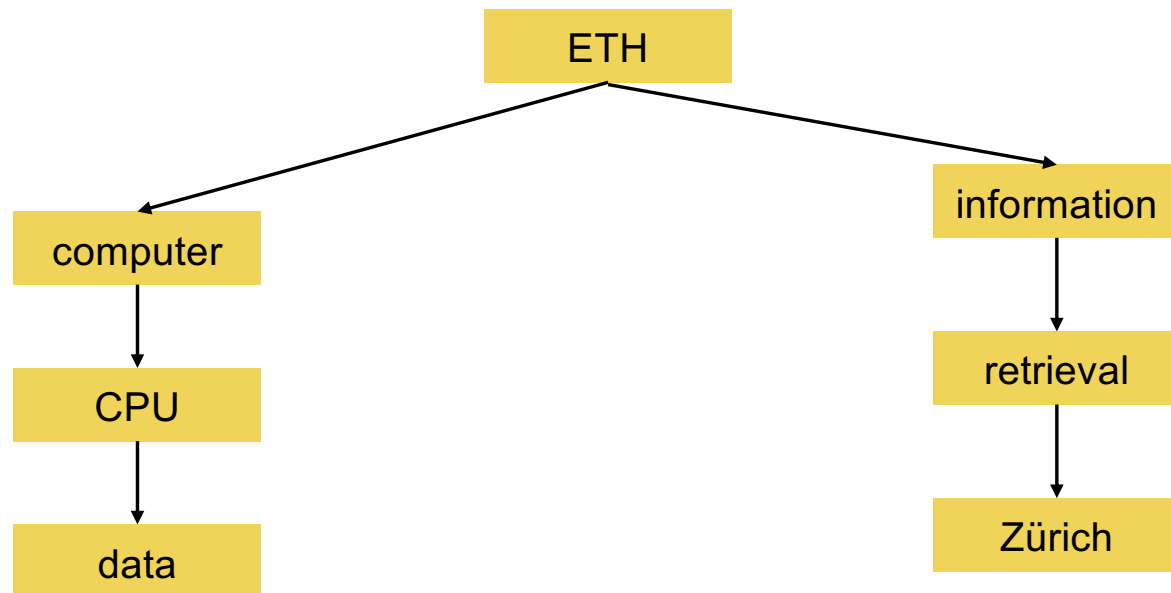


Binary search steps (no blocking)

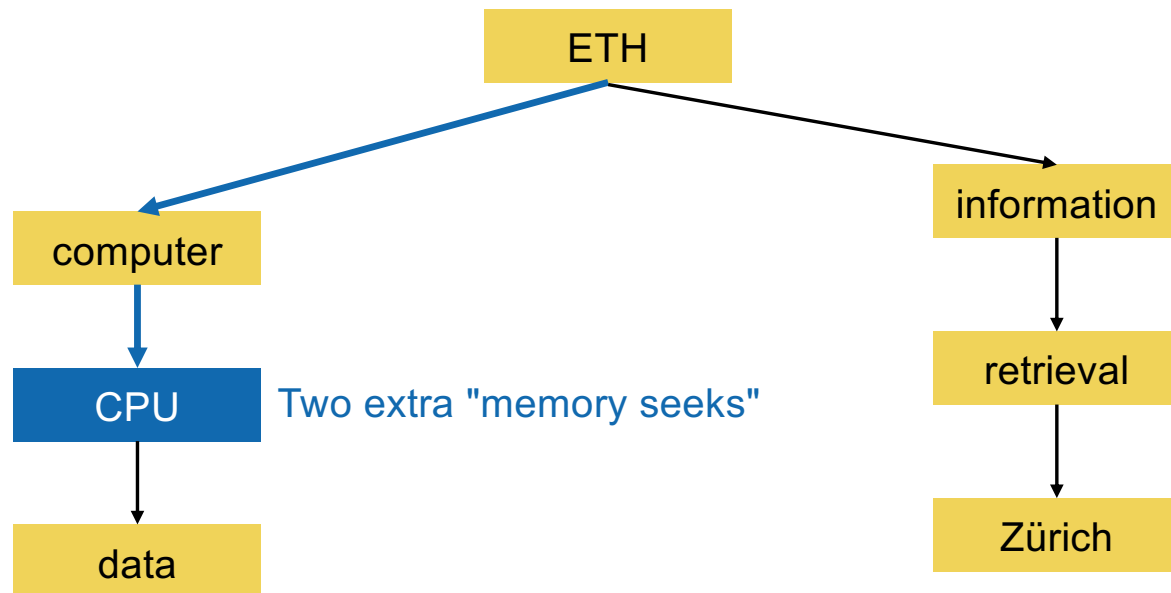


Average: $\text{avg}(0,1,2,2,1,2,2) = 1.4$

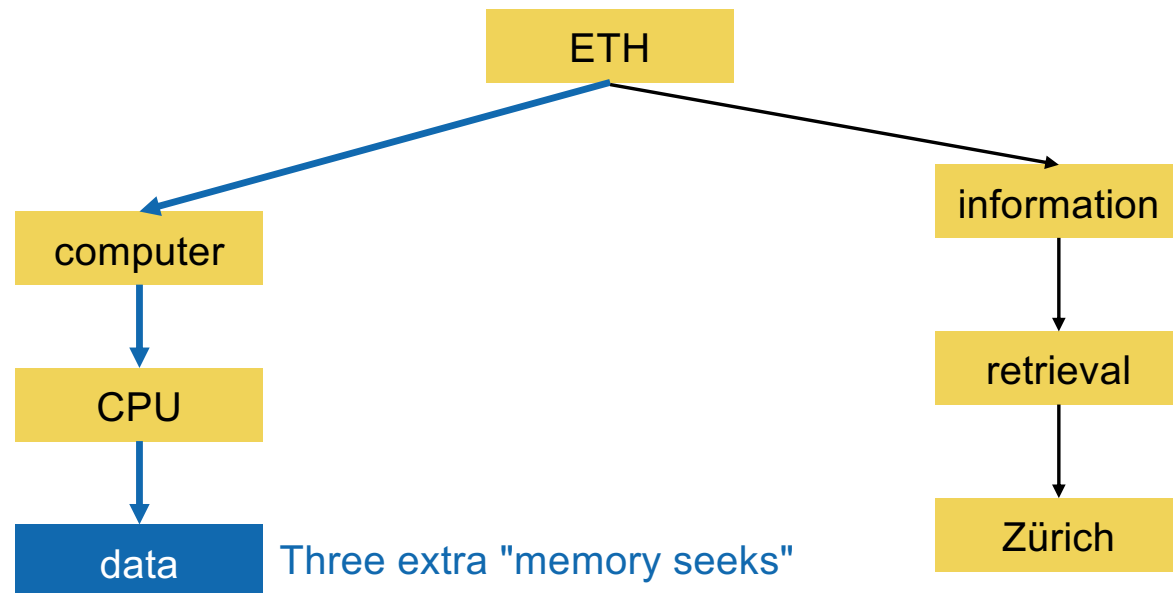
Binary search steps (with blocking)



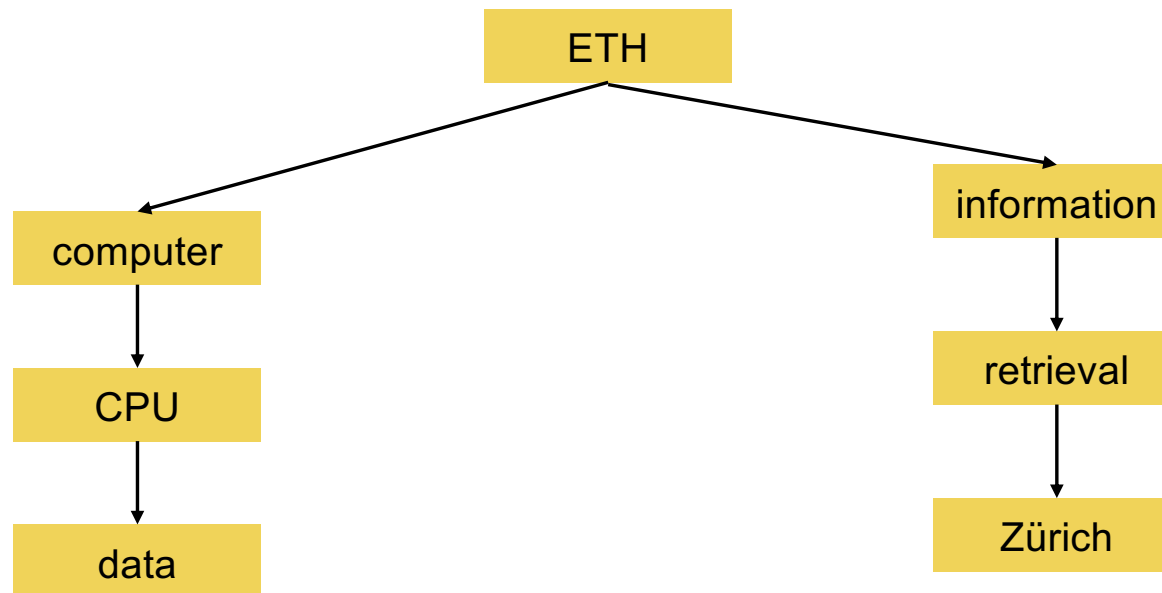
Binary search steps (with blocking)



Binary search steps (with blocking)

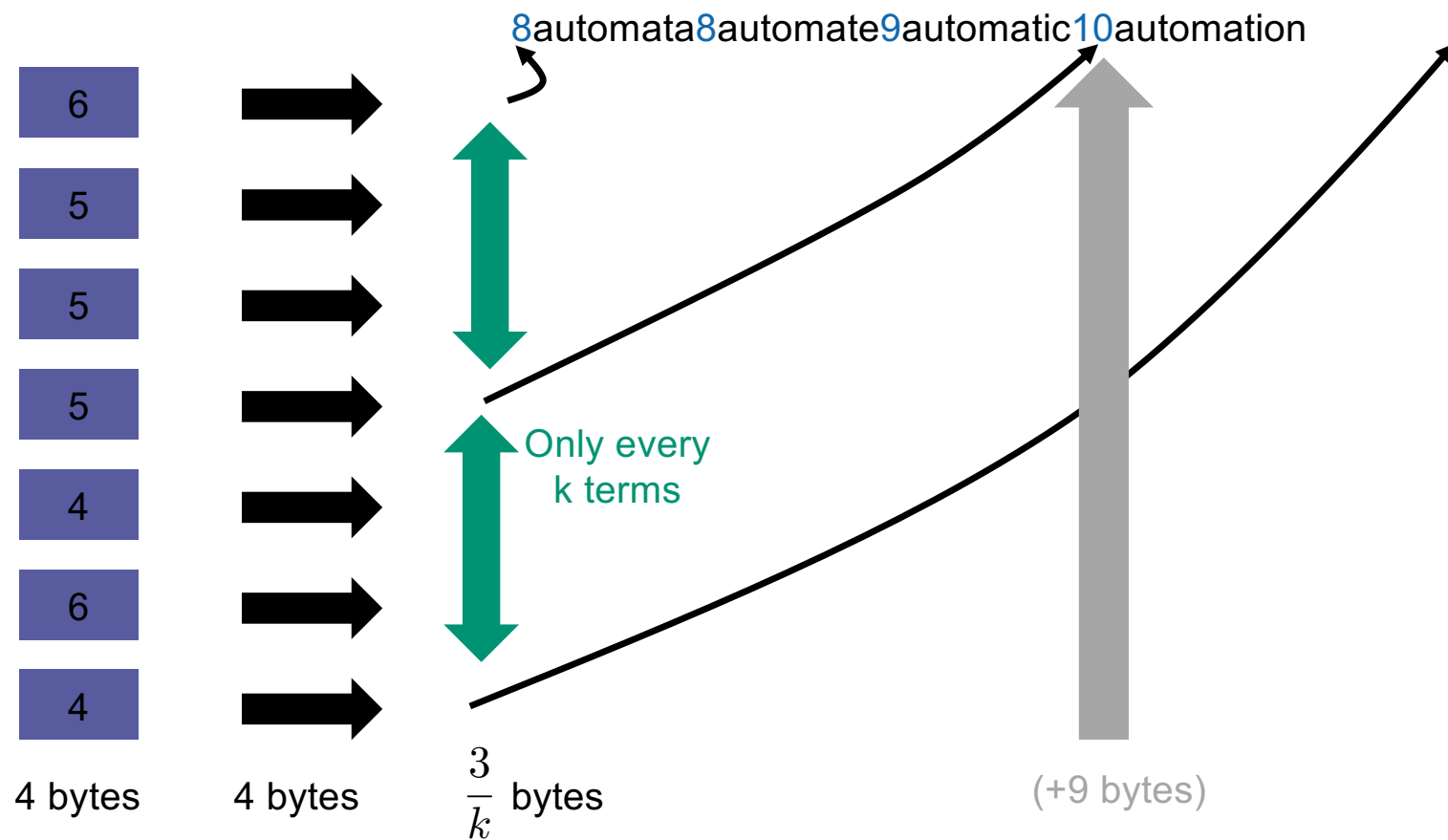


Binary search steps (with blocking)

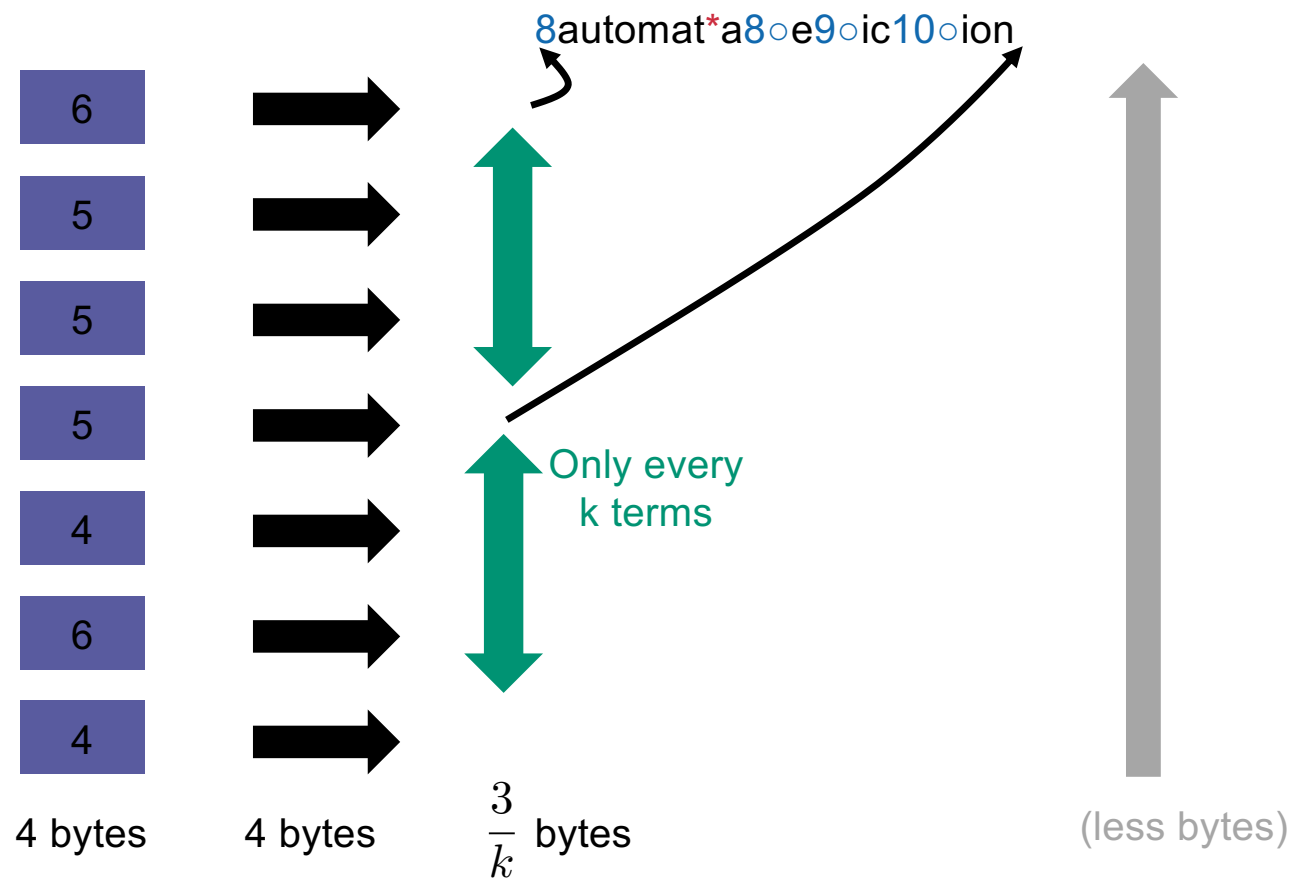


Average: $\text{avg}(0,1,2,3,1,2,3) = 1.7$

Approach 4: Front coding



Approach 4: Front coding



How did we do?



Collection: 960 MB

Source: Information Retrieval book

How did we do?



Fixed Width

11.2 MB



Collection: 960 MB

Source: Information Retrieval book

How did we do?



Fixed Width

11.2 MB

Unique string and pointers

7.6 MB



Collection: 960 MB

Source: Information Retrieval book

How did we do?



Fixed Width

11.2 MB

Unique string and pointers

7.6 MB

Blocking (k=4)

7.1 MB



Collection: 960 MB

Source: Information Retrieval book

How did we do?



Fixed Width

11.2 MB

Unique string and pointers

7.6 MB

Blocking (k=4)

7.1 MB

Blocking and front coding

5.9 MB



Collection: 960 MB

Source: Information Retrieval book

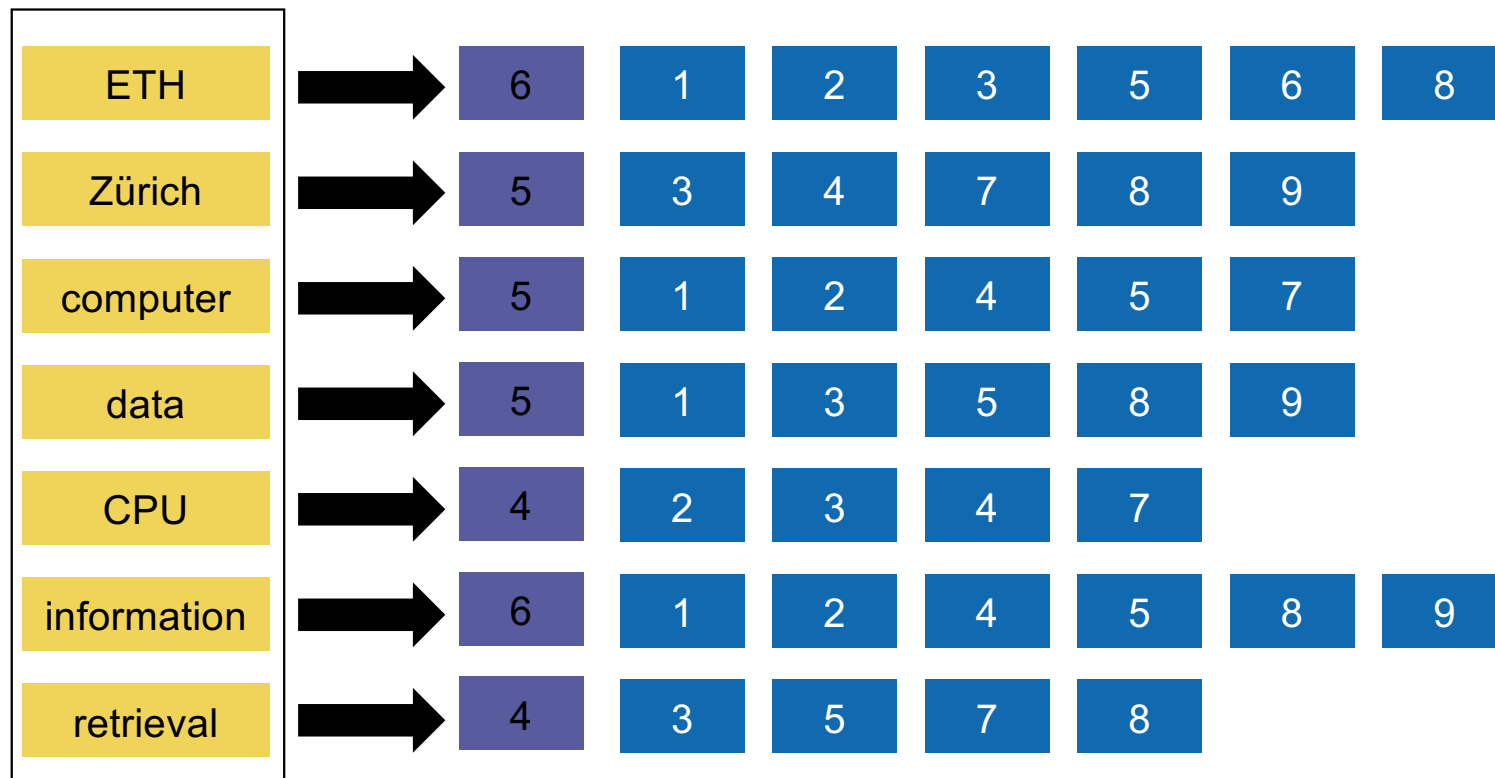


Postings file compression

Standard inverted index

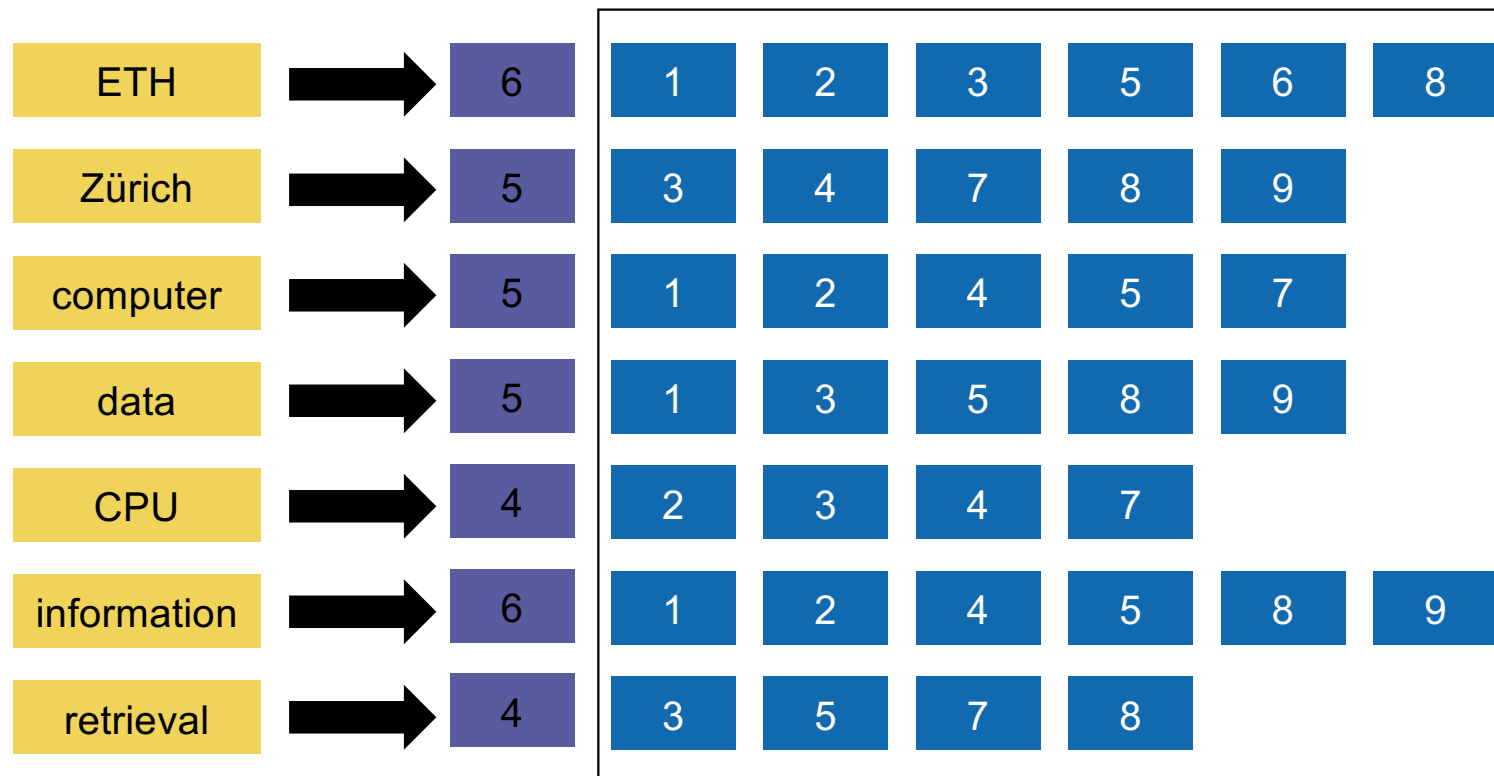
ETH	→	6	1	2	3	5	6	8
Zürich	→	5	3	4	7	8	9	
computer	→	5	1	2	4	5	7	
data	→	5	1	3	5	8	9	
CPU	→	4	2	3	4	7		
information	→	6	1	2	4	5	8	9
retrieval	→	4	3	5	7	8		

Standard inverted index



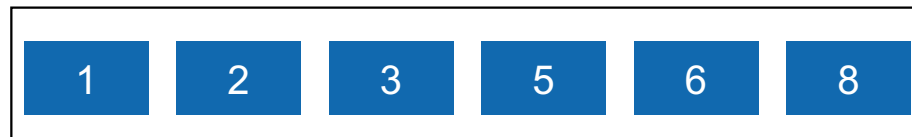
We compressed this...

Standard inverted index



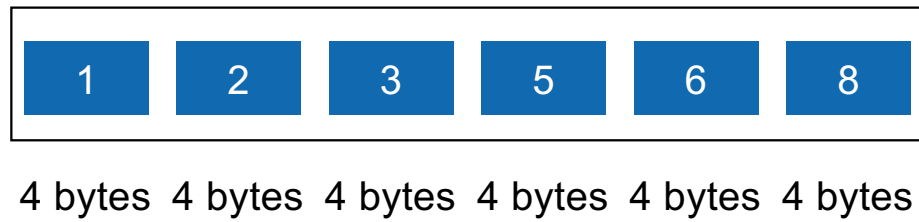
Now, we want to compress this.

Standard inverted index



In other words, we want to
compress lists of
integers

Standard storage



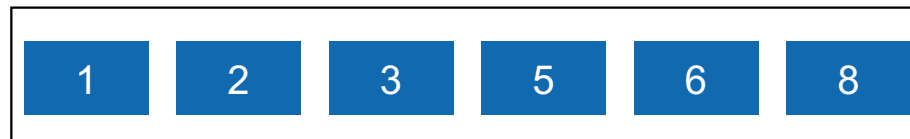
Standard storage



4 bytes 4 bytes 4 bytes 4 bytes 4 bytes 4 bytes

(4 bytes = 32 bits)

Standard storage



4 bytes 4 bytes 4 bytes 4 bytes 4 bytes 4 bytes

(4 bytes = 32 bits)

Numbers between 0 and 4,294,967,296

Encoding gaps



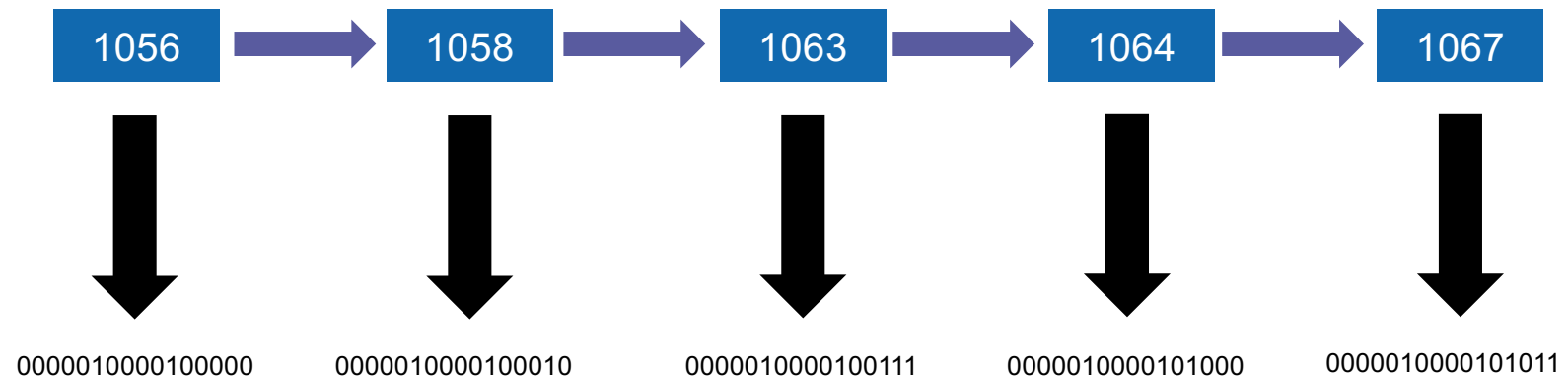
4 bytes 4 bytes 4 bytes 4 bytes 4 bytes 4 bytes
(4 bytes = 32 bits)

Can we encode with less space?

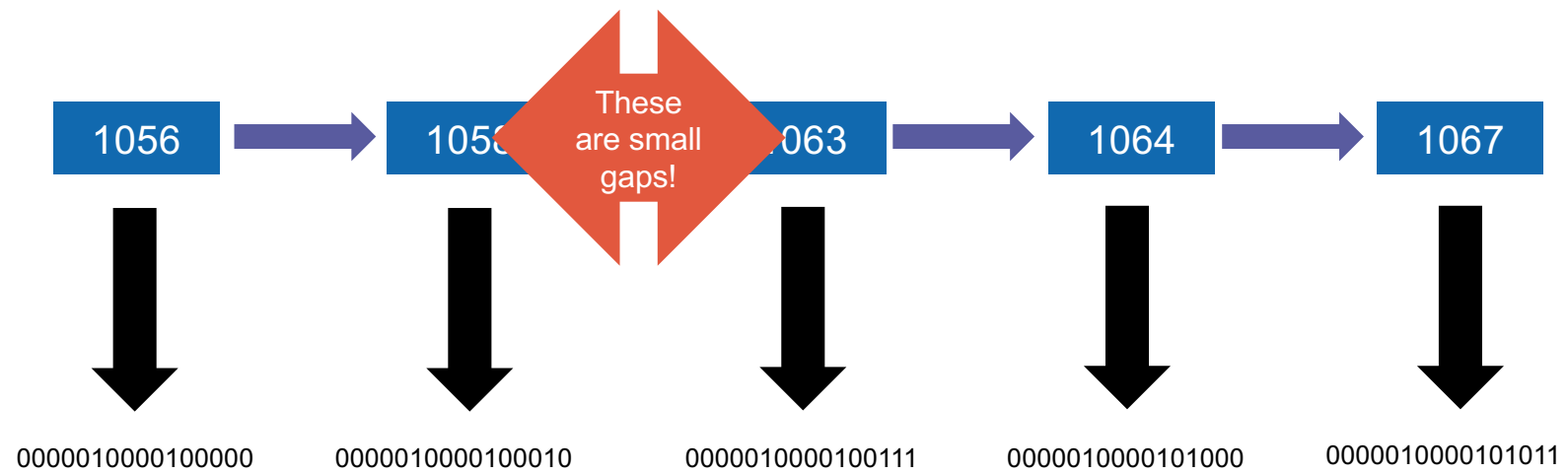
Encoding gaps



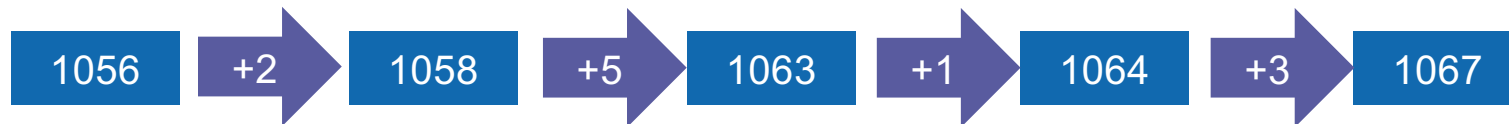
Encoding gaps



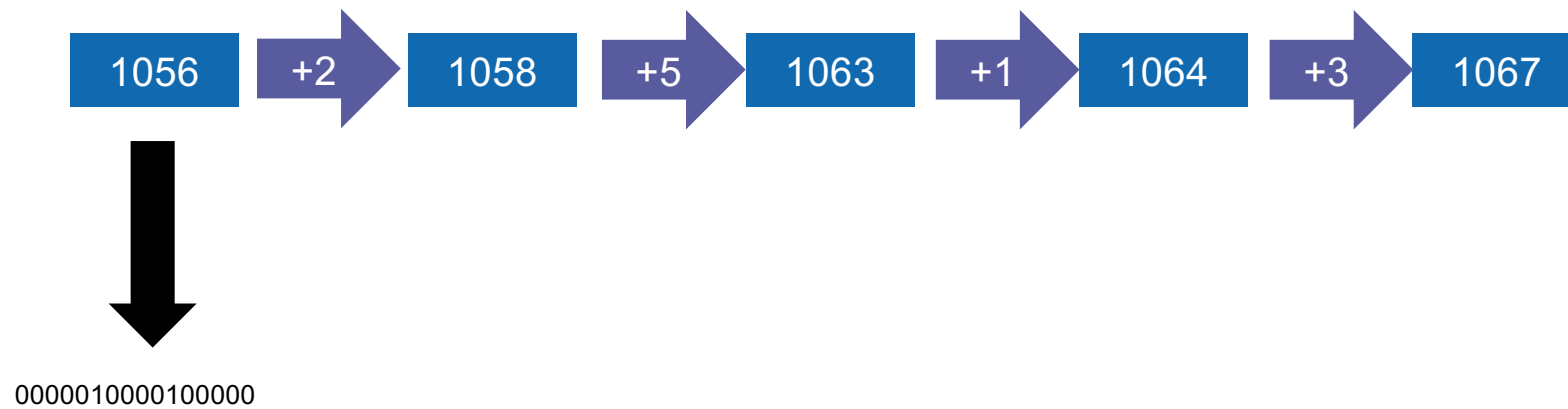
Encoding gaps



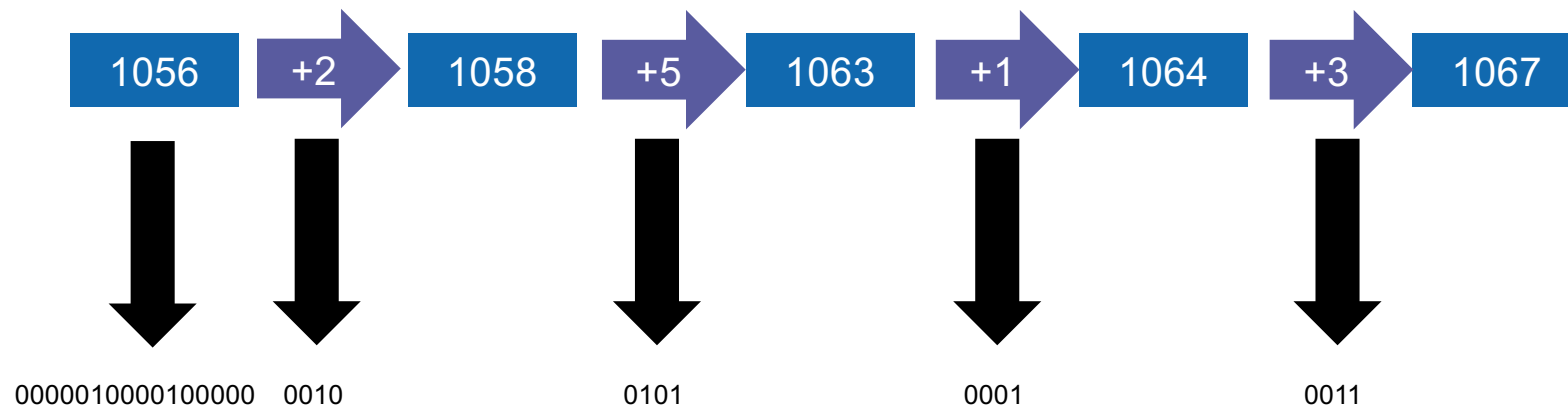
Encoding gaps



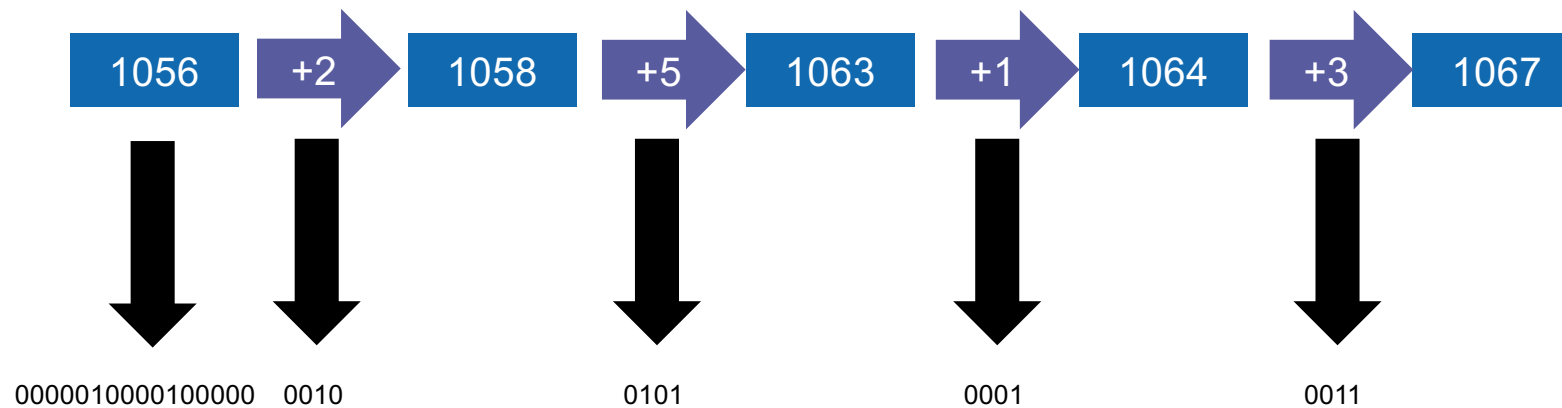
Encoding gaps



Encoding gaps

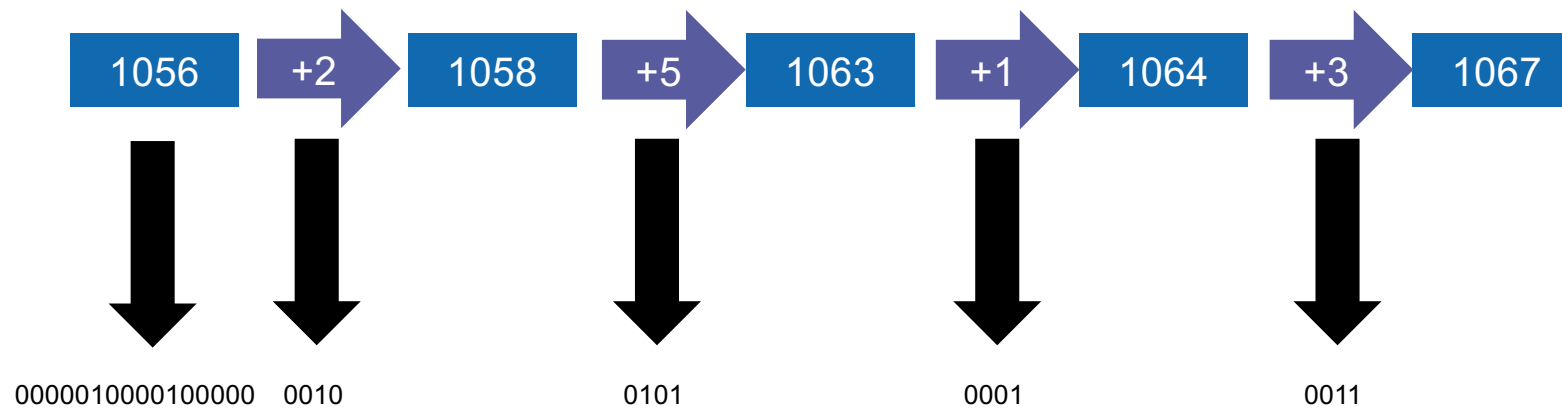


Encoding gaps



But this only works for frequent terms!

Encoding gaps

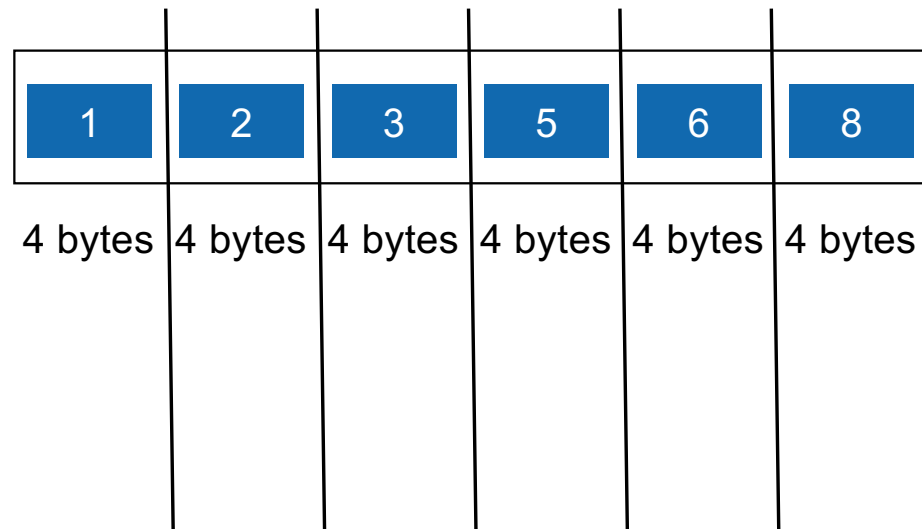


Can we have **variable gap size**?



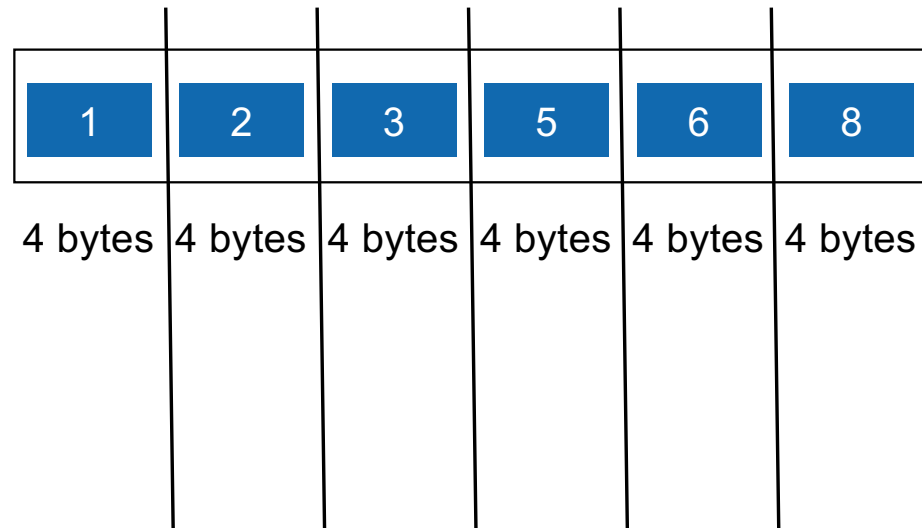
Variable byte encoding

Fix-length encoding



We know exactly where the boundaries are.

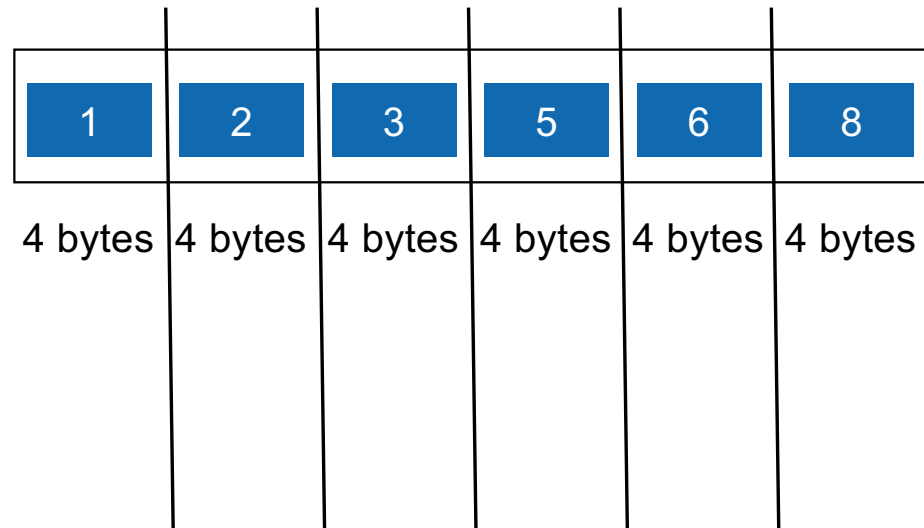
Fix-length encoding



We know exactly where the boundaries are.

0100110101000101010101001010111100100010101001010101010101010...

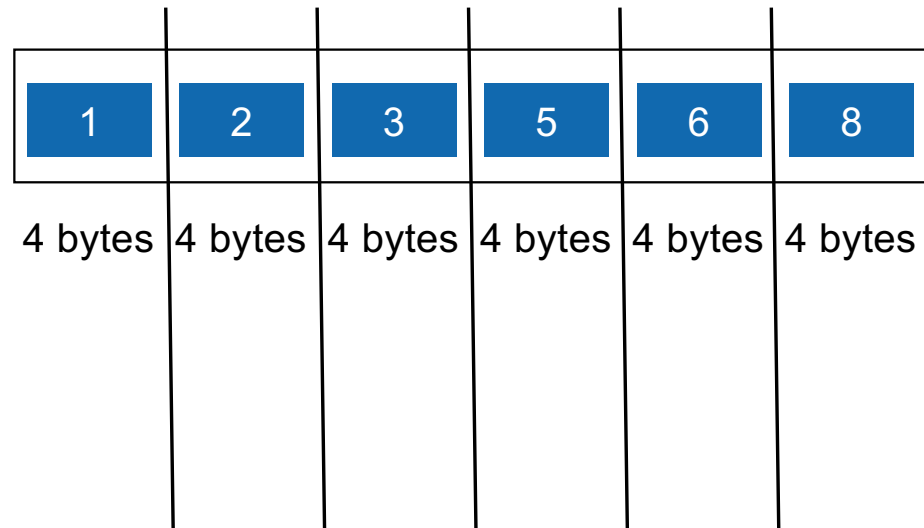
Fix-length encoding



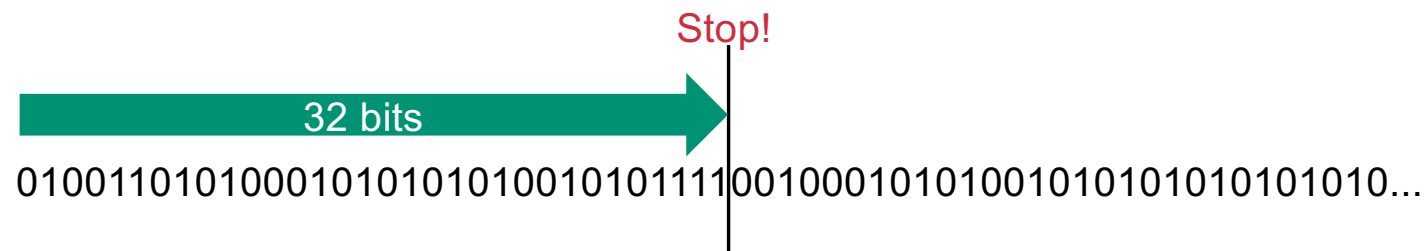
We know exactly where the boundaries are.

32 bits →
010011010100010101010100101011110010001010100101010101010101010...

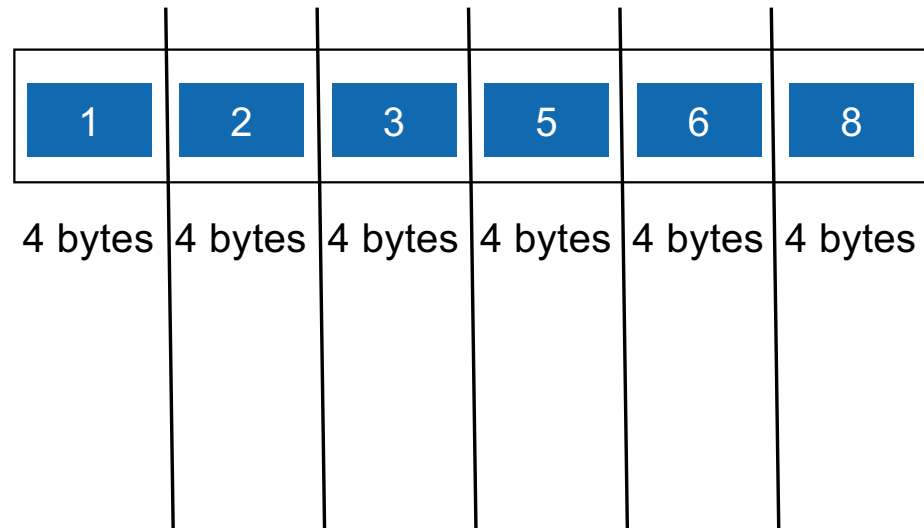
Fix-length encoding



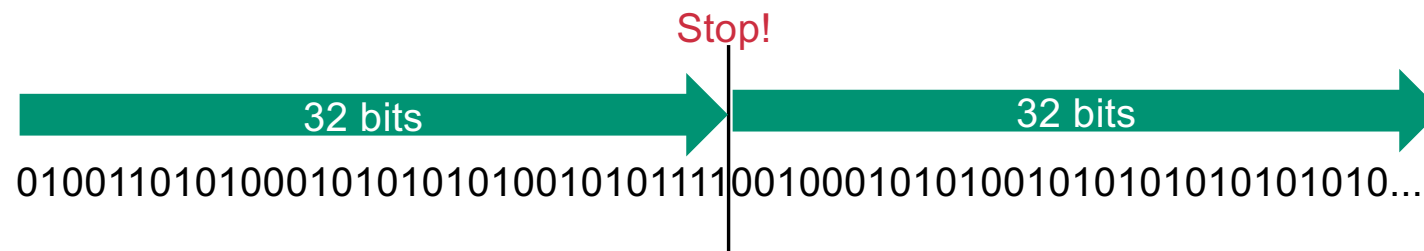
We know exactly where the boundaries are.



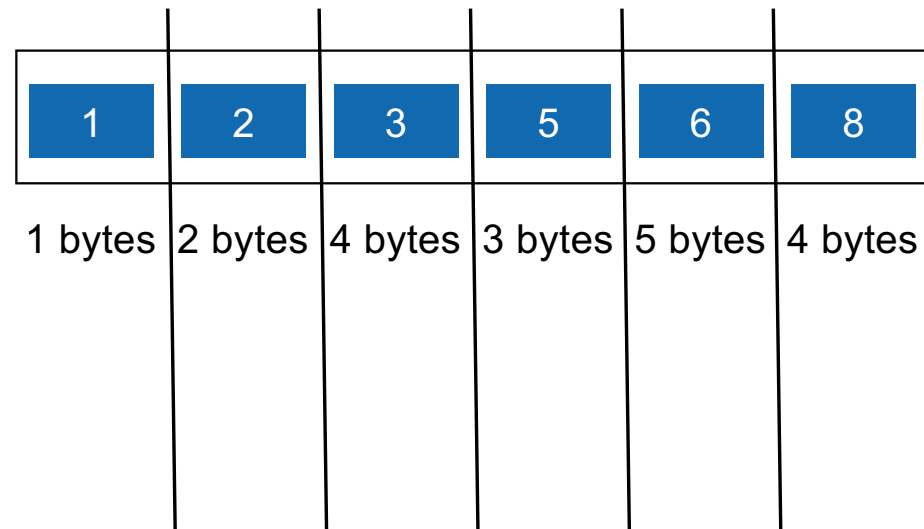
Fix-length encoding



We know exactly where the boundaries are.



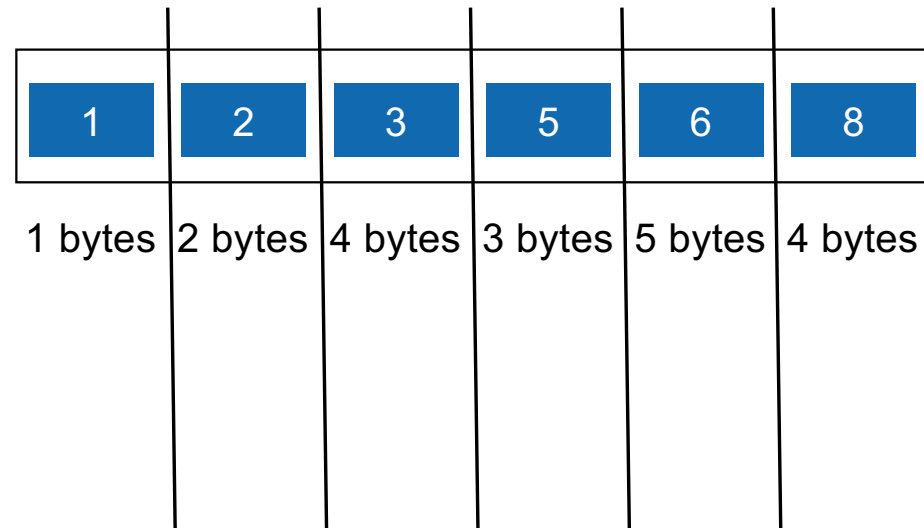
Variable length encodings



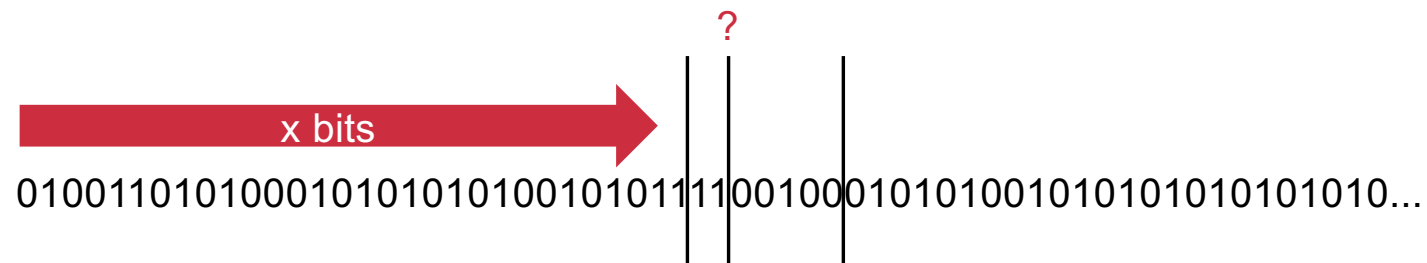
We do not know *a priori* where the boundaries are.

0100110101000101010101001010111100100010101001010101010101010...

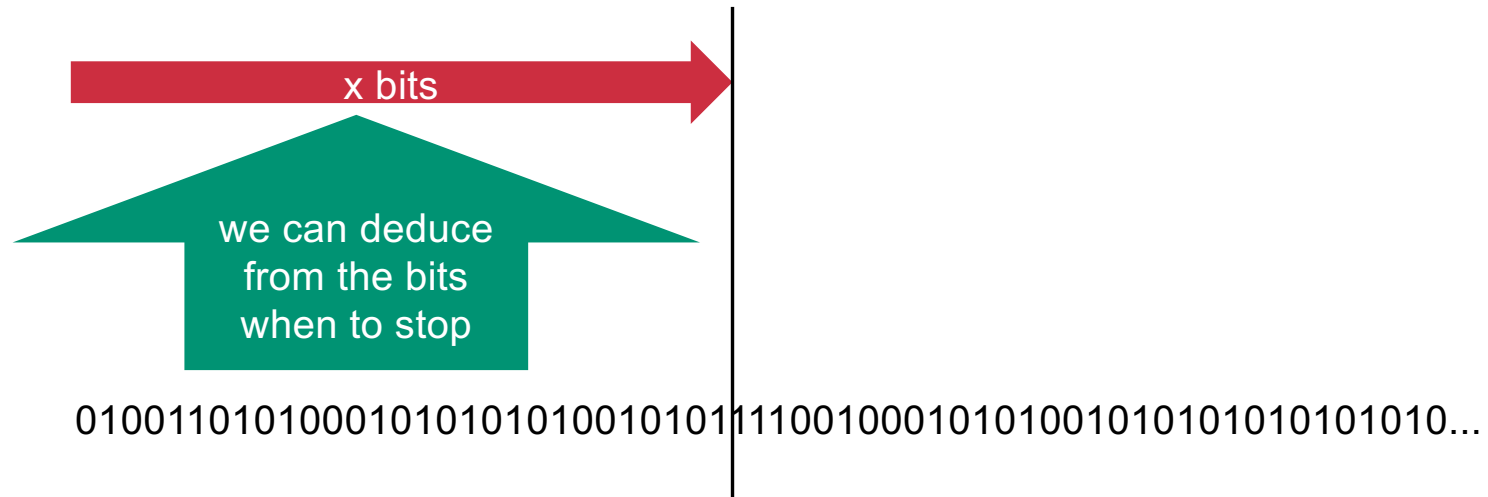
Variable length codings



We do not know *a priori* where the boundaries are.



Prefix codes



Prefix codes: phone numbers

Example

117

0446321111

0016507232300

Internally

888

21111

00446341111

00016507232300

Prefix codes: phone numbers

Example

117

0446321111

0016507232300

Internally

888

21111

00446341111

00016507232300

00165072323001170446321111

Prefix codes: phone numbers

Example

117

0446321111

0016507232300

Internally

888

21111

00446341111

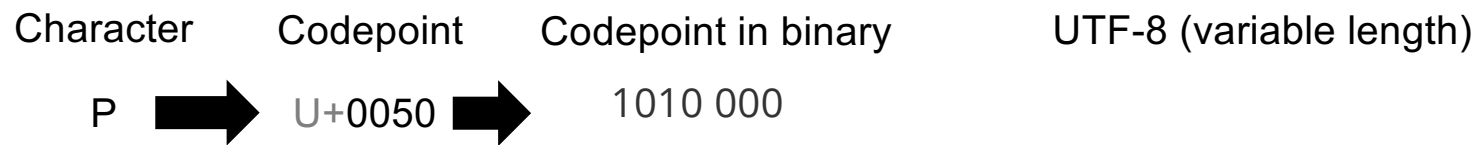
00016507232300

00165072323001170446321111

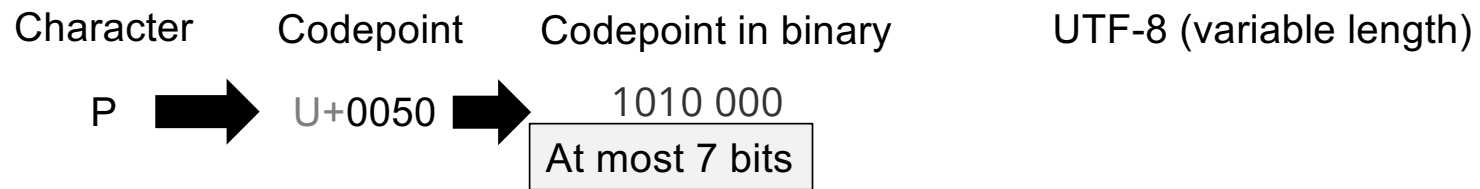
Prefix codes: UTF-8

Character	Codepoint	Codepoint in binary	UTF-8 (variable length)
-----------	-----------	---------------------	-------------------------

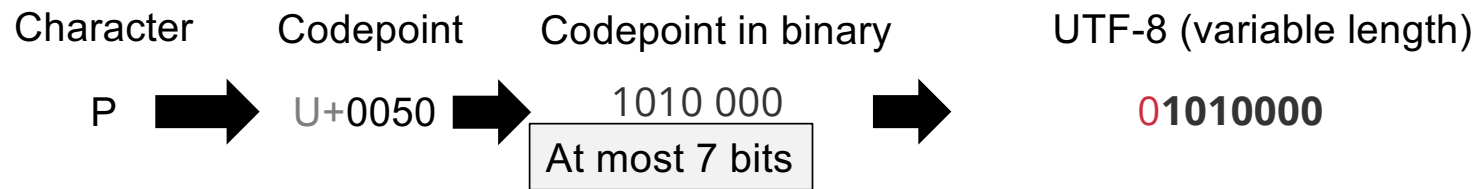
Prefix codes: UTF-8



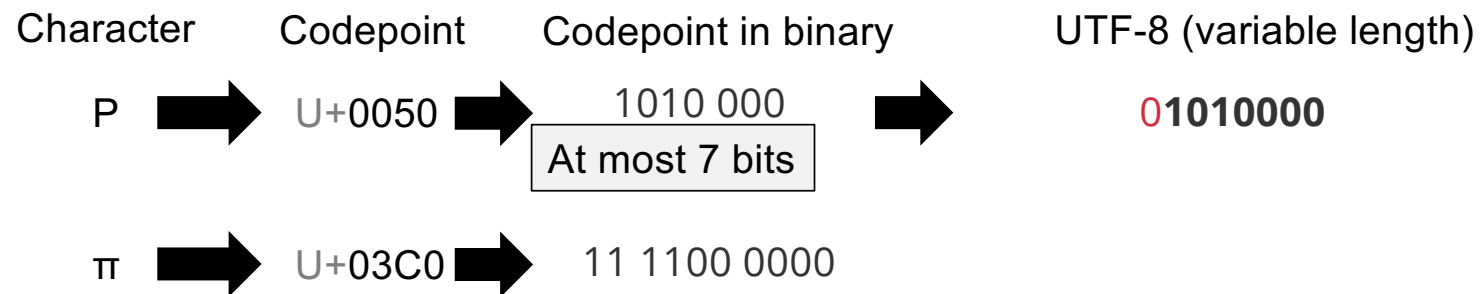
Prefix codes: UTF-8



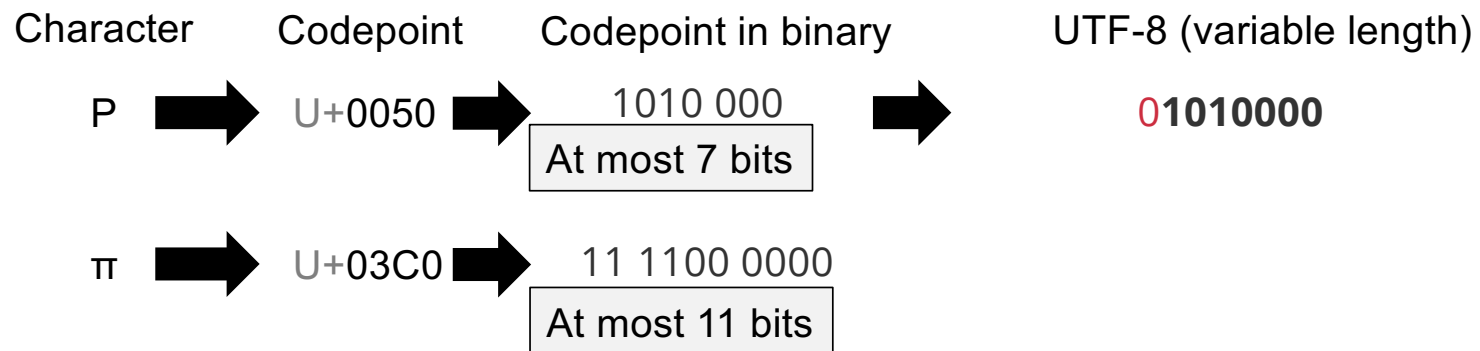
Prefix codes: UTF-8



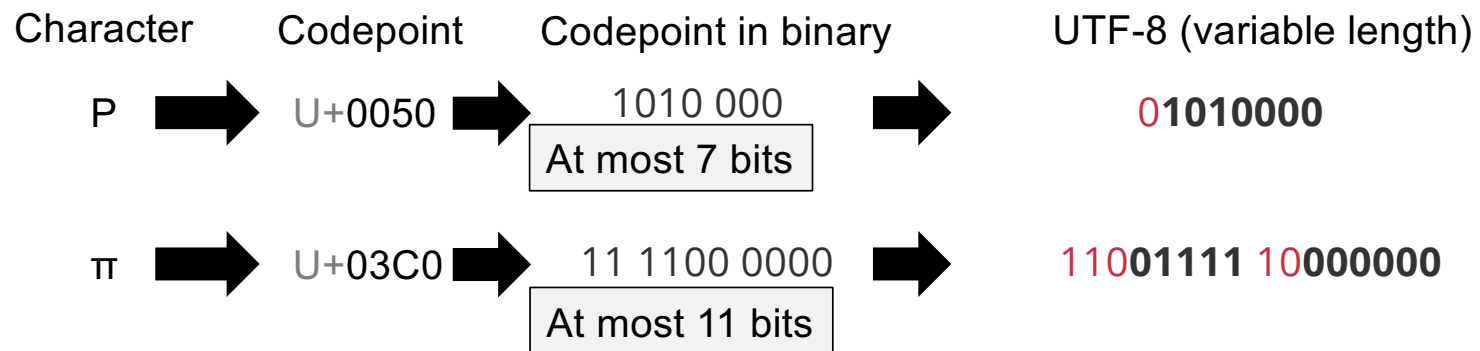
Prefix codes: UTF-8



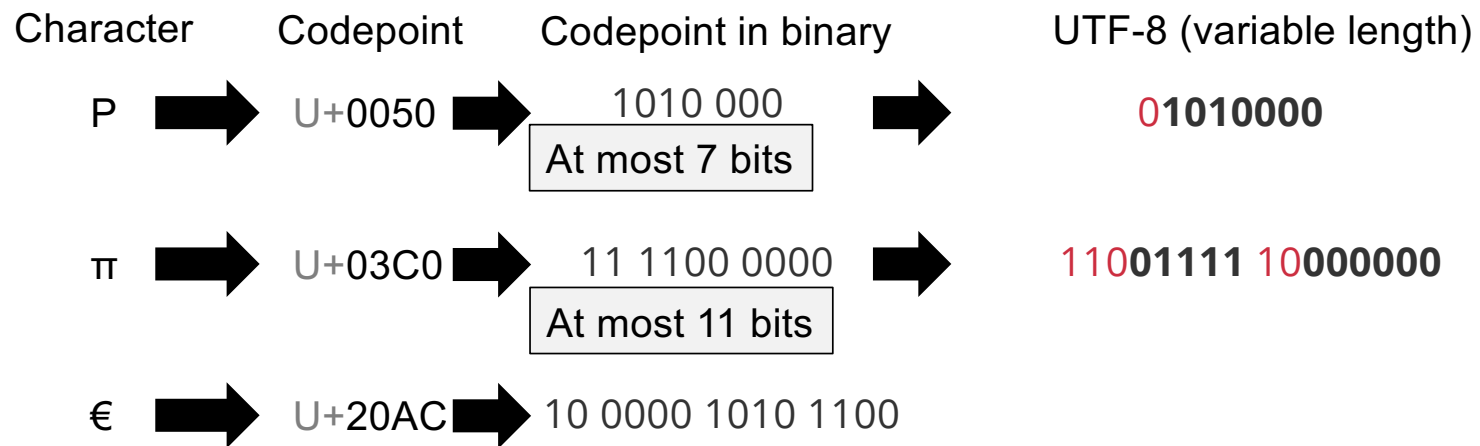
Prefix codes: UTF-8



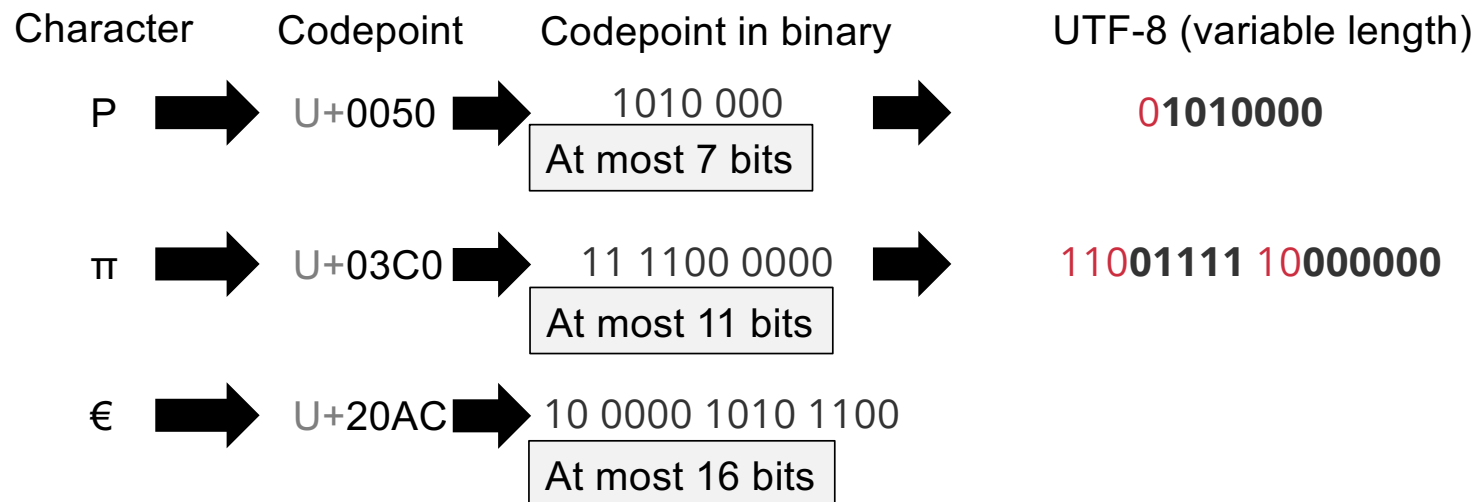
Prefix codes: UTF-8



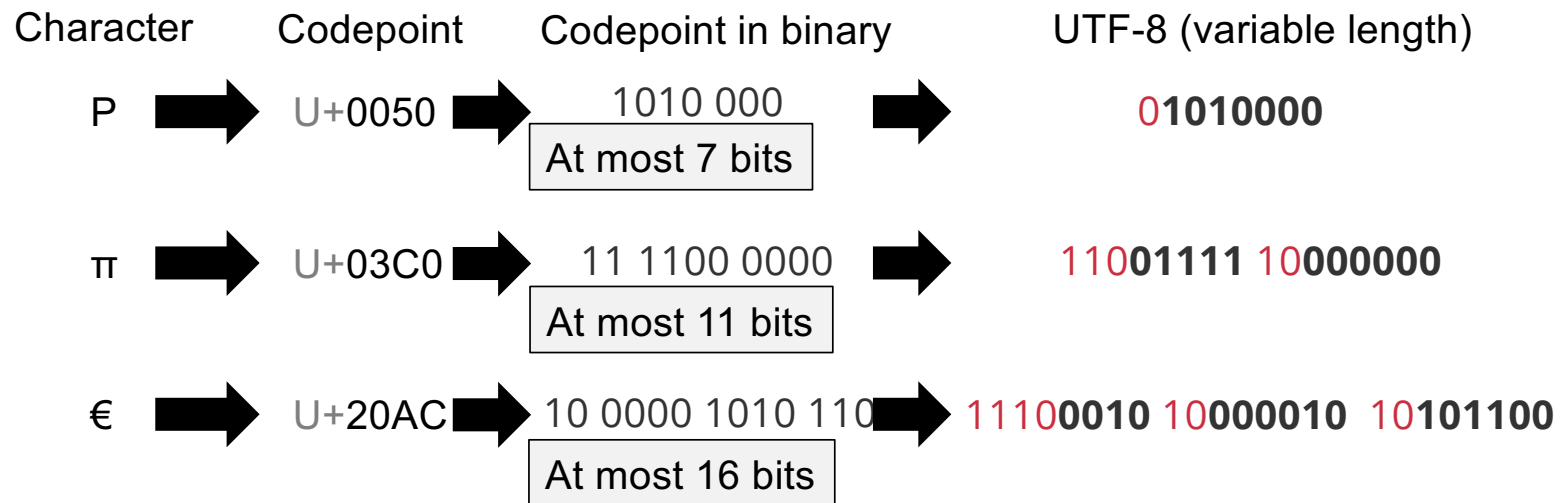
Prefix codes: UTF-8



Prefix codes: UTF-8



Prefix codes: UTF-8



Variable byte encoding (here with 8 bit packets)

00000000

Variable byte encoding (here with 8 bit packets)

00000000
continuation bit encoding on n-1 bits (here 7)

Case 1: less than 7 bits required

4 (100)



Case 1: less than 7 bits required

4 (0000100)



Case 1: less than 7 bits required

4 (0000100)



0000100

Case 1: less than 7 bits required

4 (0000100)



10000100

1 = ends here



Case 2: Between 8 and 14 bits required

270 (1 0000 1110)



Case 2: Between 8 and 14 bits required

270 (00000010 0001110)



Case 2: Between 8 and 14 bits required

270 (00000010 0001110)



00000010 0001110

Case 2: Between 8 and 14 bits required

270 (00000010 0001110)



00000010 10001110

0 = doesn't end here



1 = ends here



And so on and so forth...

0000010 0001110 0101101 0011010 1110101

And so on and so forth...

0000010 0001110 0101101 0011010 1110101



0000010 0001110 0101101 0011010 1110101

And so on and so forth...

0000010 0001110 0101101 0011010 1110101



Variable byte encoding: example with 4 bit packets

decimal	binary	variable byte encoding
0	0	1000

Variable byte encoding: example with 4 bit packets

decimal	binary	variable byte encoding
0	0	1000
1	1	1001
2	10	1010
3	11	1011
4	100	1100
5	101	1101
6	110	1110
7	111	1111
...	...	...

Variable byte encoding: example with 4 bit packets

decimal	binary	variable byte encoding
0	0	1000
1	1	1001
2	10	1010
3	11	1011
4	100	1100
5	101	1101
6	110	1110
7	111	1111
8	1000	0001 1000
9	1001	0001 1001
10	1010	0001 1010
11	1011	0001 1011
12	1100	0001 1100
13	1101	0001 1101
14	1110	0001 1110
15	1111	0001 1111
...	...	...

Variable byte encoding: example with 4 bit packets

decimal	binary	variable byte encoding
0	0	1000
1	1	1001
2	10	1010
3	11	1011
4	100	1100
5	101	1101
6	110	1110
7	111	1111
8	1000	0001 1000
9	1001	0001 1001
10	1010	0001 1010
11	1011	0001 1011
12	1100	0001 1100
13	1101	0001 1101
14	1110	0001 1110
15	1111	0001 1111
16	10000	0010 1000
17	10001	0010 1001
18	10010	0010 1010
19	10011	0010 1011
20	10100	0010 1100
21	10101	0010 1101
22	10110	0010 1110
23	10111	0010 1111
...	...	...

Variable byte encoding: example with 4 bit packets



decimal	binary	variable byte encoding
0	0	1000
1	1	1001
2	10	1010
3	11	1011
4	100	1100
5	101	1101
6	110	1110
7	111	1111
8	1000	0001 1000
9	1001	0001 1001
10	1010	0001 1010
11	1011	0001 1011
12	1100	0001 1100
13	1101	0001 1101
14	1110	0001 1110
15	1111	0001 1111
16	10000	0010 1000
17	10001	0010 1001
18	10010	0010 1010
19	10011	0010 1011
20	10100	0010 1100
21	10101	0010 1101
22	10110	0010 1110
23	10111	0010 1111
...	...	...

Variable byte encoding: example with 4 bit packets



decimal	binary	variable byte encoding
0	0	1000
1	1	1001
2	10	1010
3	11	1011
4	100	1100
5	101	1101
6	110	1110
7	111	1111
8	1000	0001 1000
9	1001	0001 1001
10	1010	0001 1010
11	1011	0001 1011
12	1100	0001 1100
13	1101	0001 1101
14	1110	0001 1110
15	1111	0001 1111
16	10000	0010 1000
17	10001	0010 1001
18	10010	0010 1010
19	10011	0010 1011
20	10100	0010 1100
21	10101	0010 1101
22	10110	0010 1110
23	10111	0010 1111
...	...	...
64	1000000	0001 0000 1000

Variable byte encoding: example with 4 bit packets



decimal	binary	variable byte encoding
0	0	1000
1	1	1001
2	10	1010
3	11	1011
4	100	1100
5	101	1101
6	110	1110
7	111	1111
8	1000	0001 1000
9	1001	0001 1001
10	1010	0001 1010
11	1011	0001 1011
12	1100	0001 1100
13	1101	0001 1101
14	1110	0001 1110
15	1111	0001 1111
16	10000	0010 1000
17	10001	0010 1001
18	10010	0010 1010
19	10011	0010 1011
20	10100	0010 1100
21	10101	0010 1101
22	10110	0010 1110
23	10111	0010 1111
...	...	...
64	1000000	0001 0000 1000

50%
less space

Variable byte encoding is a parameterized encoding

xx	xxxx	xxxxxxxx	xxxxxxxxxxxxxxxx
n=2	n=4	n=8	n=16

Example (here, 8 bits)

100101011101111110010000011100000110100100101100100101000101010111

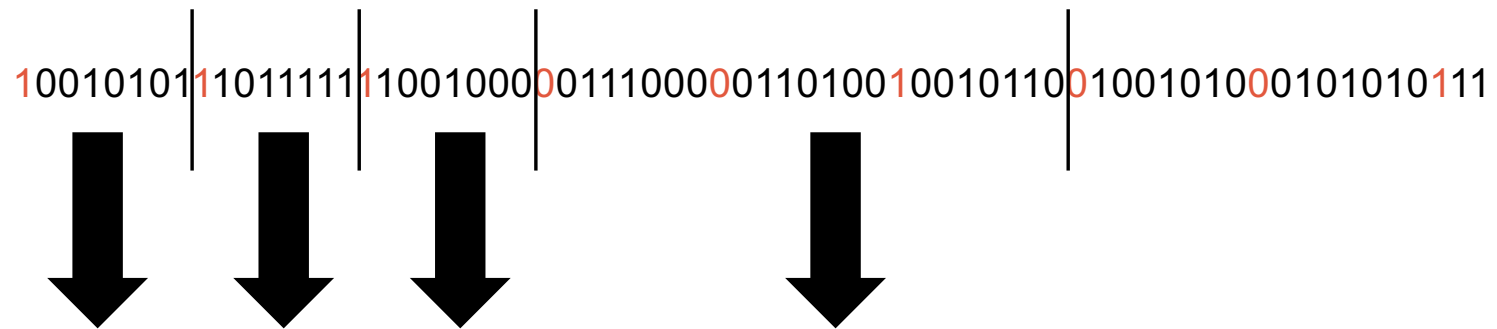
Example (here, 8 bits)

1001010111011111110010000011100000110100100101100100101000101010111

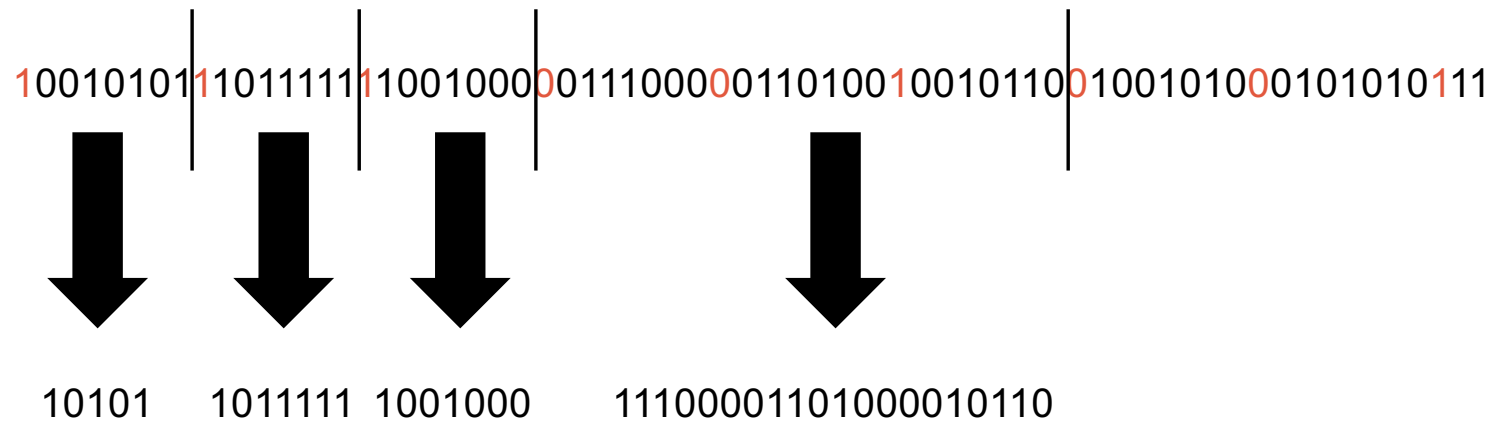
Example (here, 8 bits)

1001010111011111110010000011100000110100100101100100101000101010111

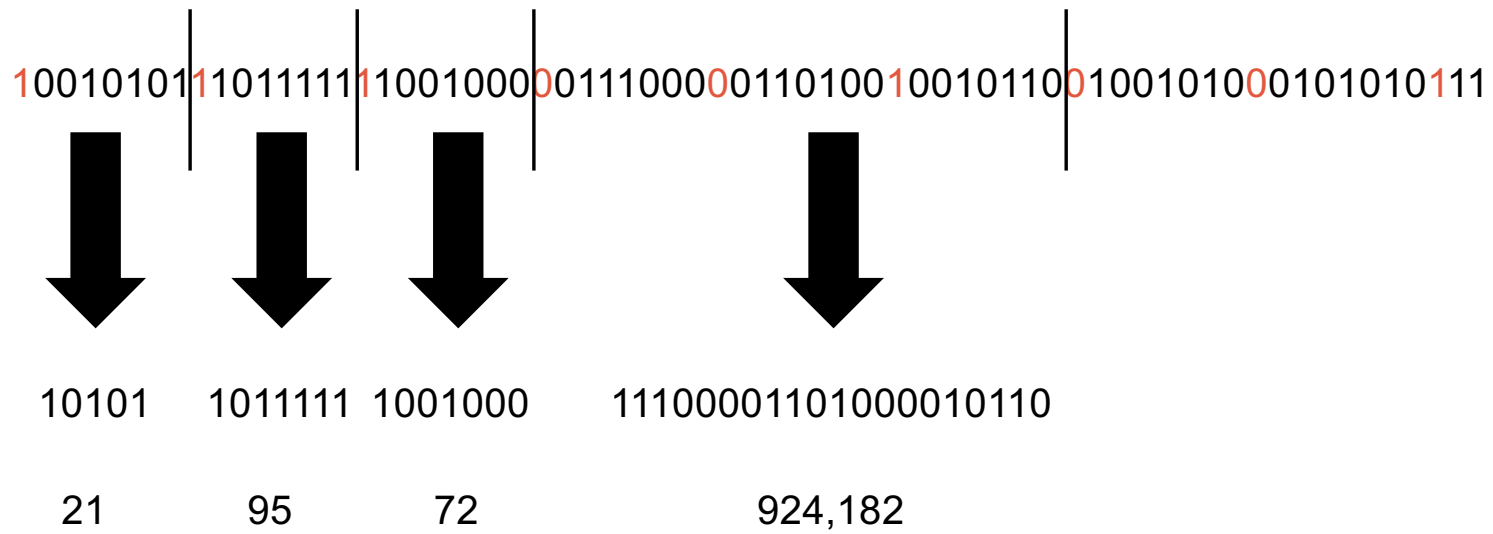
Example (here, 8 bits)



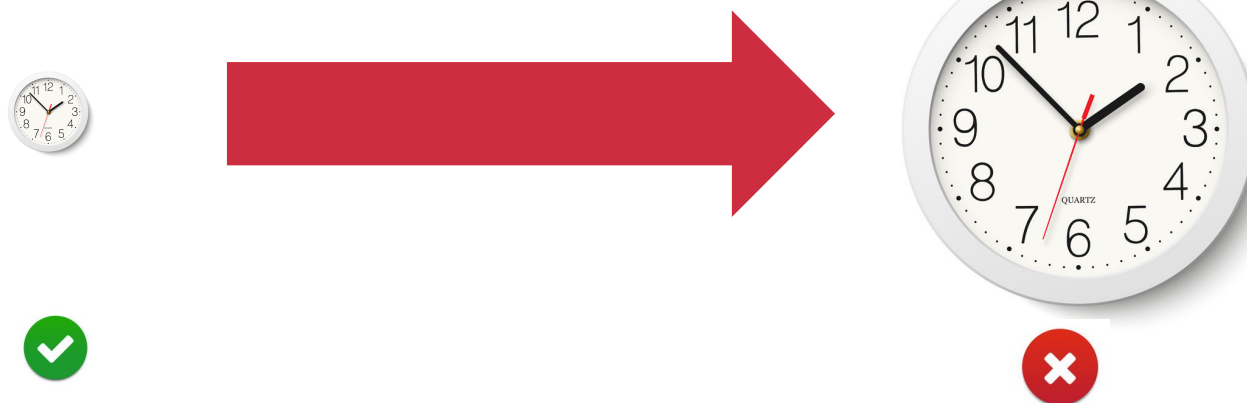
Example (here, 8 bits)



Example (here, 8 bits)



No free lunch



Compromise for variable byte encoding

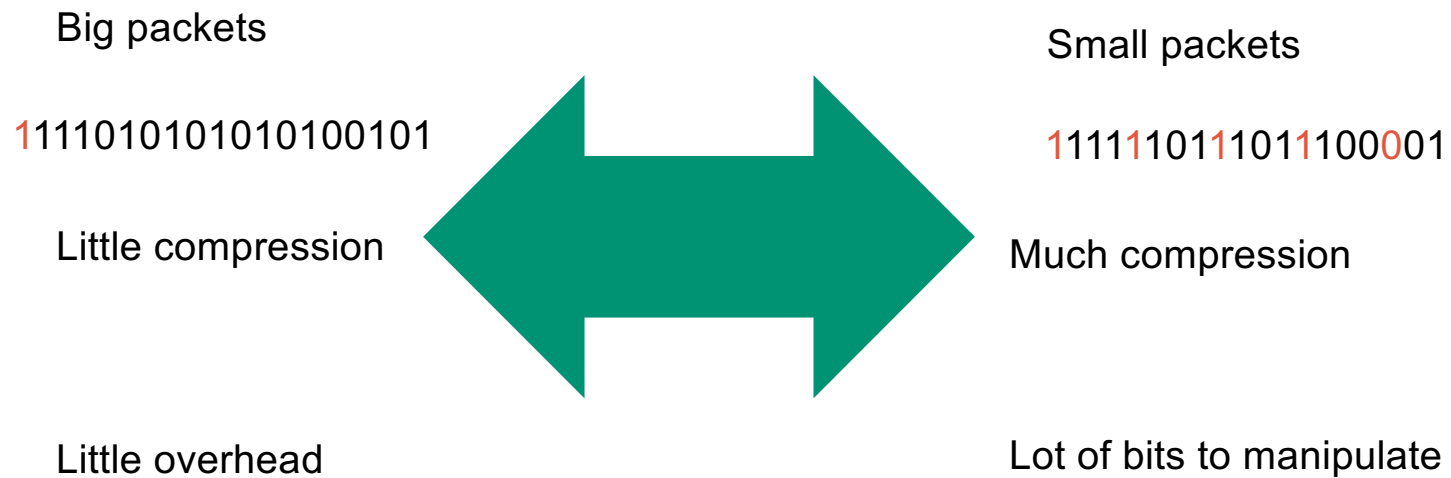
Big packets

1111010101010100101

Little compression

Little overhead

Compromise for variable byte encoding



Can we compress even more?

Can we compress even more?

bitwise?



Gamma encoding

Peter Elias

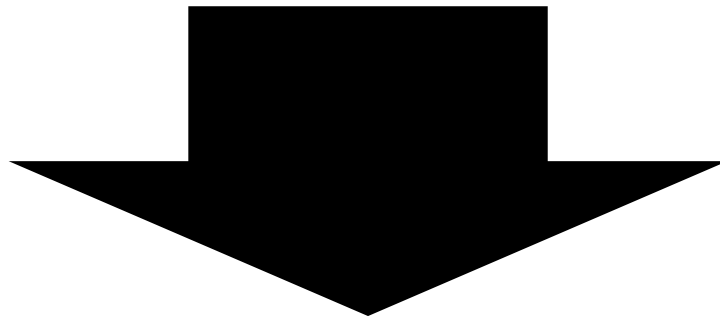


1923 - 2001



Unary code

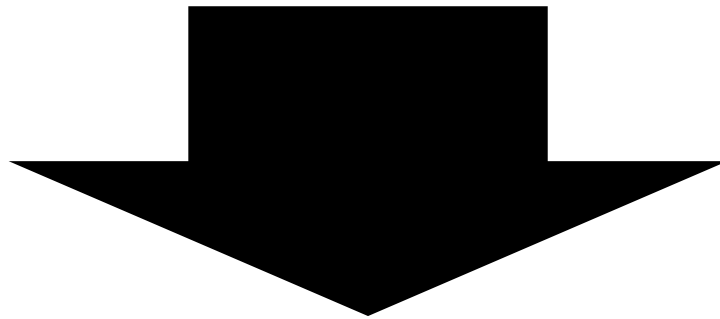
12



1111111111110

Unary code

12

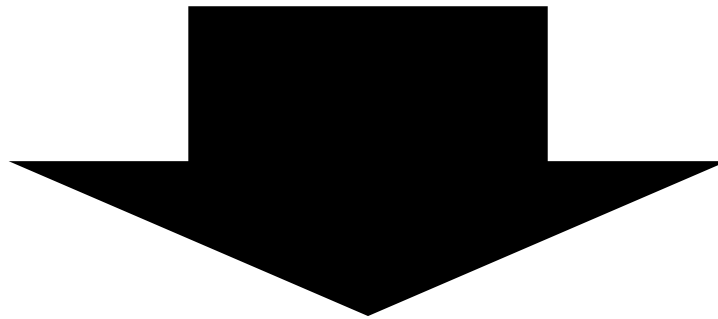


1111111111110

12 ones

Unary code

12



1111111111110

and a zero to mark the stop

First integers in unary code

integer	length (unary)
0	0
1	10
2	110
3	1110
4	11110
5	111110
6	1111110
7	11111110
8	111111110
9	1111111110
...	...

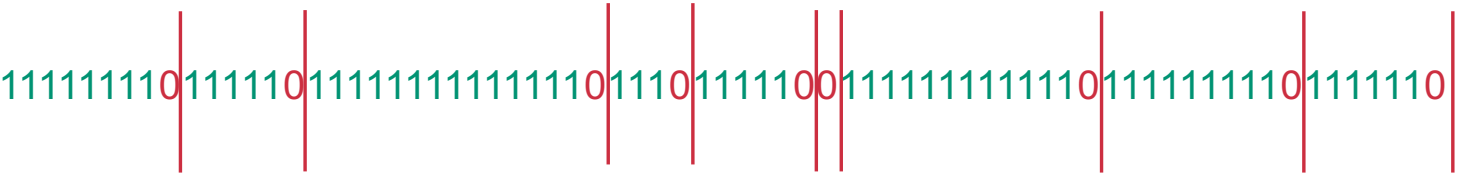
Example (here, 8 bits)

1111111011111011111111111110111011111001111111111101111111101111110

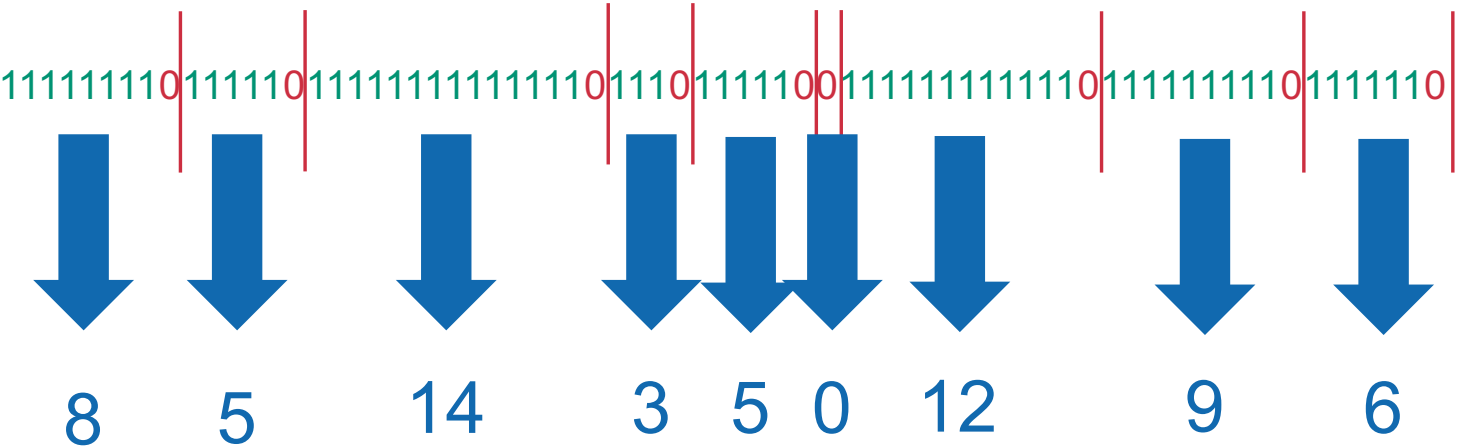
Example (here, 8 bits)

11111111011111011111111111110111011111001111111111101111111101111110

Example (here, 8 bits)



Example (here, 8 bits)



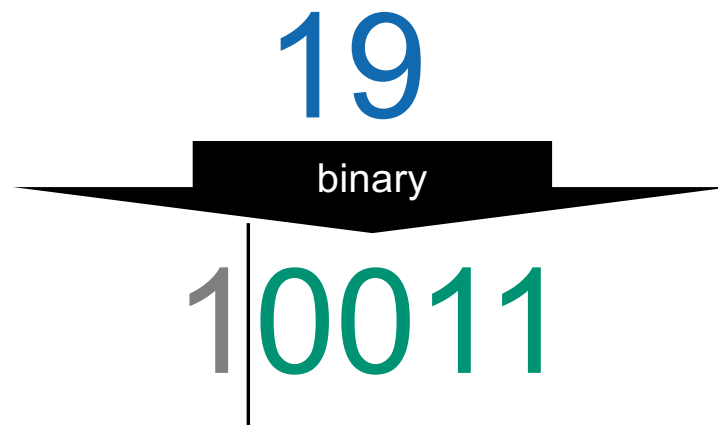
Gamma encoding: example

19

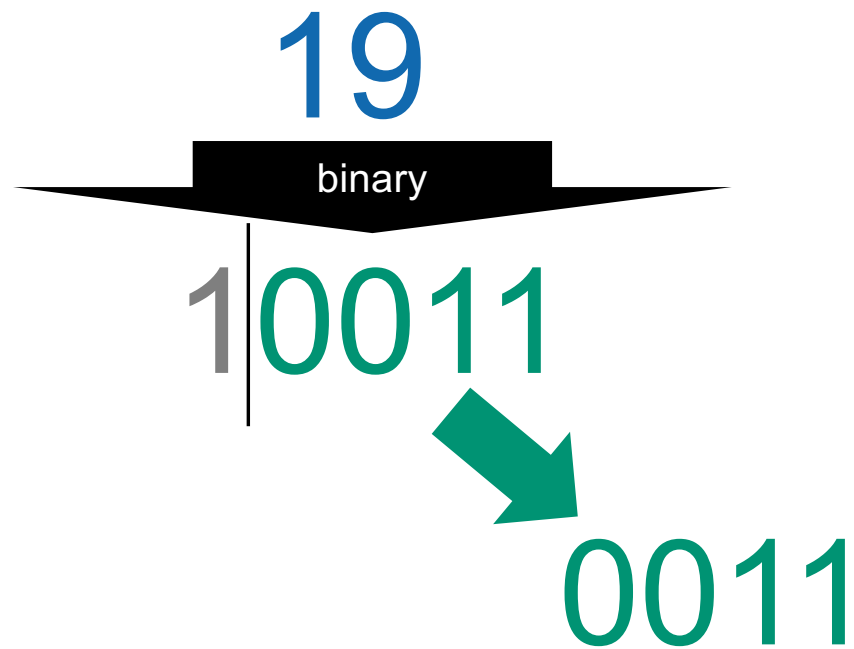
Gamma encoding: example



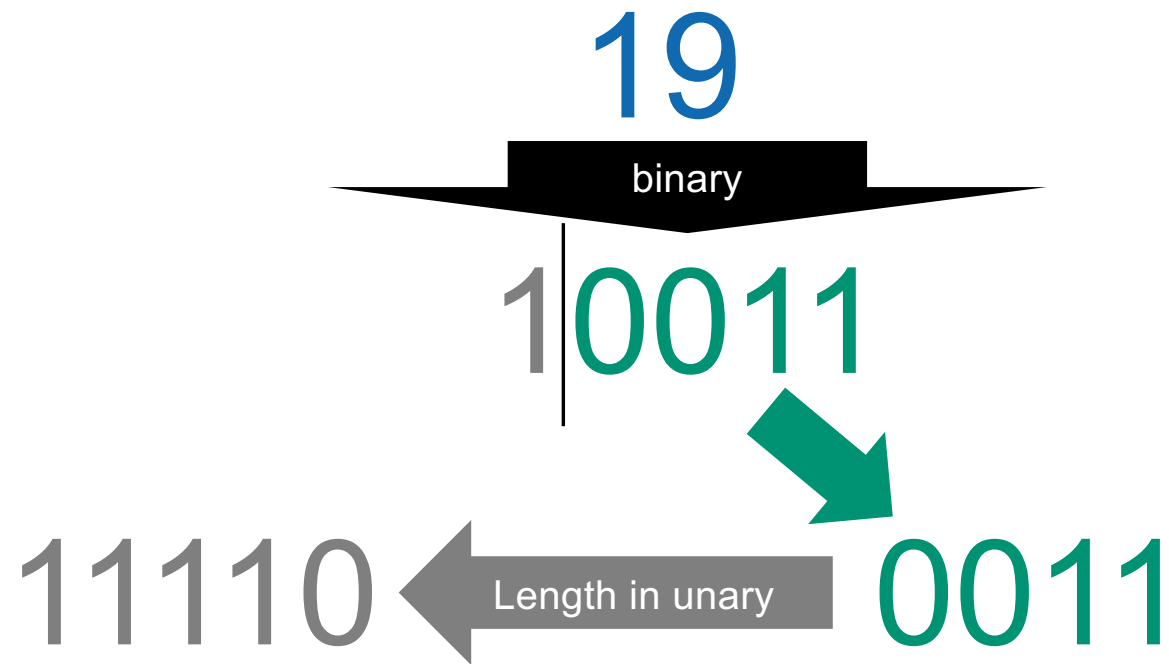
Gamma encoding: example



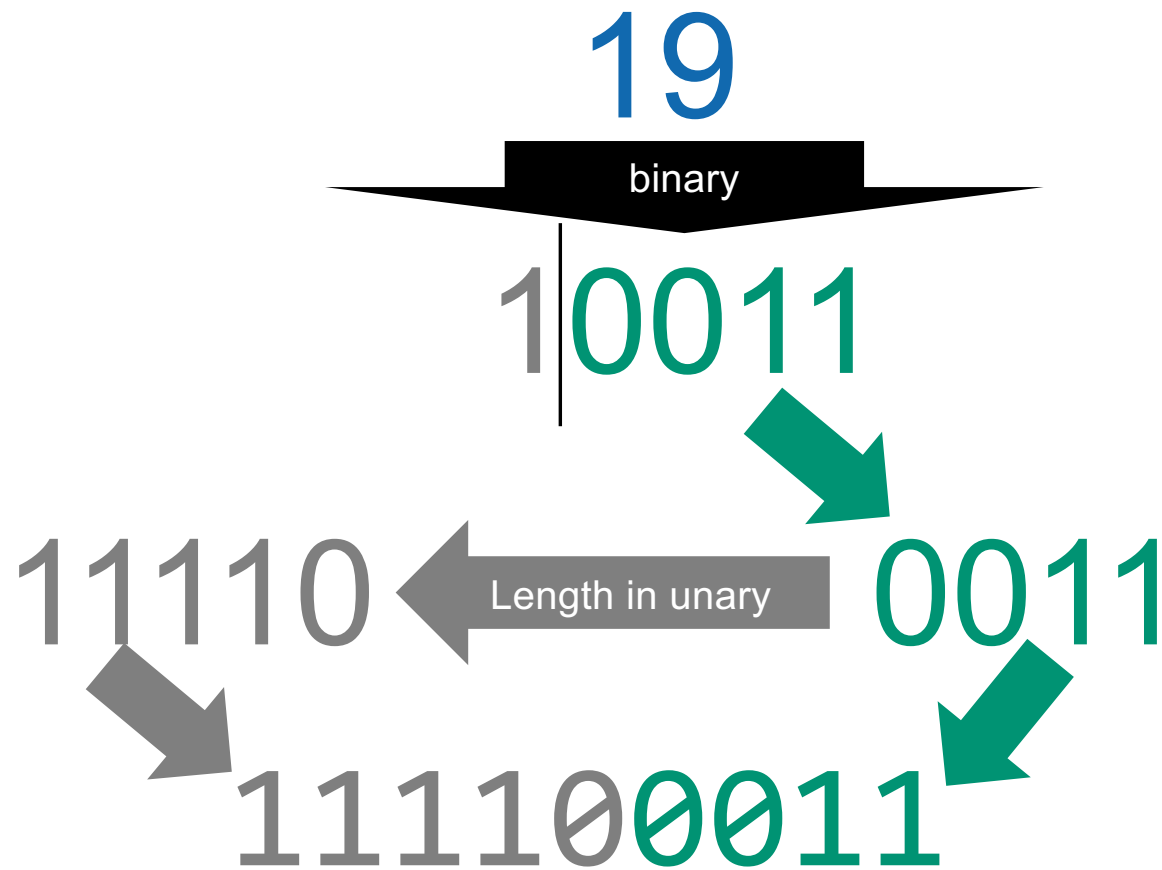
Gamma encoding: example



Gamma encoding: example



Gamma encoding: example



Gamma encoding on the first integers

decimal

0

1

2

3

Gamma encoding on the first integers

decimal	binary
0	0
1	1
2	10
3	11

Gamma encoding on the first integers

decimal	binary	binary without leading 1
0	0	-
1	1	-
2	10	0
3	11	1

Gamma encoding on the first integers

decimal	binary	binary without leading 1	length
0	0	-	0
1	1	-	0
2	10	0	1
3	11	1	1

Gamma encoding on the first integers

decimal	binary	binary without leading 1	length	length (unary)
0	0	-	0	
1	1	-	0	
2	10	0	1	10
3	11	1	1	10

Gamma encoding on the first integers

decimal	binary	binary without leading 1	length	length (unary)	gamma code
0	0	-	0		N/A
1	1	-	0		0
2	10	0	1	10	100
3	11	1	1	10	101

Gamma encoding on the first integers

decimal	binary	binary without leading 1	length	length (unary)	gamma code
0	0	-	0		N/A
1	1	-	0		0
2	10	0	1	10	100
3	11	1	1	10	101
4	100	00	2	110	11000
5	101	01	2	110	11001
6	110	10	2	110	11010
7	111	11	2	110	11011

Gamma encoding on the first integers

decimal	binary	binary without leading 1	length	length (unary)	gamma code
0	0	-	0		N/A
1	1	-	0		0
2	10	0	1	10	100
3	11	1	1	10	101
4	100	00	2	110	11000
5	101	01	2	110	11001
6	110	10	2	110	11010
7	111	11	2	110	11011
8	1000	000	3	1110	1110000
9	1001	001	3	1110	1110001
10	1010	010	3	1110	1110010
11	1011	011	3	1110	1110011
12	1100	100	3	1110	1110100
13	1101	101	3	1110	1110101
14	1110	110	3	1110	1110110
15	1111	111	3	1110	1110111

Gamma encoding on the first integers

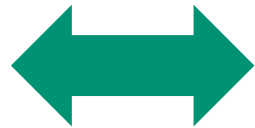
decimal	binary	binary without leading 1	length	length (unary)	gamma code
0	0	-	0		N/A
1	1	-	0		0
2	10	0	1	10	100
3	11	1	1	10	101
4	100	00	2	110	11000
5	101	01	2	110	11001
6	110	10	2	110	11010
7	111	11	2	110	11011
8	1000	000	3	1110	1110000
9	1001	001	3	1110	1110001
10	1010	010	3	1110	1110010
11	1011	011	3	1110	1110011
12	1100	100	3	1110	1110100
13	1101	101	3	1110	1110101
14	1110	110	3	1110	1110110
15	1111	111	3	1110	1110111
16	10000	0000	4	11110	111100000
17	10001	0001	4	11110	111100001
18	10010	0010	4	11110	111100010
19	10011	0011	4	11110	111100011
20	10100	0100	4	11110	111100100
21	10101	0101	4	11110	111100101
22	10110	0110	4	11110	111100110
23	10111	0111	4	11110	111100111
...	...	...	...	...	...
64	1000000	000000	6	1111110	1111110000000

Gamma encoding properties



Variable length encoding

Gamma encoding properties



Variable length encoding



Prefix encoding

Gamma encoding properties



Variable length encoding

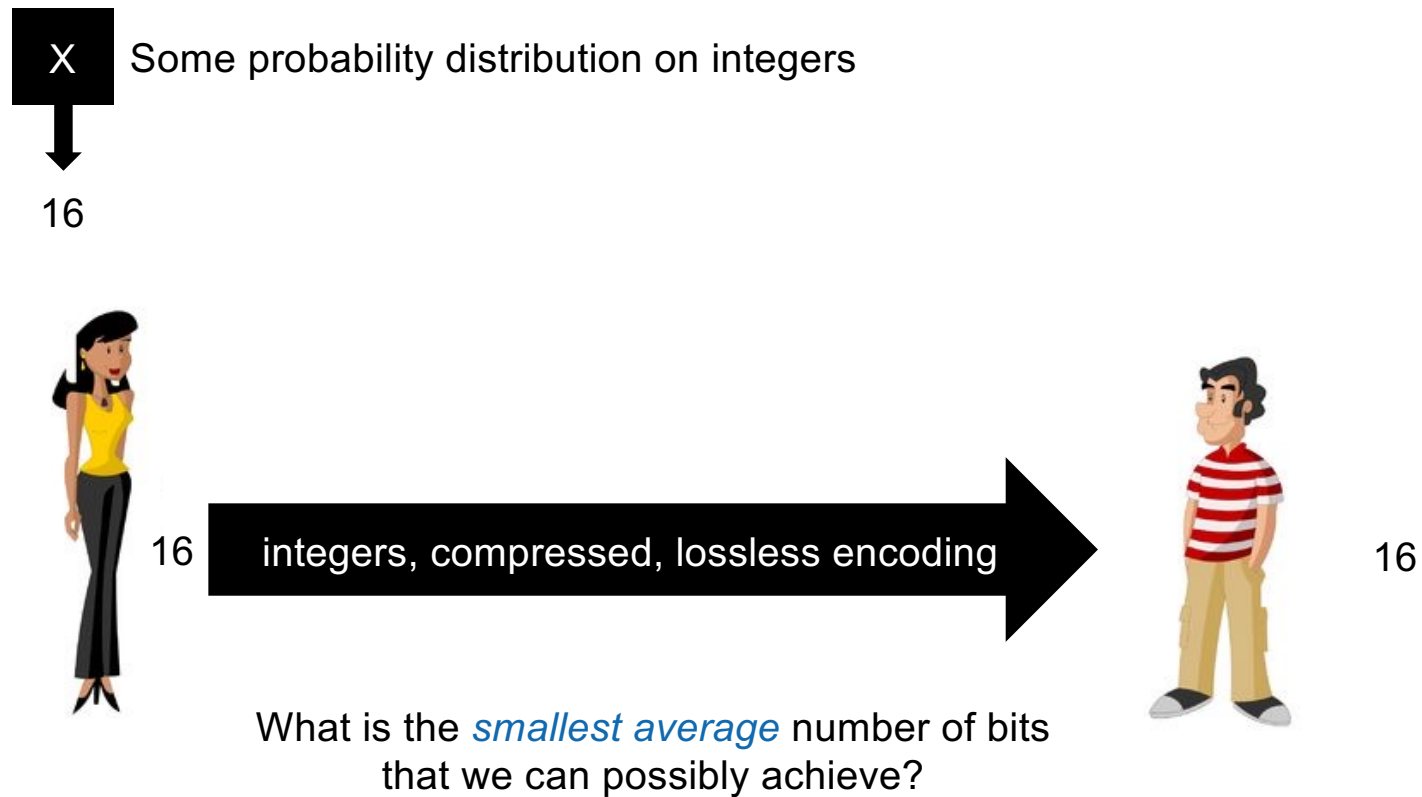


Prefix encoding



Universal encoding

Shannon Entropy



Shannon Entropy

$$H(X) = \mathbb{E}[I(X)]$$

Shannon Entropy

$$H(X) = \mathbb{E}[\underline{I(X)}]$$

"Amount of information" = number of bits

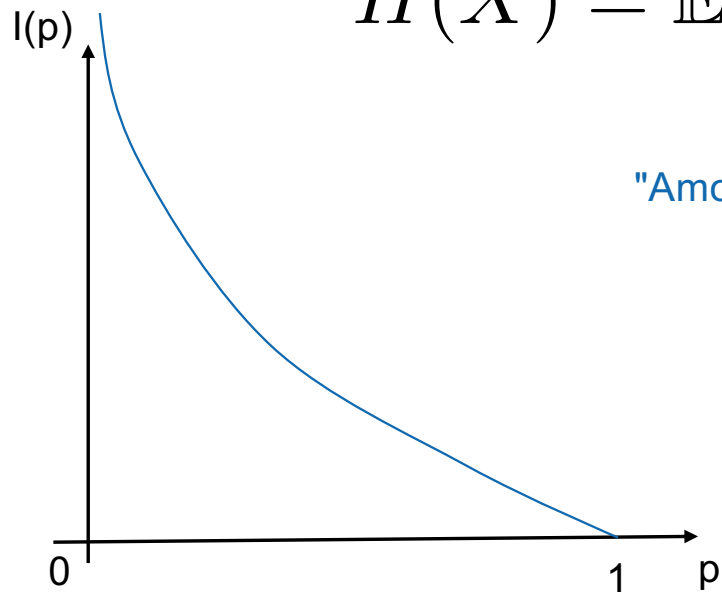
Shannon Entropy

$$H(X) = \mathbb{E}[\underline{I(X)}]$$

"Amount of information" = number of bits

Shannon Entropy

$$H(X) = \mathbb{E}[\underline{-\log_2(p_X(X))}]$$

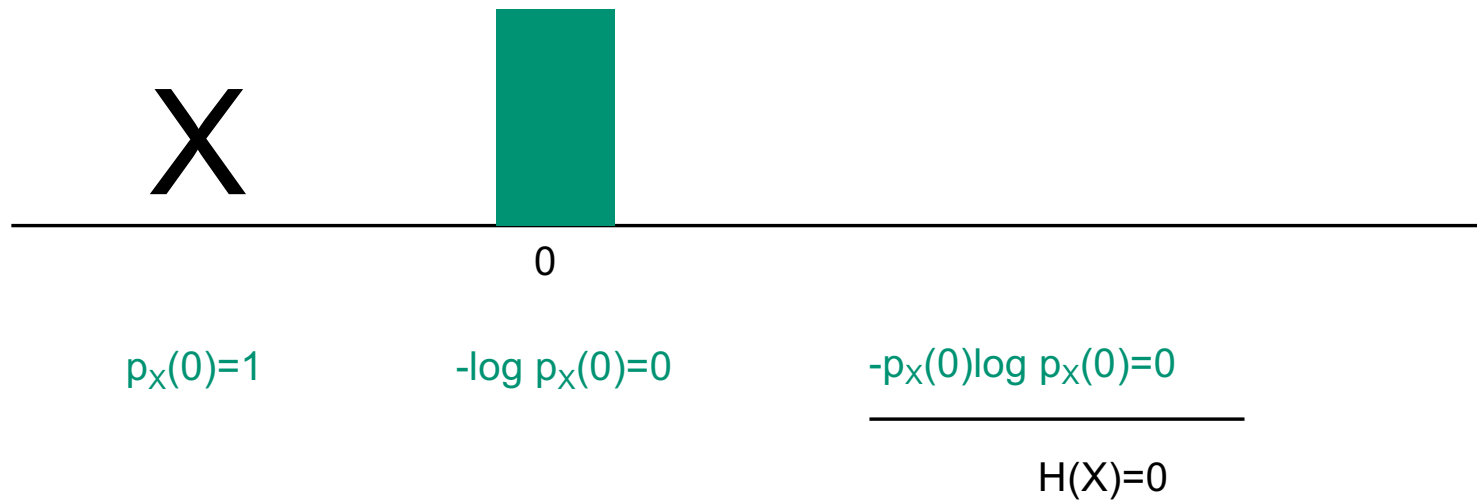


"Amount of information" = number of bits

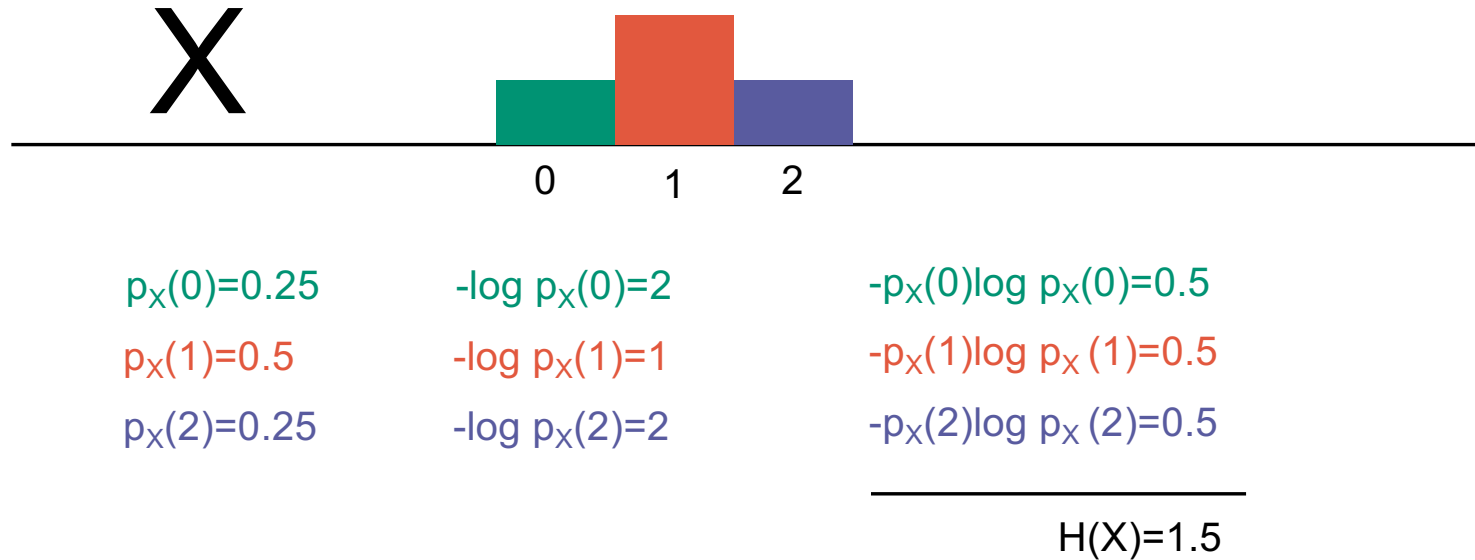
Shannon Entropy

$$H(X) = \mathbb{E}[I(X)] = - \sum_{x \in X(\Omega)} p_X(x) \log_2 p_X(x)$$

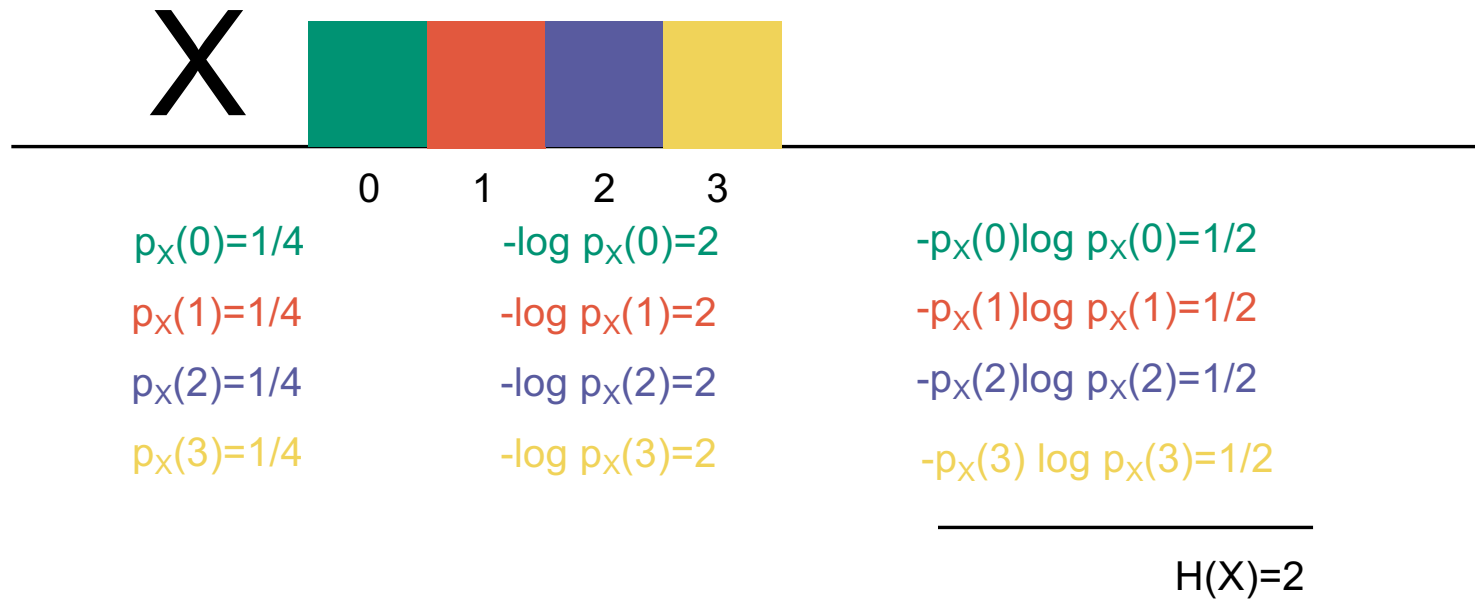
Entropy of deterministic distribution



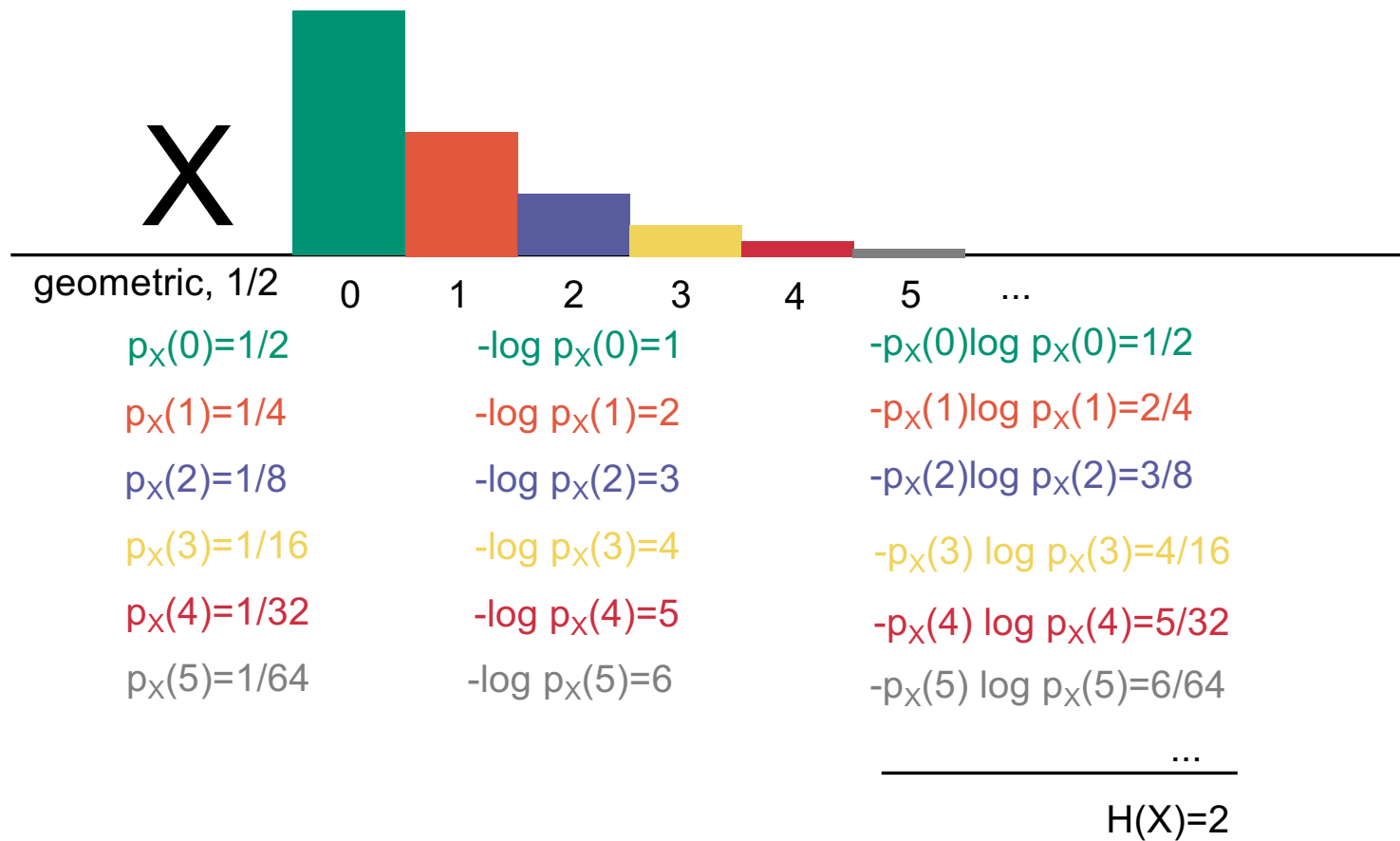
Entropy of binomial distribution (2, $\frac{1}{2}$), offset -1



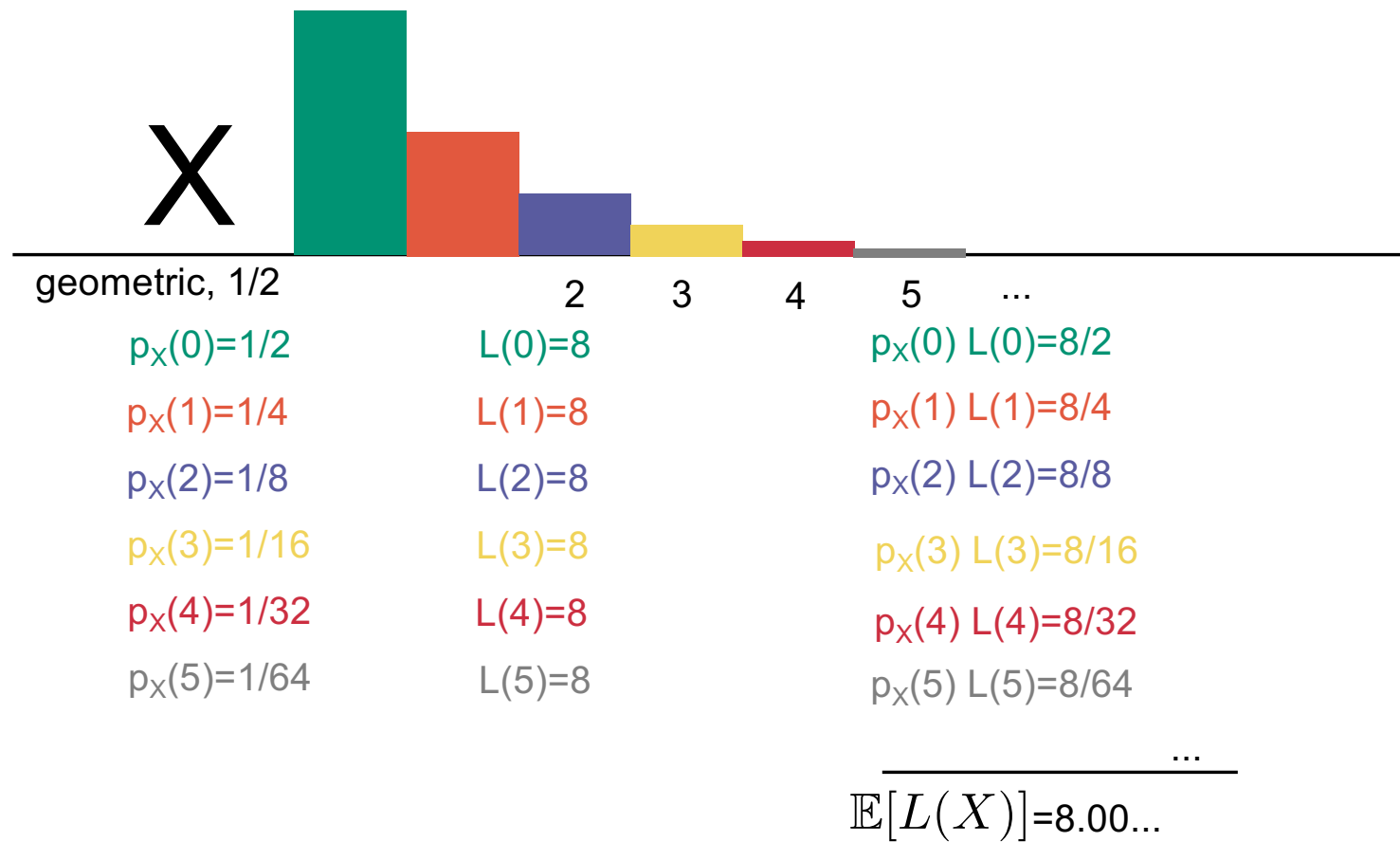
Entropy of uniform distribution



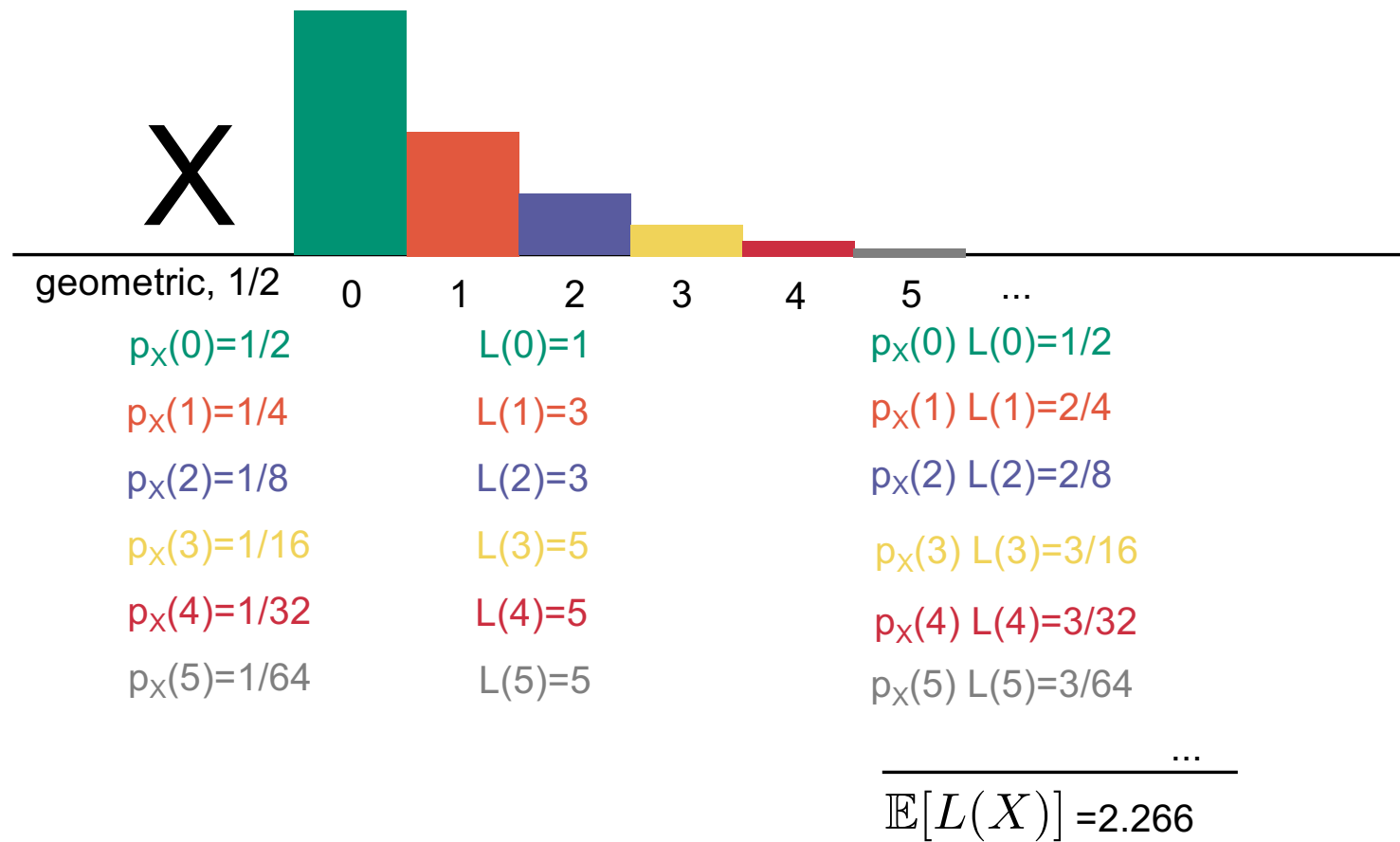
Entropy of geometric distribution ($\frac{1}{2}$)



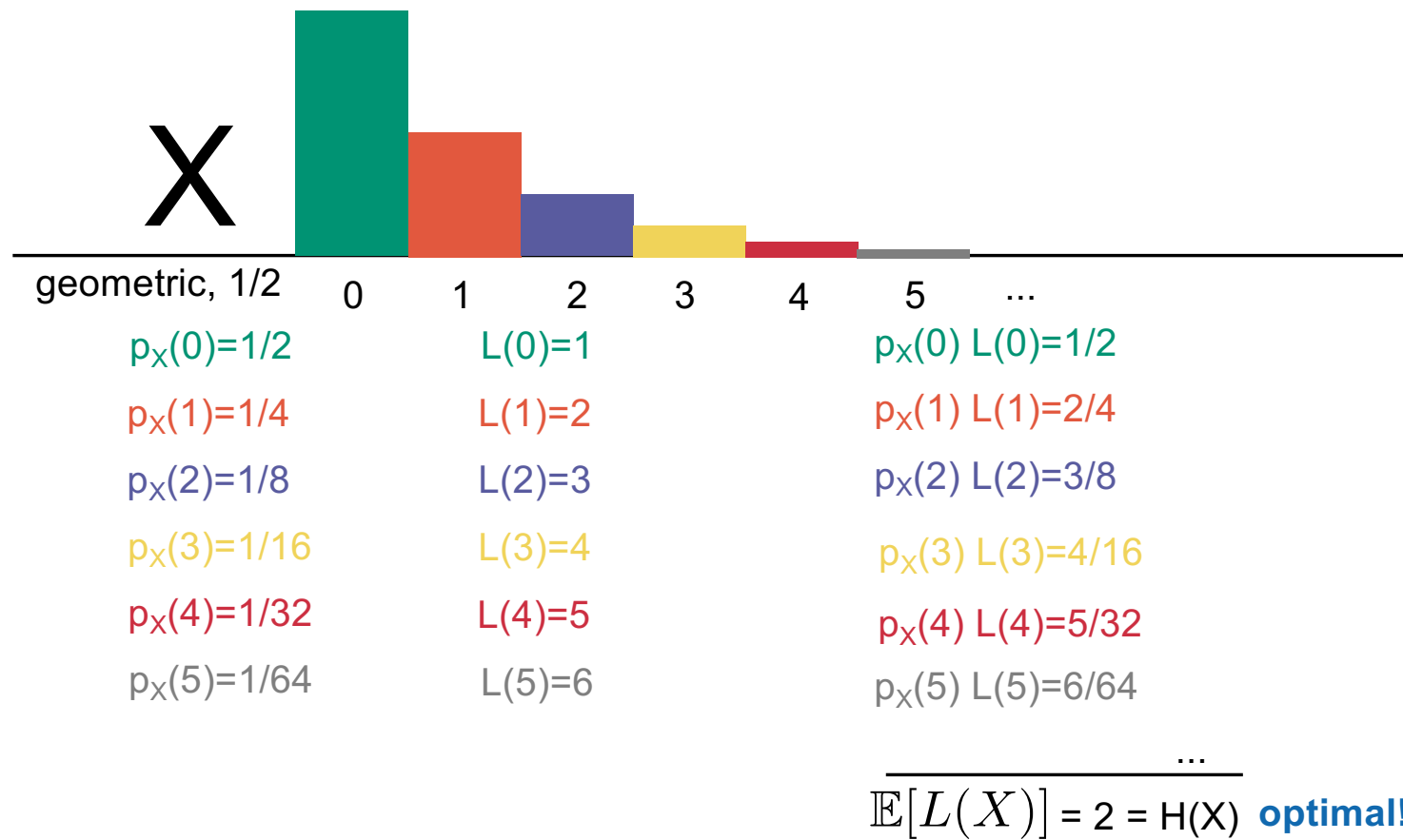
Average length of variable-byte encoding (8-bit)



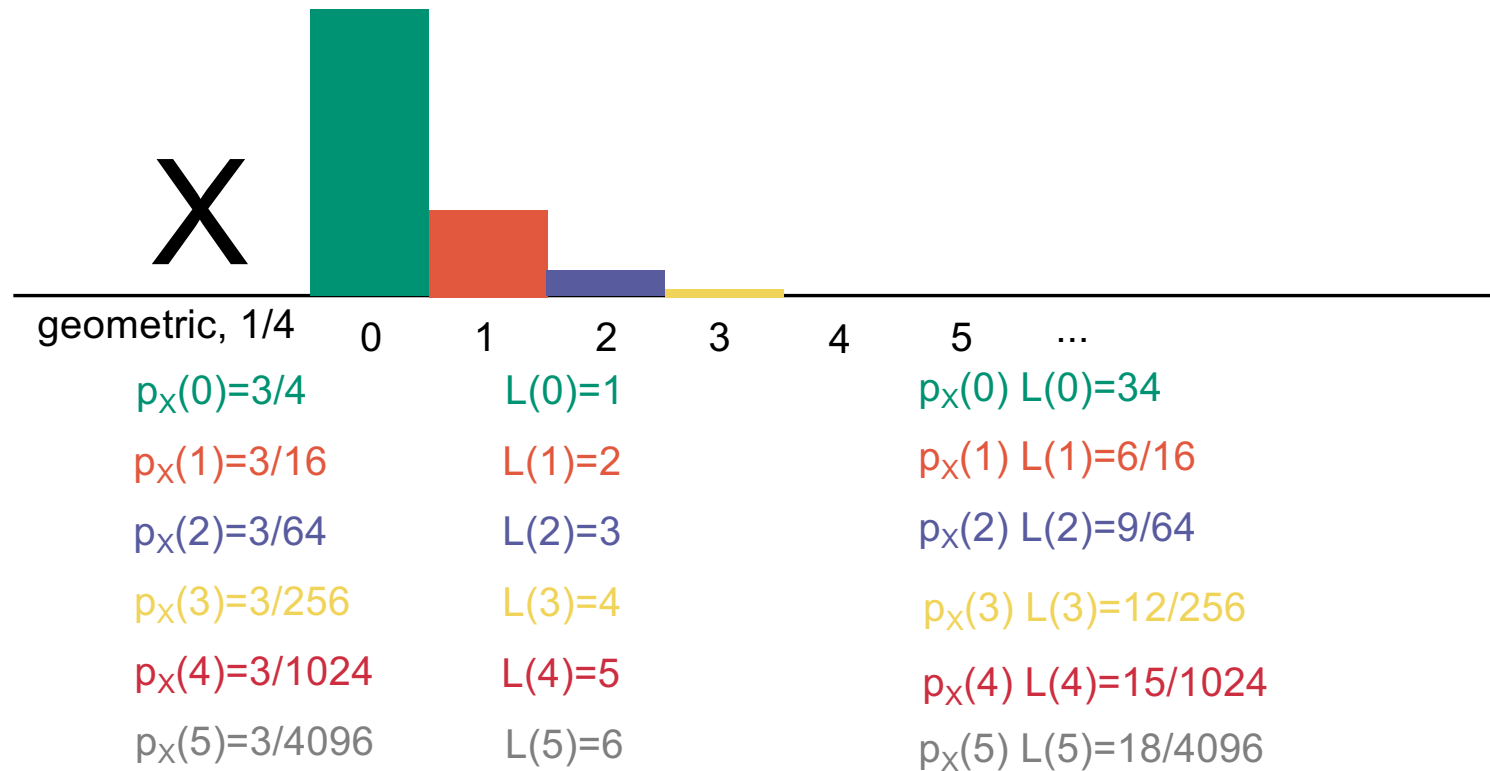
Average length of Gamma encoding (offset -1)



Average length of unary encoding



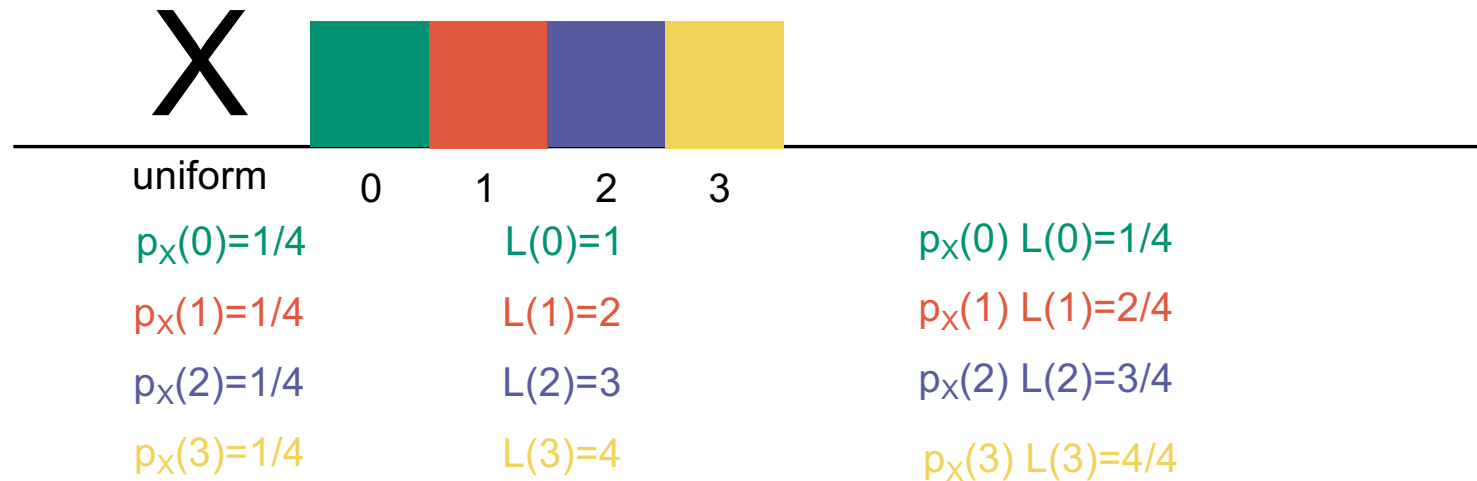
Average length of unary encoding



This is true for any value of p !

$$\mathbb{E}[L(X)] = 1.34 = H(X) \quad \text{optimal!}$$

Average length of unary encoding



$$\overline{\mathbb{E}[L(X)]} = 2.5 > H(X)$$

Average length of unary encoding



$$\mathbb{E}[L(X)] = 500000000.5$$

Expected length of gamma encoding

for *any* distribution

$$\mathbb{E}[L_\gamma(X)] \leq \underline{3}H(X) = 3\mathbb{E}[I(X)]$$

one factor from optimal!

Expected length of gamma encoding

How much can we compress the inverted index?

Zipf's law (renormalized)

$$\text{Renormalized frequency} = \frac{c}{\textit{Rank}}$$

Zipf's law (renormalized)

$$\text{Renormalized frequency} = \frac{c}{Rank}$$

$$\sum_{i=1}^{i=M} \frac{c}{Rank} = 1$$

Zipf's law

$$\text{Number of occurrences per document} = \frac{\text{Document length} \times c}{\text{Rank}}$$

Zipf's law

$$\text{Number of occurrences per document} = \frac{Lc}{\text{Rank}}$$

Zipf's law



Number of postings = Number of documents $\times$ Number of occurrences per documents

Zipf's law



$$\text{Number of postings} = \frac{NLc}{Rank}$$

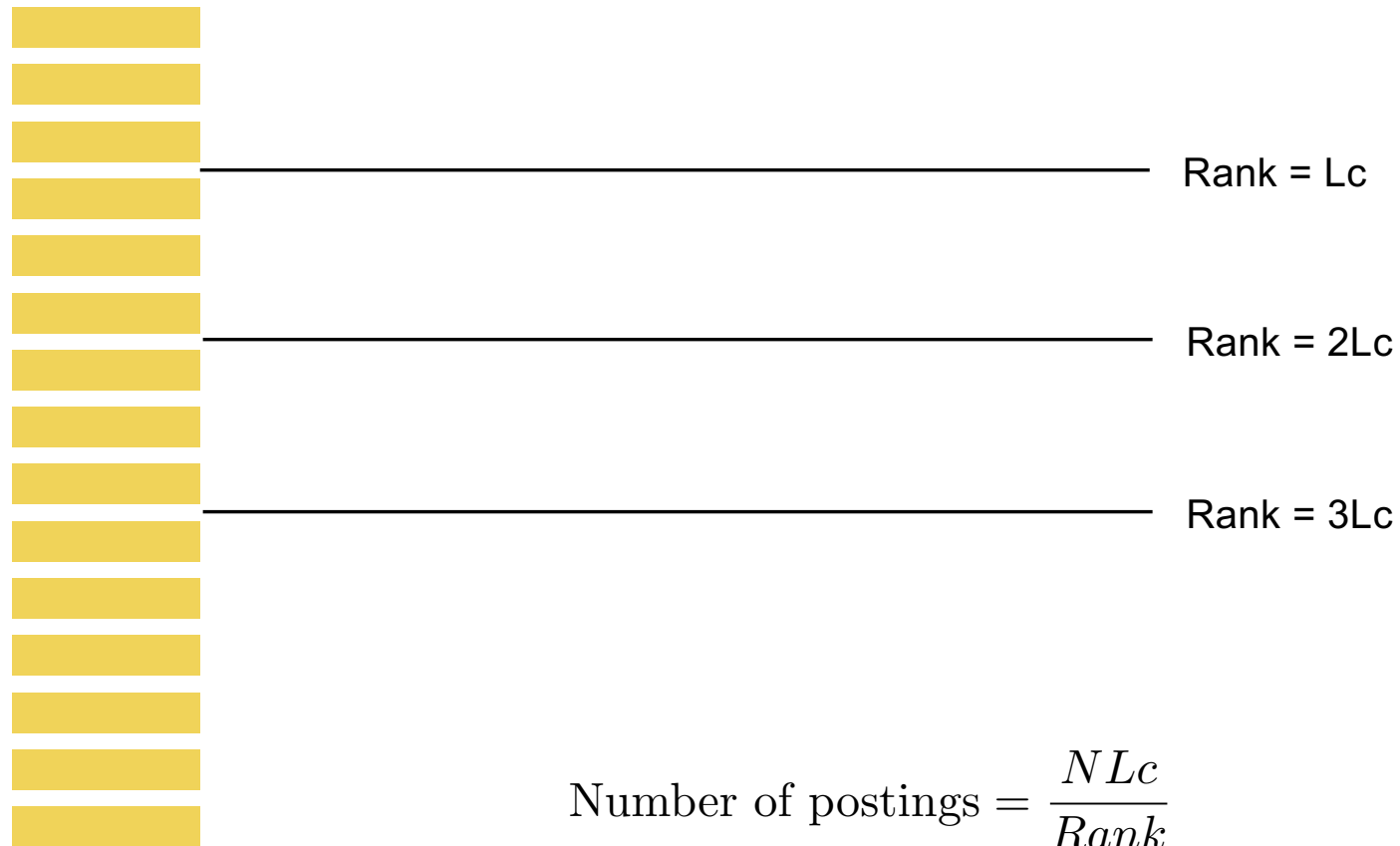
Zipf's law



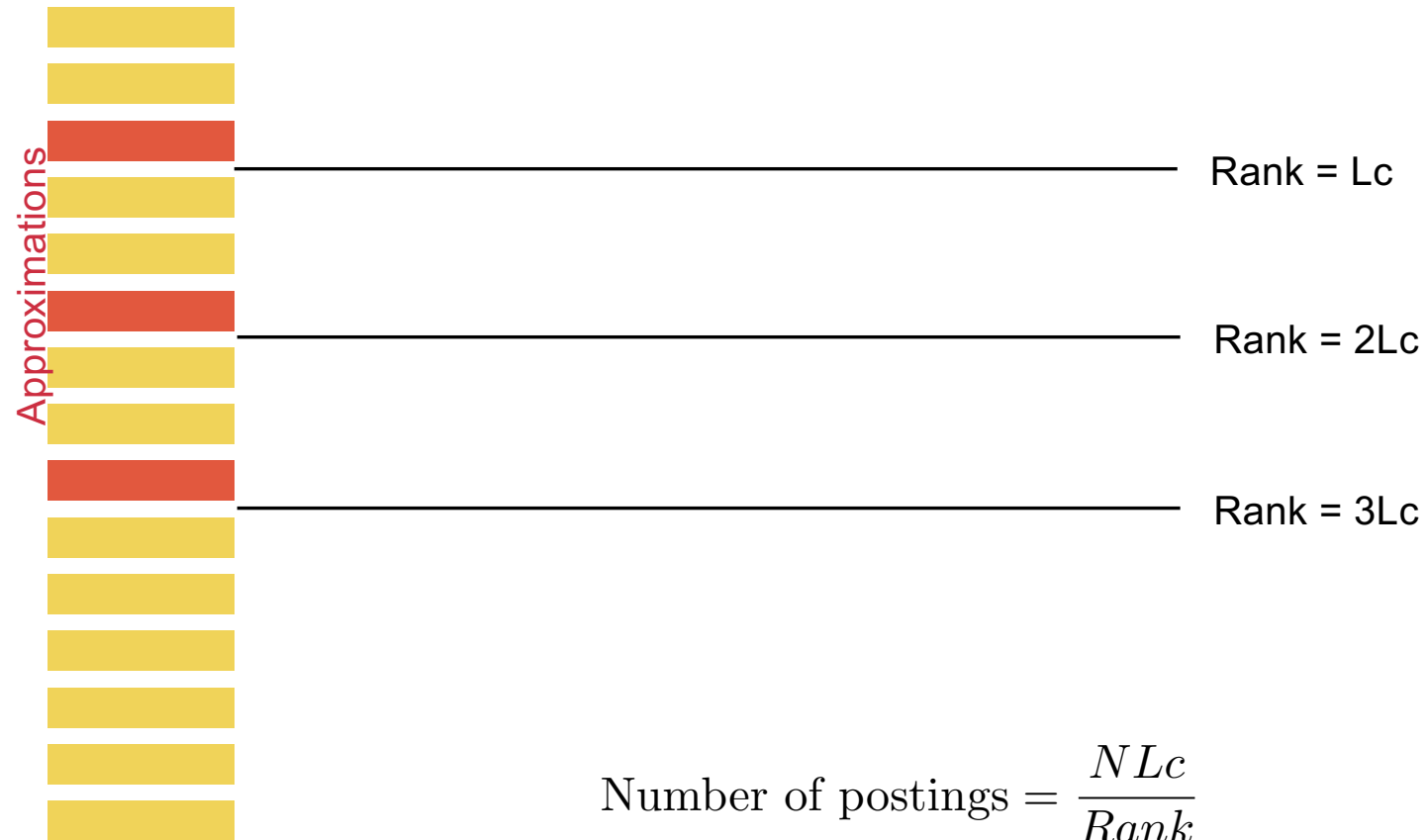
Blocks with L_c terms

$$\text{Number of postings} = \frac{NL_c}{Rank}$$

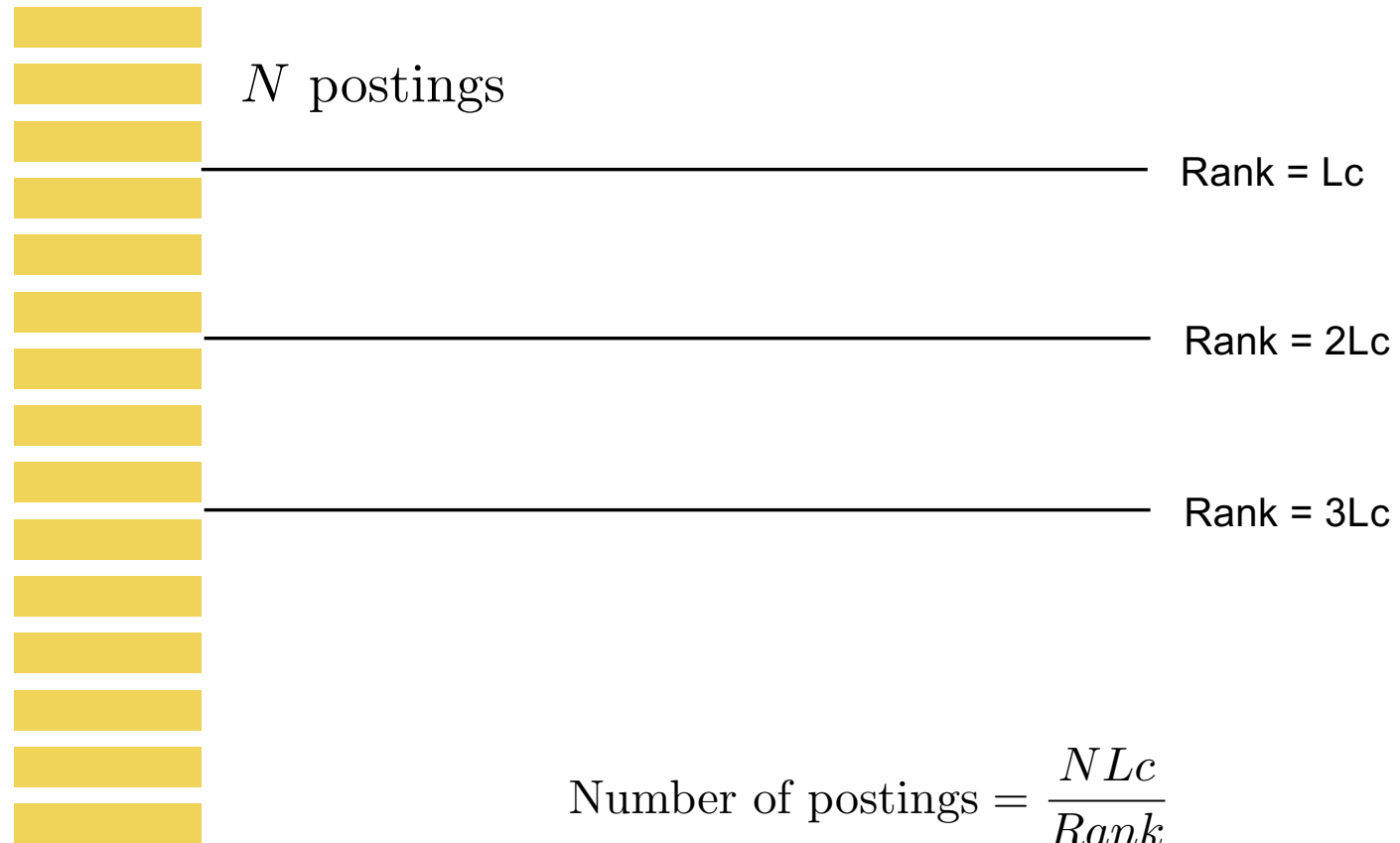
Zipf's law



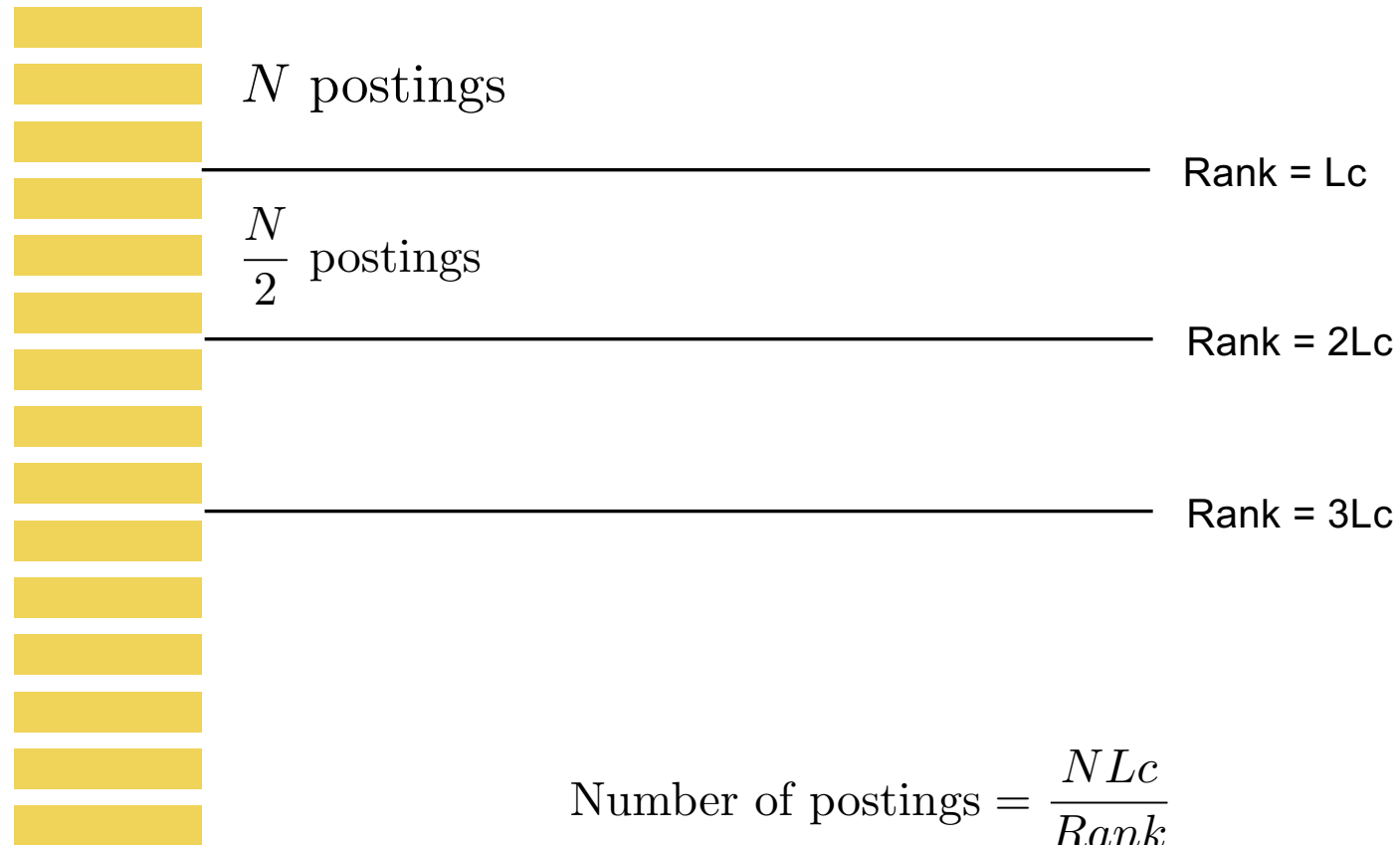
Zipf's law



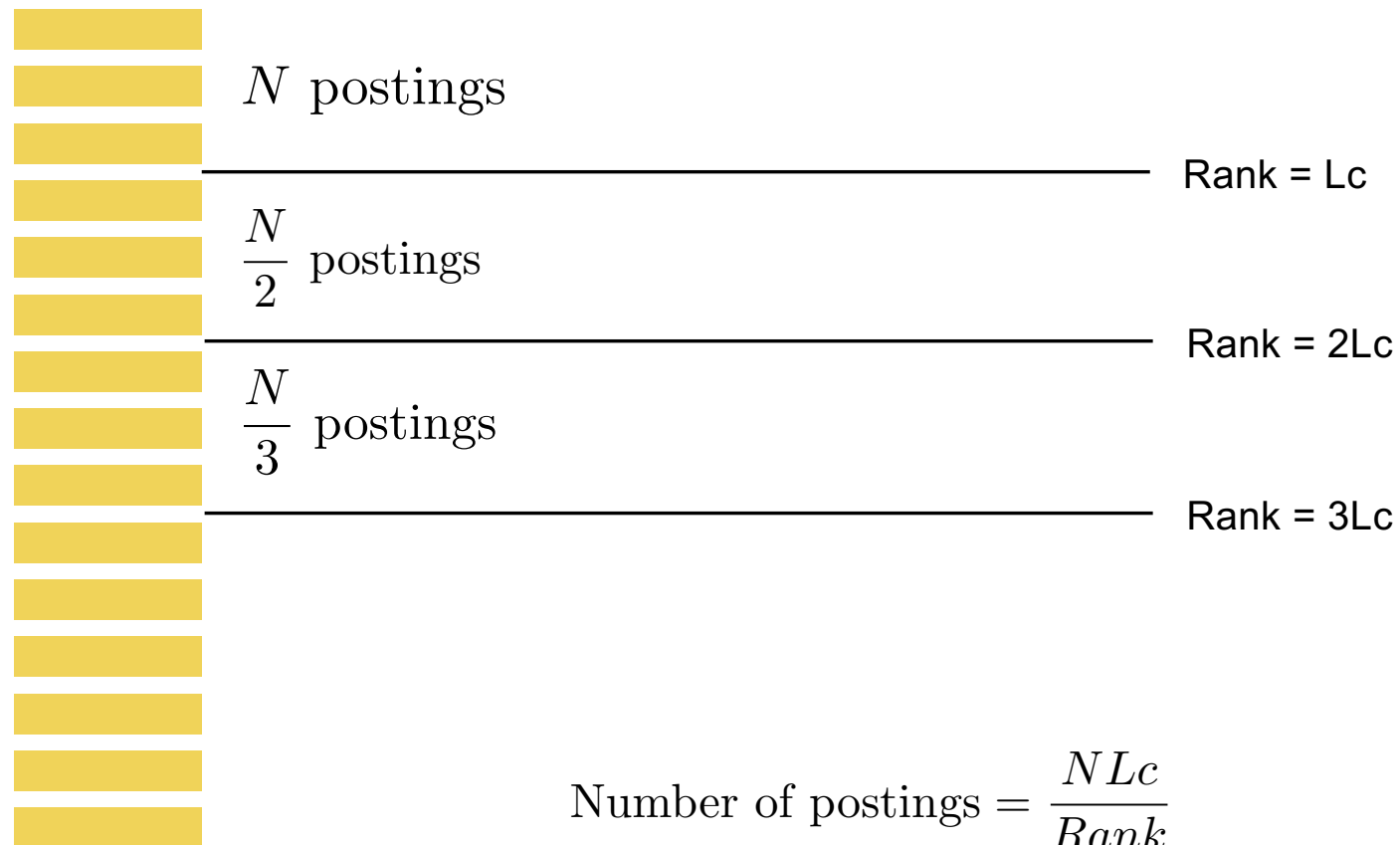
Zipf's law



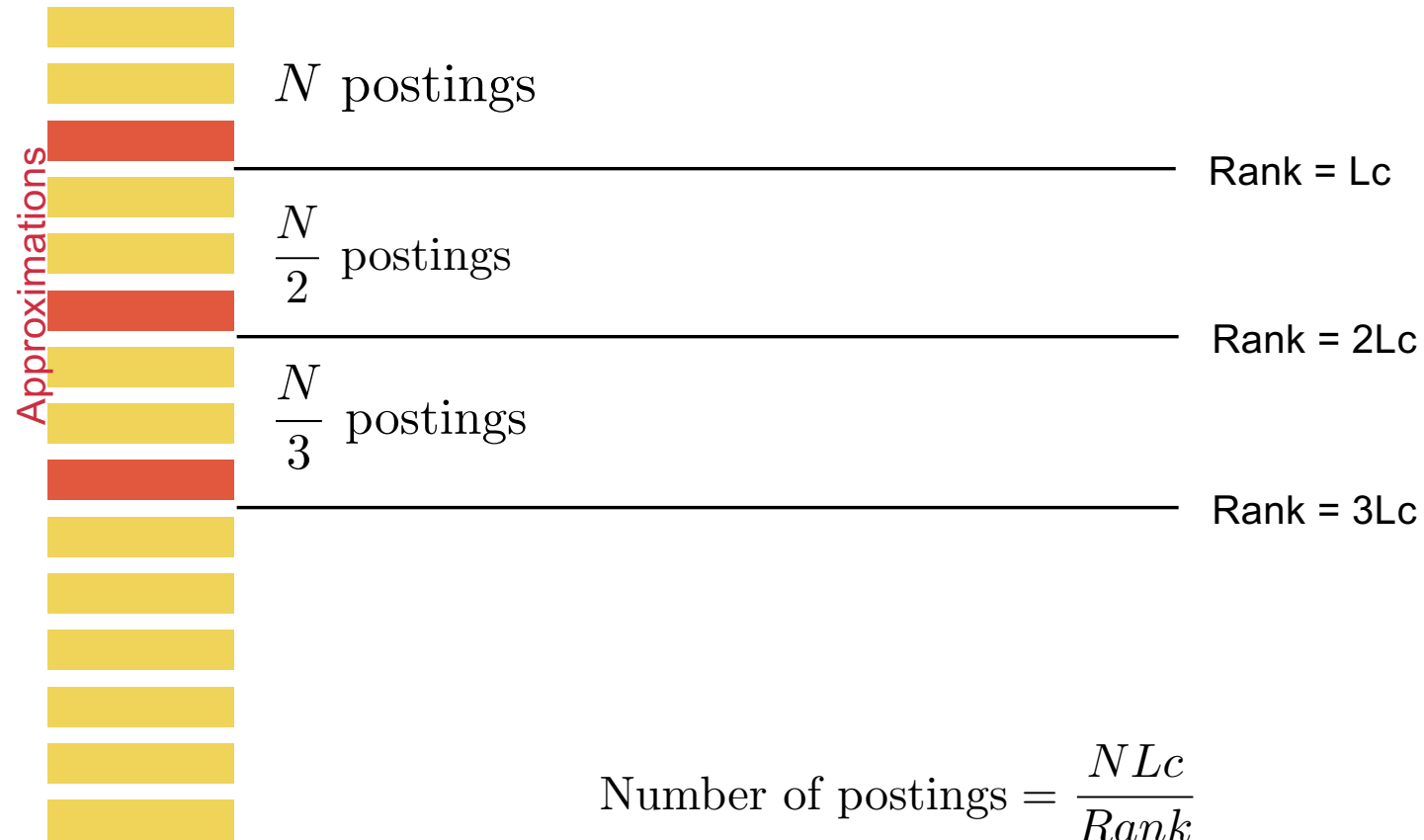
Zipf's law



Zipf's law



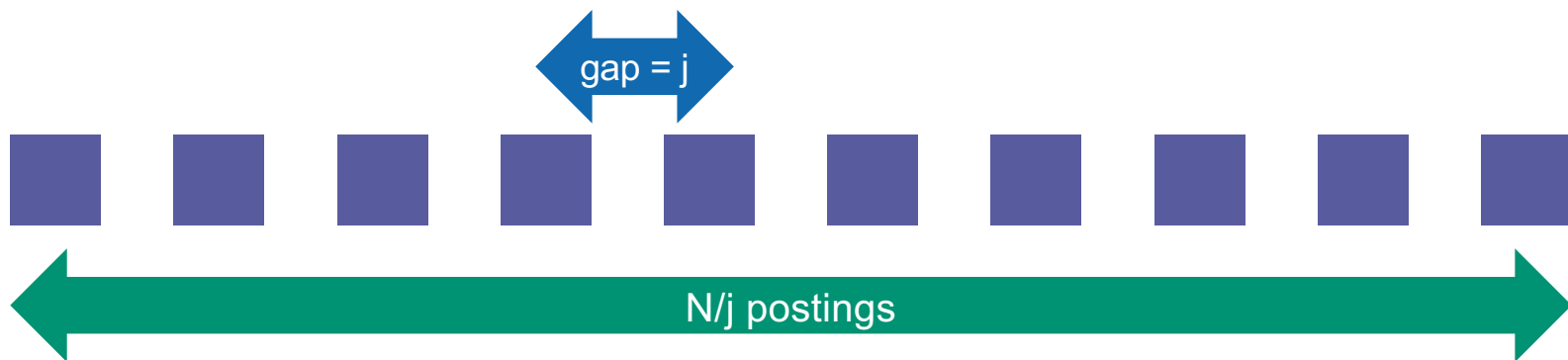
Zipf's law



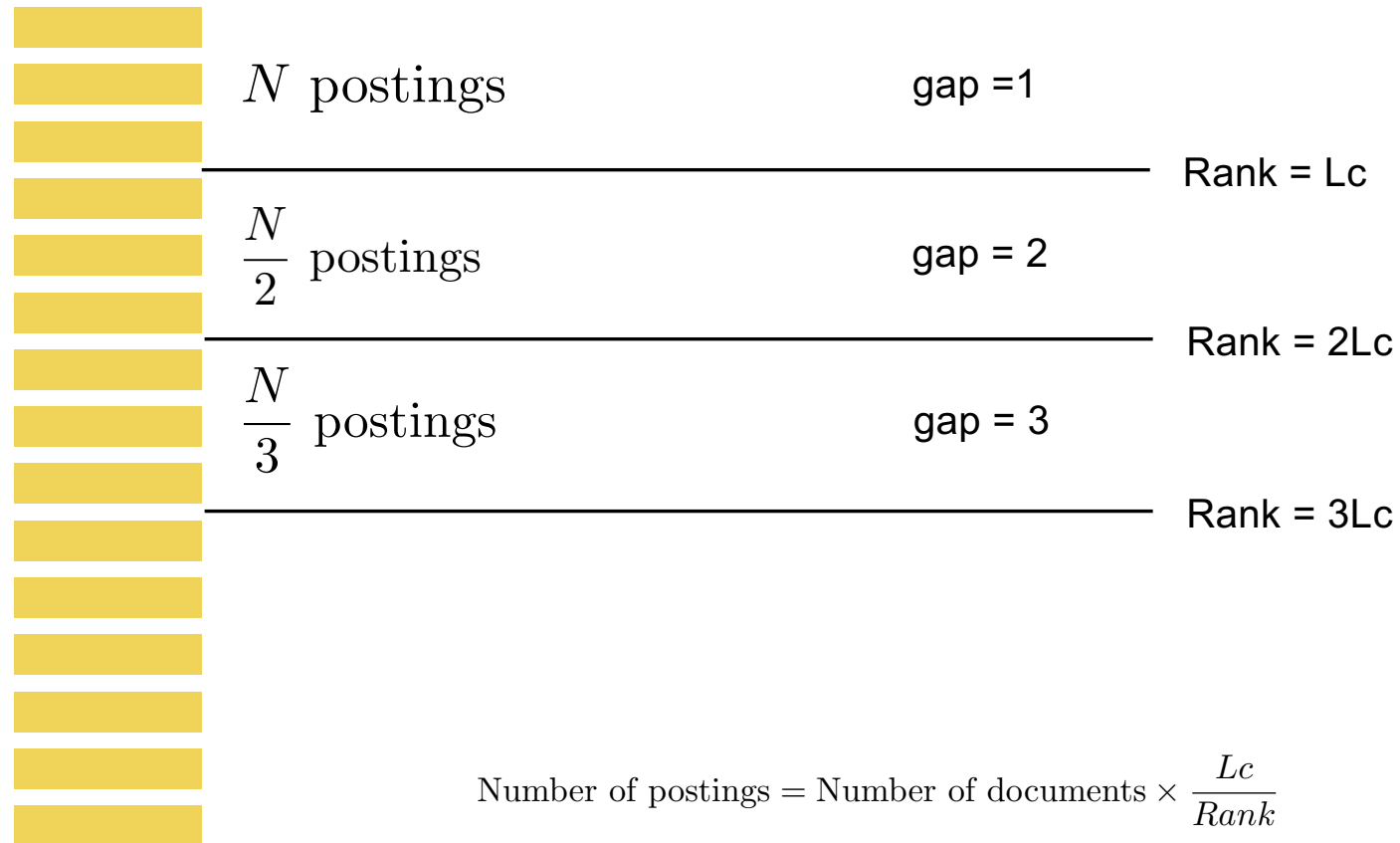
Zipf's law



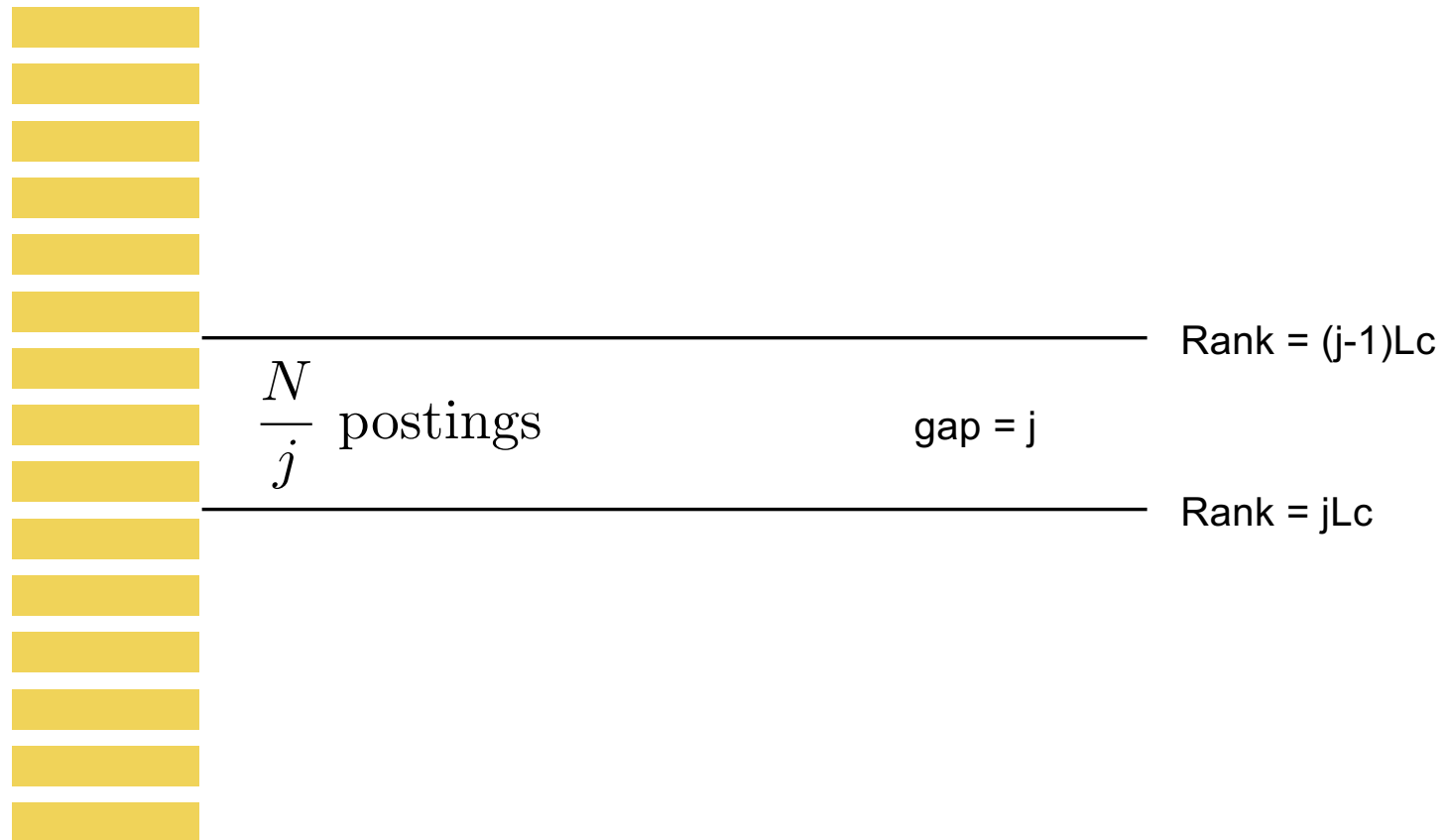
Zipf's law



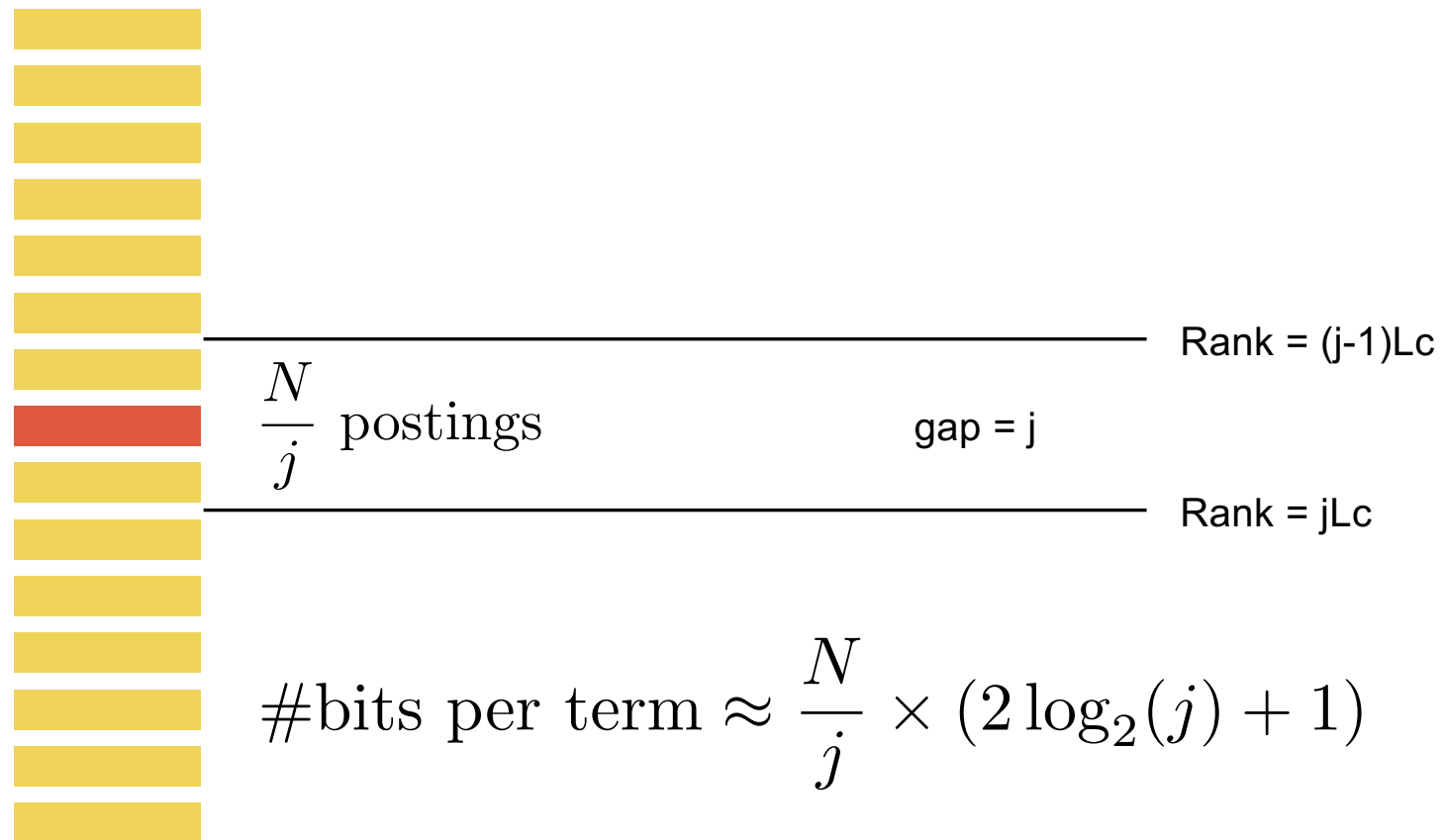
Zipf's law



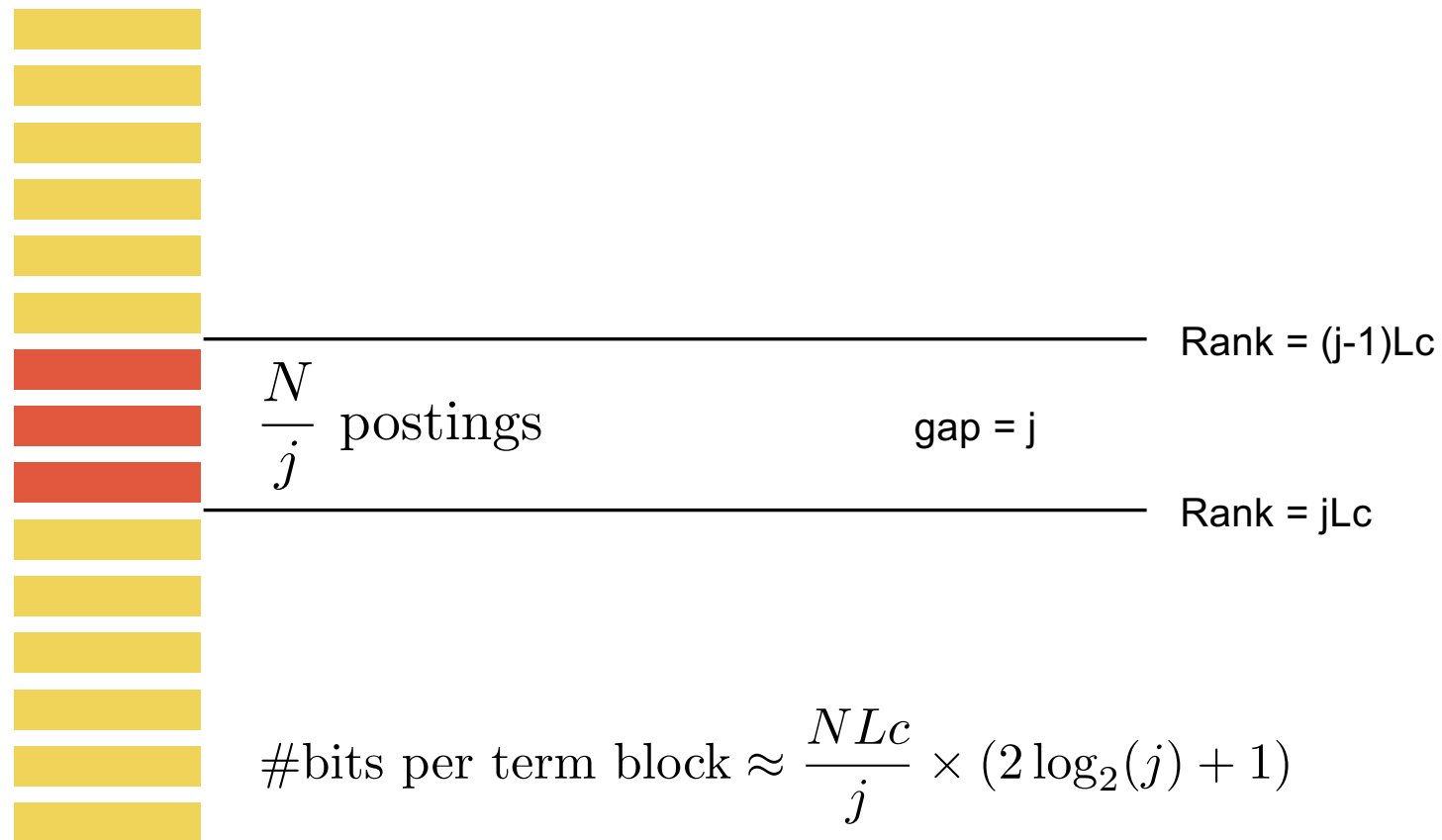
Zipf's law



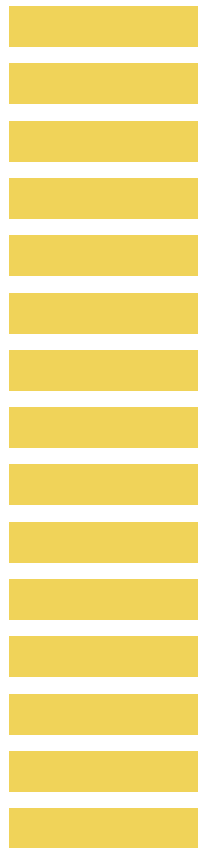
Zipf's law



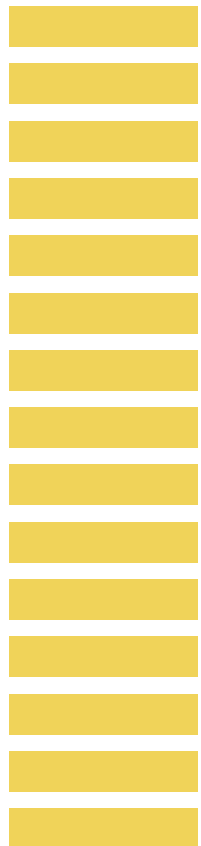
Zipf's law



Zipf's law


$$\# \text{bits} \approx \sum_{j=1}^{j=\frac{M}{L_c}} \frac{N L_c}{j} \times (2 \log_2(j) + 1)$$

Zipf's law


$$\# \text{bits} \approx \sum_{j=1}^{j=\frac{M}{L_c}} \frac{2NL_c \log_2(j)}{j}$$

How did we do?



Collection: 960 MB

How did we do?



Uncompressed on 32 bits

400 MB



Collection: 960 MB

How did we do?



Uncompressed on 32 bits

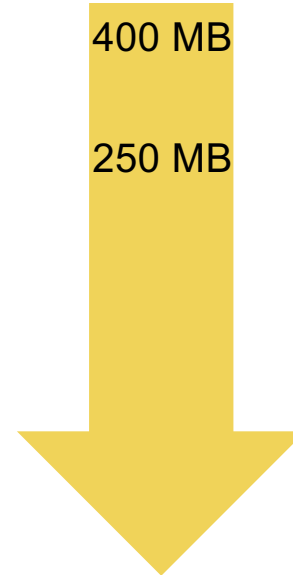
400 MB

Uncompressed on 20 bits

250 MB



Collection: 960 MB



How did we do?



Uncompressed on 32 bits

400 MB

Uncompressed on 20 bits

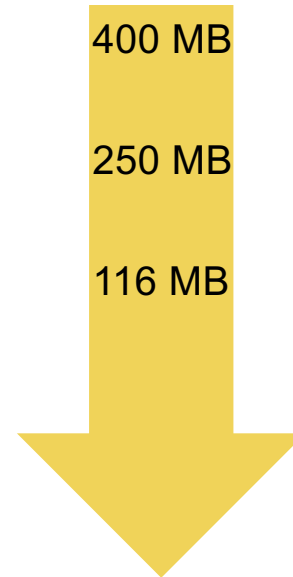
250 MB

Variable byte encoding (gaps)

116 MB



Collection: 960 MB



How did we do?



Uncompressed on 32 bits

400 MB

Uncompressed on 20 bits

250 MB

Variable byte encoding (gaps)

116 MB

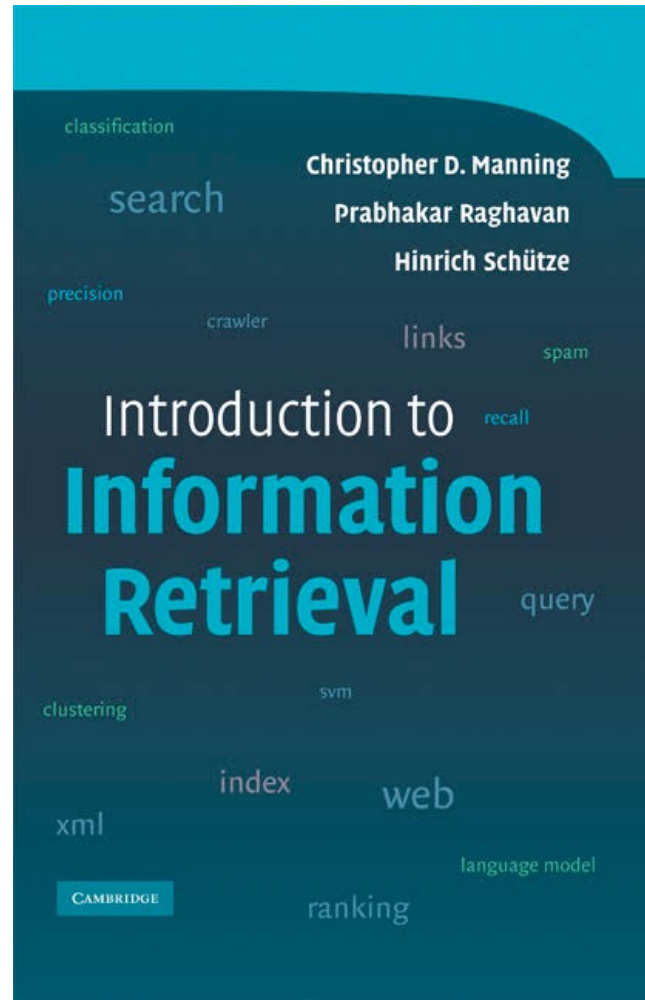
Elias γ encoding (gaps)

101 MB



Collection: 960 MB

Credits



This week:

Chapter 5

Index compression